

JProlog Reference Manual

Built-in predicates and operators, with examples

Version 4.2.0 - 26 August 2026

JProlog - a Prolog interpreter in Java
<https://github.com/DenzoSOFTHub/JProlog>
Generated from the source tree and the reference documentation.

Contents

Part I - Introduction	4
About this manual	4
Running JProlog	4
Part II - Language essentials	6
Terms	6
Clauses, programs and queries	6
Unification and control	7
Arithmetic	7
Definite clause grammars	7
Modules and tabling	7
Part III - Operators	8
Default operator table	8
Clause and directive operators	8
Control operators	9
Unification and comparison operators	10
Arithmetic operators (evaluable functors)	11
CLP(FD) operators	12
User-defined operators	12
Part IV - Built-in predicates	13
1. Type Checking Predicates	14
2. Term Manipulation	20
3. List Operations	27
4. Arithmetic Predicates	36
5. Control Flow	41
6. Meta-Predicates	47
7. Input/Output	53
8. Database Operations	63
9. Atom and String Operations	68
10. Character Processing	75
11. DCG (Grammar) Predicates	78
12. Exception Handling	83
13. System Predicates	86
14. Cryptographic Predicates	89
15. JSON Predicates	93
16. Date/Time Predicates	95
17. Filesystem Predicates	98
18. OS Predicates	102
19. Regex Predicates	105
20. XML Predicates	107
21. Threading Predicates	108
22. CSV Predicates	112
23. Logging Predicates	114
24. CLP(FD) Predicates	116
25. Tabling Predicates	120
26. HTTP Predicates	123

27. Persistence Predicates	126
28. Graph Algorithm Predicates	129
29. Java FFI Predicates	132
30. Concurrent Execution Predicates	135
31. Module System	137
32. String predicates	140
33. Stream predicates	143
34. Global variables	146
35. Attributed variables and coroutining	147
36. More list and pair predicates	149
37. Term inspection and modification	151
38. Clause inspection and operators	153
39. Debugging and profiling	154
40. System and memory	155
41. JDBC database access	156
42. TCP, UDP and DNS	158
43. DCG support predicates	160
Appendix A - Prolog flags	161
Appendix B - Engine configuration and embedding	162
Appendix C - ISO error terms	163
Appendix D - Standard order of terms and text conversion summary	164
Appendix E - Libraries not loaded by default	165
Index of predicates and operators	166

Part I - Introduction

About this manual

This manual is the complete reference to the **default** JProlog system: every operator in the initial operator table and every built-in predicate registered when a Prolog engine is created (the CLI, the IDE and the Java embedding API all start from the same registry). Each predicate entry gives its purpose, the argument modes, notes on behaviour that differs from other Prolog systems, and at least one runnable code example. Libraries that ship as source packages but are not loaded by default (expert-system, ASP, CHR, CLP(R), ... extensions) are listed in Appendix E and are not described here.

Conventions used throughout:

- `name/N` - a predicate with `N` arguments; `name//N` - a DCG non-terminal.
- Argument modes: `+Arg` must be instantiated, `-Arg` is an output (normally unbound), `?Arg` may be either, `@Arg` is not further instantiated by the call, `:Goal` is a callable meta-argument.
- Interactive examples show the query after the `?-` prompt followed by the answer as printed by the JProlog CLI: bindings such as `X = 3.`, `true.` for success without bindings, `false.` for failure, and `ERROR: ...` when an exception escapes to the top level. Alternative solutions are separated by `;`.
- Program examples are ordinary clauses you can put in a file and load with `consult/1` or `:consult`.
- Answers are written in the usual Prolog notation, which is what the JProlog console prints since v3.14.0 - except that this manual puts a space after a comma inside a compound term or a list (`[1, 2], f(a, b)`) for readability, where the console writes none (`[1, 2], f(a, b)`): bindings appear in quoted operator notation - `a-1`, `foo/1`, `(p, q)`, the atom `'42'` - and a variable that is still free prints as `_A`, `_B`, ... (a query variable that comes back unbound keeps its own name, so `X = f(Y)` prints as `X = f(Y)`). If an answer still carries constraints, they are listed after the bindings: `freeze(X, Goal), dif(X, a), X in 1..3`.

Running JProlog

JProlog is a Prolog interpreter written in Java (source level 1.8, runs on any modern JDK). It has no runtime dependencies beyond the JDK.

Command-line console - start it with `java -cp target/classes`

`it.denzosoft.jprolog.PrologCLI` (or run the jar). Queries are typed after the `?-` prompt and end with a full stop. The console also understands these commands:

Command	Effect
<code>:consult <file> (:c)</code>	Load facts and rules from a file
<code>:compile <file> (:cc)</code>	Compile a <code>.pl</code> file to the binary <code>.jpc</code> format
<code>:listing (:l)</code>	Show all loaded clauses
<code>:save <file> (:s)</code>	Save the current knowledge base to a file
<code>:clear</code>	Remove all user clauses
<code>:trace [on off]</code>	Toggle four-port call tracing (same as <code>trace.</code> / <code>notrace.</code>)
<code>:help (:h)</code>	Show the command list
<code>:quit (:q)</code>	Exit

When several solutions exist the console prints the first, writes `;` and waits: type `;` and Enter for the next one, or Enter alone to stop. **When stdin is not a terminal** - a pipe, a here-document, a redirected file, a CI job - or when the console is started with `--batch` or `-q`, it prints every solution at once instead, separated by `;` and terminated by `.`, and reads nothing back (before v4.0.0 it consumed the next input line as the answer to that prompt, silently swallowing the next query):

```
$ printf 'between(1, 3, X).\n:quit\n' | java -cp target/classes it.denzosoft.jprolog.PrologCLI
?- X = 1 ;
X = 2 ;
X = 3.
```

IDE - `java -cp target/classes it.denzosoft.jprolog.editor.PrologIDE` (or `mvn exec:java@run-ide`) opens the Swing IDE: a project tree, a multi-tab editor with syntax highlighting, completion and a source formatter, a Build tab with line-accurate compile diagnostics, a Run console for queries with streaming/table results, and a four-port debugger with breakpoints, stepping, variable and watch views.

Embedding in Java - the engine is the class `it.denzosoft.jprolog.core.engine.Prolog`:

```
import it.denzosoft.jprolog.core.engine.Prolog;
import it.denzosoft.jprolog.core.terms.Term;
import java.util.List;
import java.util.Map;

Prolog prolog = new Prolog();
prolog.consult("parent(tom, bob).\nparent(bob, ann).\n"
    + "grandparent(X, Z) :- parent(X, Y), parent(Y, Z).");

List<Map<String, Term>> solutions = prolog.solve("grandparent(tom, who)");
for (Map<String, Term> s : solutions) {
    System.out.println(s.get("Who"));          // ann
}

// Lazy, cancellable enumeration: the sink returns false to stop early
prolog.solveStream("between(1, inf, N)", sol -> ((Number) sol.get("N")).intValue() < 5);

// Hardening for untrusted programs (see Appendix B)
prolog.enableSafeMode();                      // remove OS/file/network/JDBC/FFI/thread built-ins
prolog.setInferenceBudget(1_000_000);         // abort runaway queries
```

`consultWithDiagnostics(source, fileName)` compiles clause by clause and returns every syntax error with its line number instead of stopping at the first one; `compileFile("prog.pl")` produces `prog.jpc`, which `consultSmart("prog.pl")` loads when it is newer than the source.

Part II - Language essentials

This part is a compact tour of the Prolog language as implemented by JProlog, enough to read the reference entries. JProlog follows ISO/IEC 13211-1 for syntax, the standard order of terms, arithmetic and error terms, and adds the SWI-Prolog extensions that most programs rely on (strings, global variables, corouting, CLP(FD), tabling, `format/2`, `must_be/2`, `pairs_*`, and the `apply` and `lists` libraries).

Terms

Every datum is a *term*:

- **Atoms** - symbolic constants: `hello`, `'New York'`, `[]`, `+`. Quote an atom when it does not start with a lowercase letter or contains characters other than letters, digits and `_`. Quoted atoms accept the ISO escapes `\n`, `\t`, `\\`, `\'`, `\xHH` and `\NNN` (octal).
- **Numbers** - integers of unbounded size (`42`, `-7`, `123456789012345678901234567890`, `0xFF`, `0o17`, `0b1010`, `0'a` for a character code) and IEEE double floats (`3.14`, `1.0e10`, `-2.5E-3`). An integer and a float are *different terms* even when numerically equal: `1 == 1.0` fails, `1 == 1.0` succeeds.
- **Strings** - `"text"` is a string object by default (`double_quotes` flag = `string`); set the flag to `codes`, `chars` or `atom` for the traditional readings. Back-quoted text `abc` is always a code list.
- **Variables** - start with an uppercase letter or `_`: `X`, `Result`, `_Tmp`. The anonymous variable `_` is a fresh variable at every occurrence.
- **Compound terms** - a functor with arguments: `point(1, 2)`, `person(Name, Age)`, `'-(a, b)`. Operators are syntactic sugar for compound terms: `1 + 2` is `+(1, 2)`, `X = Y` is `=(X, Y)`.
- **Lists** - `[1, 2, 3]` is sugar for `'.(1, '.(2, '.(3, []))'`; `[H|T]` splits head and tail; `[a, b|T]` is a partial list. Strings written in double quotes are not lists unless the `double_quotes` flag says so.
- **Curly terms** - `{a, b}` is `'{ }'((a, b))`, used by DCG and CLP(FD) syntax.

Comments run from `%` to the end of the line or between `/*` and `*/`.

Clauses, programs and queries

A program is a sequence of clauses terminated by `.` and whitespace:

```
% Facts: unconditionally true
parent(tom, bob).
parent(bob, ann).

% Rules: Head :- Body. The head is true when the body goals succeed in order.
grandparent(GP, GC) :-
    parent(GP, P),
    parent(P, GC).

% Directives are executed while loading
:- dynamic counter/1.
:- initialization(main).
```

Clauses for the same predicate (name/arity) should be contiguous; predicates that are modified at runtime with `assert/retract` are declared `dynamic`. A query is a goal (or conjunction of goals) evaluated against the program; Prolog searches clauses top to bottom, tries goals left to right, and *backtracks* to the most recent choice point on failure:

```
?- grandparent(tom, Who).
Who = ann.

?- parent(X, Y).
X = tom, Y = bob ;
X = bob, Y = ann.
```

Unification and control

- `=/2` unifies two terms, binding variables as needed; `\=/2` succeeds when they do not unify; `==/2` tests structural identity without binding.
- Conjunction `,`, disjunction `;`, if-then-else `(Cond -> Then ; Else)`, soft-cut `(Cond *-> Then ; Else)`, negation as failure `\+ Goal`, and the cut `!` (commits to the current clause and to the choices made so far in its body).
- `call/1..8` calls a goal built at runtime, adding extra arguments: `call(plus(1), 2, X)` gives `X = 3`. `findall/3`, `bagof/3`, `setof/3`, `forall/2` and `aggregate_all/3` collect or check all solutions of a goal. Lambda expressions of SWI-Prolog's library(`yall`) (`[X]>>Goal`) are **not** supported: pass a named auxiliary predicate to `maplist/2..5`, `include/3`, `foldl/4..6`,
- Exceptions: `throw(Ball)` unwinds to the nearest `catch(Goal, Catcher, Recovery)` whose `Catcher` unifies with the ball. Built-ins raise ISO error terms of the form `error(Formal, Context)`, e.g. `error(type_error(integer, abc), context(atom_length/2, _))` - see Appendix C.

```
max(X, Y, X) :- X >= Y, !.
max(_, Y, Y).

safe_div(X, Y, Z) :-
    catch(Z is X / Y, error(evaluation_error(zero_divisor), _), Z = undefined).

?- safe_div(1, 0, Z).
Z = undefined.
```

Arithmetic

Arithmetic is evaluated only by `is/2` and the comparison predicates `==`, `=\=`, `<`, `>`, `=<`, `>=`. Integers are exact and unbounded; `/` yields an integer when the division is exact and a float otherwise; `//`, `mod`, `rem`, `div` and the bitwise operators require integers. Every evaluable functor is listed in Part III (operators) and Chapter 4 (arithmetic predicates).

```
?- X is 2 ** 10, Y is 2 ^ 100, Z is 7 mod -2, W is max(3, 4.0).
X = 1024.0, Y = 1267650600228229401496703205376, Z = -1, W = 4.0.
```

Definite clause grammars

`Head --> Body` clauses are translated into ordinary predicates with two extra arguments (the difference list being parsed). Terminals are written as lists or strings, `{Goal}` embeds Prolog goals, `!`, `\+`, `->`, `;` and `call//N` are supported, and `phrase/2,3` runs a grammar:

```
greeting --> [hello], name.
name --> [world].
name --> [prolog].

?- phrase(greeting, [hello, prolog]).
true.
```

Modules and tabling

`:- module(Name, [pred/arity, ...])` starts a module; `:- use_module(Name)` imports its exports, and `Module:Goal` calls a predicate in a specific module. `:- table pred/arity` enables SLG tabling (memoisation with left-recursion support) for a predicate - see Chapter 25.

Part III - Operators

Operators let terms be written in infix, prefix or postfix notation. Each operator has a **priority** (1..1200; lower binds tighter) and a **type** that fixes its position and associativity: `xfx` (infix, non-associative), `xfy` (infix, right-associative), `yfx` (infix, left-associative), `fy/fx` (prefix), `yf/xf` (postfix). In a type, `x` marks an argument whose priority must be strictly lower than the operator's, `y` an argument whose priority may be equal. Parentheses always override precedence, and an operator written in canonical form - '+' (1, 2) - is an ordinary compound term. New operators are declared with `op/3`; `current_op/3` enumerates the active table.

Default operator table

Priority	Type	Operators	Meaning
1200	xfx	<code>:- --></code>	Clause, DCG rule
1200	fx	<code>:- ?-</code>	Directive, query
1150	fx	<code>dynamic discontiguous multifile table meta_predicate module_transparent</code>	Declarations
1100	xfy	<code>;</code>	Disjunction
1050	xfy	<code>-> *-></code>	If-then, soft-cut
1000	xfy	<code>,</code>	Conjunction
900	fy	<code>\+</code>	Negation as failure
700	xfx	<code>= \= == \== @< @=< @> @>= =.. is := =\= < >=</code>	Unification, comparison, arithmetic
700	xfx	<code>in ins #= #\= #< #> #=< #>=</code>	CLP(FD) constraints (<code>ins</code> is reserved: parsed, not implemented)
600	xfy	<code>:</code>	Module qualification
500	yfx	<code>+ - /\ \ / xor</code>	Additive and bitwise
450	xfx	<code>..</code>	CLP(FD) domain range
400	yfx	<code>* / // rem mod div rdiv << >></code>	Multiplicative and shifts
200	xfx	<code>**</code>	Float exponentiation
200	xfy	<code>^</code>	Integer power / existential quantifier
200	fy	<code>- + \</code>	Unary minus, plus, bitwise complement

The list constructor `|` and the curly braces `{}` are part of the term syntax rather than operators.

Clause and directive operators

`:-/2, :-/1, ?-/1, -->/2`

Head `:-` Body is a rule; Head `:-` Goal is a directive executed at load time; Head `?-` Goal is an interactive query (also accepted in files); Head `-->` Body is a DCG rule.

```
:- dynamic visited/1.
:- initialization((write('loaded'), nl)).

double(X, Y) :- Y is X * 2.
digits --> [D], { code_type(D, digit) }, digits.
digits --> [].
```

dynamic/1, discontinuous/1, multifile/1, table/1, meta_predicate/1, module_transparent/1

Prefix declaration operators; they take a predicate indicator or a comma/list of indicators.

```
:- dynamic stock/2, price/2.
:- discontinuous rule/2.
:- table fib/2.
:- meta_predicate twice(0).

twice(G) :- call(G), call(G).
```

Control operators

,/2 - conjunction

Both goals must succeed, left to right; bindings made by the left goal are visible to the right one.

```
?- X = 3, Y is X * 2.
X = 3, Y = 6.
```

;/2 - disjunction

Succeeds if either branch succeeds; on backtracking the second branch is tried.

```
?- (X = 1 ; X = 2).
X = 1 ;
X = 2.
```

->/2 and *->/2 - if-then(-else) and soft-cut

(C -> T ; E) runs T after the *first* solution of C, otherwise E; without an else branch it fails when C fails. (C *-> T ; E) runs T for every solution of C and E only if C has none. A cut inside C is local to the condition.

```
sign(X, S) :- ( X > 0 -> S = positive ; X < 0 -> S = negative ; S = zero ).

?- sign(-4, S).
S = negative.

?- (member(X, [1,2]) *-> true ; X = none).
X = 1 ;
X = 2.
```

\+/1 - negation as failure

\+ G succeeds iff G has no solution; it never binds variables.

```
?- \+ member(4, [1,2,3]).
true.
```

!/0 - cut

Discards the choice points created since the clause was entered, including the remaining clauses of the predicate. call/1, \+, findall/3 and the condition of -> are opaque to cut.

```
first_even([X|_], X) :- 0 is X mod 2, !.
first_even([_|T], X) :- first_even(T, X).
```

Unification and comparison operators

`=/2` and `\=/2`

```
?- f(X, b) = f(a, Y).
X = a, Y = b.

?- f(X) \= g(X).
true.
```

`==/2`, `\==/2`, `@</2`, `@=</2`, `@>/2`, `@>=/2` - identity and standard order

Structural comparison without unification. The standard order of terms is Var < Number < Atom < String < Compound; numbers compare by value (a float precedes an equal integer), atoms and strings alphabetically, compound terms by arity, then name, then arguments.

```
?- X == Y.
false.

?- 1.0 @< 1, a @< b, f(a) @> z.
true.

?- msort([b, 2, a, 1.5, f(x), "s"], L).
L = [1.5, 2, a, b, "s", f(x)].
```

`=../2` - univ

Converts between a term and the list [Functor|Args].

```
?- foo(a, b) =.. L.
L = [foo, a, b].

?- T =.. [point, 1, 2].
T = point(1, 2).
```

`is/2` and the arithmetic comparisons `:=/2`, `=\=/2`, `</2`, `=</2`, `>/2`, `>=/2`

`X is Expr` evaluates `Expr`; the comparisons evaluate both sides.

```
?- X is (3 + 4) * 2, 10 := 10.0, 2 =\= 3, 1 < 2.
X = 14.
```

`:/2` - module qualification

Calls a predicate defined in a named module (see *Modules and tabling*). Built-in predicates are global and cannot be qualified with a library name (`lists:append(...)` fails).

```
:- module(shapes, [area/2]).
area(square(S), A) :- A is S * S.
area(circle(R), A) :- A is pi * R * R.

?- shapes:area(square(3), A).
A = 9.
```

Arithmetic operators (evaluable functors)

All of these are evaluated inside `is/2` and the arithmetic comparisons.

`+/2`, `-/2`, `*/2` - exact on integers, float otherwise

```
?- X is 2 + 3 * 4, Y is 2.5 * 2, Z is 10 - 12.
X = 14, Y = 5.0, Z = -2.
```

`//2`, `///2`, `div/2`, `mod/2`, `rem/2`, `rdiv/2`

`/` is exact division (integer result when it divides evenly, float otherwise); `//` truncates toward zero; `div` floors; `mod` takes the sign of the divisor, `rem` the sign of the dividend; `rdiv` builds a rational number.

```
?- A is 7 / 2, B is 8 / 2, C is -7 // 2, D is -7 div 2, E is -7 mod 2, F is -7 rem 2.
A = 3.5, B = 4, C = -3, D = -4, E = 1, F = -1.
```

`**/2` and `^/2` - exponentiation

`**` always returns a float; `^` is the integer power when both operands are integers (an error for a negative exponent unless the base is 1 or -1) and a float otherwise.

```
?- X is 2 ** 3, Y is 2 ^ 3, Z is 2 ^ 200, W is 2.0 ^ 0.5.
X = 8.0, Y = 8, Z = 1606938044258990275541962092341162602522202993782792835301376, W = 1.4142135623730951.
```

Unary `-/1`, `+/1` and `V/1`

```
?- X is - (3), Y is \ 5, Z is +(7).
X = -3, Y = -6, Z = 7.
```

`/2`, `V/2`, `xor/2`, `<</2`, `>>/2` - bitwise and shifts

```
?- A is 12 /\ 10, B is 12 \/ 10, C is 12 xor 10, D is 1 << 10, E is 1024 >> 3.
A = 8, B = 14, C = 6, D = 1024, E = 128.
```

`^/2` as existential quantifier

Inside `bagof/3` and `setof/3`, `Var^Goal` excludes `Var` from the free variables that split the result into groups. Elsewhere `V^G` simply calls `G`.

```
age(ann, 30). age(bob, 25). age(cid, 30).

?- setof(Name, Age^age(Name, Age), Names).
Names = [ann, bob, cid].

?- bagof(Name, age(Name, Age), Names).
Age = 25, Names = [bob] ;
Age = 30, Names = [ann, cid].
```

CLP(FD) operators

in/2, ins/2, ..I/2, #=/2, #\=/2, #</2, #>/2, #=</2, #>=/2

Constraint syntax for the finite-domain solver (Chapter 24): `X in 1..9` restricts a domain and the `#` -comparisons post arithmetic constraints that are propagated before and during `label/1`. `ins/2` is present in the operator table for compatibility but is not implemented as a predicate: restrict list elements with `in/2` (or `maplist/2` with an auxiliary predicate).

```
?- X in 1..10, Y in 1..10, X + Y #= 12, X - Y #= 4, label([X, Y]).
X = 8, Y = 4.

?- A in 0..1, B in 0..1, C in 0..1, A + B + C #= 2, A #\= B, label([A, B, C]).
A = 0, B = 1, C = 1 ;
A = 1, B = 0, C = 1.
```

User-defined operators

`op(+Priority, +Type, +Name)` adds (or, with priority 0, removes) an operator; `Name` may be a list. Definitions made in a file apply from that point on and are stored with compiled `.jpc` files. A query is parsed as a whole before it runs, so an operator must be declared in an earlier directive or query before it can be used in operator syntax.

```
:- op(700, xfx, ==>).
:- op(200, xfy, likes).

rule(rain ==> wet).
fact(mary likes wine).

?- rule(A ==> B), X = (john likes (mary likes wine)).
A = rain, B = wet, X = john likes mary likes wine.

?- op(650, xfx, is_bigger), current_op(P, T, is_bigger).
P = 650, T = xfx.
```

Since v3.14.0 there is a single operator store per engine, so `current_op/3` reports operators declared with `op/3` at the top level **and** those declared by `:- op/3` directives while consulting a file. An `op/3` executed while a module file is being consulted is local to that module for `current_op/3`; it is still honoured by the parser everywhere in the session.

Part IV - Built-in predicates

The chapters that follow describe every registered built-in predicate, grouped by function. The first thirteen chapters cover the ISO core and the classic library; the remaining chapters cover the extended libraries that JProlog loads by default (cryptography, JSON, date/time, filesystem, OS, regular expressions, XML, threads, CSV, logging, CLP(FD), tabling, HTTP, persistence, graphs, concurrency, Java FFI, the module system, JDBC, networking, strings, streams, global variables, coroutining and profiling).

1. Type Checking Predicates

Type checking predicates are used to determine the type of a term or to ensure a term is of a specific type. These are essential for writing robust Prolog programs that handle different kinds of data correctly.

Understanding Prolog Types

In Prolog, every piece of data is a **term**. Terms can be:

- **Variables:** Uninstantiated placeholders (e.g., `X`, `_Value`)
- **Atoms:** Constant symbols (e.g., `hello`, `'New York'`)
- **Numbers:** Integers or floats (e.g., `42`, `3.14`)
- **Compound Terms:** Structures with a functor and arguments (e.g., `person(john, 25)`)
- **Lists:** Special compound terms (e.g., `[1, 2, 3]`)

var/1

Purpose: Checks if a term is an uninstantiated variable.

When to use: Use this when you need to handle unbound variables differently from bound values, often in conditional logic or to prevent errors.

```
% Check if a variable is unbound before processing
process_value(X) :-
    var(X),
    !,
    write('Error: Variable must be instantiated'),
    fail.
process_value(X) :-
    write('Processing: '), write(X).

% Example usage:
?- process_value(X).
Error: Variable must be instantiated
false.

?- process_value(42).
Processing: 42
true.
```

nonvar/1

Purpose: Checks if a term is NOT a variable (i.e., it's instantiated).

When to use: Use this to ensure a value has been provided before performing operations on it.

```
% Safe division that checks arguments are instantiated
safe_divide(X, Y, Result) :-
    nonvar(X),
    nonvar(Y),
    Y \= 0,
    Result is X / Y.

% Example usage:
?- safe_divide(10, 2, R).
R = 5.0.

?- safe_divide(X, 2, R).
false. % Fails because X is not instantiated
```

atom/1

Purpose: Checks if a term is an atom (a constant symbol).

When to use: Use when you need to verify that input is a symbolic constant, not a number or structure.

```
% A predicate that processes color names
set_color(Color) :-
    atom(Color),
    member(Color, [red, green, blue, yellow]),
    !,
    write('Color set to: '), write(Color).
set_color(_) :-
    write('Invalid color - must be an atom: red, green, blue, or yellow').

% Example usage:
?- set_color(red).
Color set to: red
true.

?- set_color(123).
Invalid color - must be an atom: red, green, blue, or yellow
true.
```

number/1

Purpose: Checks if a term is a number (integer or float).

When to use: Use before arithmetic operations to ensure the term can be used in calculations.

```
% Calculate the square of a number with type checking
square(X, Result) :-
    number(X),
    Result is X * X.
square(X, _) :-
    \+ number(X),
    write('Error: '), write(X), write(' is not a number'), nl,
    fail.

% Example usage:
?- square(5, R).
R = 25.

?- square(hello, R).
Error: hello is not a number
false.
```

integer/1

Purpose: Checks if a term is specifically an integer (whole number).

When to use: Use when you need whole numbers only, such as for array indices or counting.

```
% Get the Nth element of a list (N must be an integer)
get_nth(N, List, Element) :-
    integer(N),
    N > 0,
    nth1(N, List, Element).
get_nth(N, _, _) :-
    \+ integer(N),
    write('Error: Position must be a positive integer'), nl,
    fail.
```

```
% Example usage:
?- get_nth(2, [a, b, c], E).
E = b.

?- get_nth(2.5, [a, b, c], E).
Error: Position must be a positive integer
false.
```

float/1

Purpose: Checks if a term is a floating-point number.

When to use: Use when you need to distinguish between exact integers and decimal numbers.

```
% Format a number for display based on its type
format_number(N, Formatted) :-
    integer(N),
    !,
    atom_number(Formatted, N).
format_number(N, Formatted) :-
    float(N),
    !,
    format(atom(Formatted), '~2f', [N]).

% Example usage:
?- format_number(42, F).
F = '42'.

?- format_number(3.14159, F).
F = '3.14'.
```

atomic/1

Purpose: Checks if a term is atomic (atom or number).

When to use: Use when you need simple, non-compound values.

```
% Store simple values in a database
store_simple_value(Key, Value) :-
    atomic(Value),
    !,
    assertz(stored(Key, Value)).
store_simple_value(Key, Value) :-
    write('Cannot store complex term: '), write(Value), nl,
    fail.

% Example usage:
?- store_simple_value(age, 25).
true.

?- store_simple_value(name, 'John').
true.

?- store_simple_value(person, person(john, 25)).
Cannot store complex term: person(john, 25)
false.
```

string/1

Purpose: Checks if a term is a string (a "..." text object, distinct from atoms and character/code lists). (v3.6.0)

When to use: Use to distinguish string terms from atoms when handling text that may arrive in either

representation. With the default flag `double_quotes=string`, `"abc"` denotes a string term.

```
?- string("hello").
true.

?- string(hello).
false.    % atom, not a string

?- string(123).
false.

?- string(X).
false.    % unbound variable

% Dispatch on the text representation
text_length(T, L) :- string(T), !, string_length(T, L).
text_length(T, L) :- atom(T), atom_length(T, L).
```

compound/1

Purpose: Checks if a term is a compound structure (has a functor and arguments).

When to use: Use to identify complex data structures.

```
% Process different types of data differently
process_data(Data) :-
    atomic(Data),
    !,
    write('Simple value: '), write(Data).
process_data(Data) :-
    compound(Data),
    Data =.. [Functor|Args],
    write('Complex structure: '), write(Functor),
    write(' with '), length(Args, N), write(N), write(' arguments').

% Example usage:
?- process_data(42).
Simple value: 42
true.

?- process_data(person(john, 25, london)).
Complex structure: person with 3 arguments
true.
```

callable/1

Purpose: Checks if a term can be called as a goal (atom or compound).

When to use: Use when you need to verify that a term can be executed as a Prolog goal.

```
% Execute a goal safely
safe_call(Goal) :-
    callable(Goal),
    !,
    catch(call(Goal), Error, (write('Error: '), write(Error), nl, fail)).
safe_call(Goal) :-
    write('Not callable: '), write(Goal), nl,
    fail.

% Example usage:
?- safe_call(append([1], [2], X)).
X = [1, 2].

?- safe_call(123).
```

```
Not callable: 123
false.
```

ground/1

Purpose: Checks if a term is fully instantiated (contains no variables).

When to use: Use to ensure all data is complete before processing or storing.

```
% Only store fully instantiated facts
add_fact(Fact) :-
    ground(Fact),
    !,
    assertz(Fact),
    write('Fact added: '), write(Fact).
add_fact(Fact) :-
    write('Cannot add fact with variables: '), write(Fact), nl,
    fail.

% Example usage:
?- add_fact(parent(john, mary)).
Fact added: parent(john, mary)
true.

?- add_fact(parent(X, mary)).
Cannot add fact with variables: parent(_G123, mary)
false.
```

is_list/1

Purpose: Checks if a term is a proper list.

When to use: Use before list operations to ensure the data structure is valid.

```
% Sum all numbers in a list
sum_list(List, Sum) :-
    is_list(List),
    sum_list_helper(List, 0, Sum).
sum_list(NotList, _) :-
    \+ is_list(NotList),
    write('Error: Expected a list, got: '), write(NotList), nl,
    fail.

sum_list_helper([], Acc, Acc).
sum_list_helper([H|T], Acc, Sum) :-
    number(H),
    NewAcc is Acc + H,
    sum_list_helper(T, NewAcc, Sum).

% Example usage:
?- sum_list([1, 2, 3, 4], Sum).
Sum = 10.

?- sum_list(not_a_list, Sum).
Error: Expected a list, got: not_a_list
false.
```

<!-- START_CHANGE: ISS-2025-0179 - Add acyclic_term/1 and proper_list/1 -->

acyclic_term/1

Purpose: Checks if a term is acyclic (contains no circular references).

When to use: Use to verify term safety before operations that would loop on cyclic terms.

v3.9.0 (ISS-2025-0441): on the opt-in **v4 engine** (-Djprolog.engine=v4) this is a real test over rational trees - $X = f(X)$, `acyclic_term(X)` fails and `cyclic_term(X)` succeeds - because v4 supports cyclic terms instead of raising `representation_error(cyclic_term)`. On the default engine a cyclic term cannot normally be built, so the predicate is effectively always true.

```
% Syntax: acyclic_term(+Term)
?- acyclic_term(f(a, b)).
true.
```

cyclic_term/1

Purpose: Succeeds when Term is a rational (cyclic) tree; the complement of `acyclic_term/1`.

When to use: Guard code that walks a term structurally before handing it to something that cannot represent cycles (a writer, a serialiser, an external API).

Availability: added with engine v4 (v3.9.0). The engines deleted in 4.0.0/4.1.0 could not construct cyclic terms and did not register this predicate.

```
% Syntax: cyclic_term(+Term)
?- X = f(X), cyclic_term(X).
true.

?- cyclic_term(f(a, g(b))).
false.
```

proper_list/1

Purpose: Checks if a term is a proper list (terminates with []).

When to use: Use to distinguish proper lists from partial lists or non-list terms.

```
% Syntax: proper_list(+Term)
?- proper_list([1, 2, 3]).
true.

?- proper_list([1|2]).
false.
```

<!-- END_CHANGE: ISS-2025-0179 -->

2. Term Manipulation

Term manipulation predicates allow you to inspect and construct complex terms. These are essential for meta-programming and working with structured data.

functor/3

Purpose: Relates a compound term to its name and arity (number of arguments).

When to use: Use to inspect the structure of compound terms or create new terms with a specific structure.

```
% Example: Extract information about a term's structure
analyze_term(Term) :-
    functor(Term, Name, Arity),
    write('Functor: '), write(Name),
    write(', Arity: '), write(Arity), nl.

% Example usage:
?- analyze_term(person(john, 25, london)).
Functor: person, Arity: 3
true.

% Create a term with specific structure
?- functor(Term, employee, 4).
Term = employee(_G1, _G2, _G3, _G4).

% Practical example: Generic predicate to check term type
is_person_record(Term) :-
    functor(Term, person, 3).

?- is_person_record(person(john, 25, london)).
true.

?- is_person_record(employee(john, 25, london, 50000)).
false.
```

v4.2.0: a **non-ground compound is decomposed** (ISO 8.5.1): `functor(f(X, b), N, A)` gives $N = f$, $A = 2$. It used to raise `instantiation_error`, because the mode was chosen with a groundness test rather than a variable test. Construct mode makes fresh unnamed cells.

arg/3

Purpose: Extracts or checks a specific argument from a compound term.

When to use: Use to access fields in structured data without pattern matching.

v3.6.0: full ISO error clauses (8.5.2.3) - an unbound term raises `instantiation_error`, a non-integer index raises `type_error(integer, N)`, a negative index raises `domain_error(not_less_than_zero, N)`, a non-compound term raises `type_error(compound, T)`; an out-of-range index still just fails. Also works on non-ground compounds: `arg(1, f(X), A)` gives $A = X$ (previously failed).

v4.2.0: an **unbound index enumerates** the arguments (ISO 8.5.2), lazily - one per redo, so `once/1` stops it at the first. It used to raise `instantiation_error`.

```
?- arg(N, f(a, b), X).
N = 1, X = a ;
N = 2, X = b.
```

```
% Example: Database of person records
% person(Name, Age, City)
```

```
% Get the age of a person (2nd argument)
get_age(PersonRecord, Age) :-
    compound(PersonRecord),
    functor(PersonRecord, person, 3),
    arg(2, PersonRecord, Age).

% Example usage:
?- get_age(person(john, 25, london), Age).
Age = 25.

% Update a specific field in a record
update_age(OldPerson, NewAge, NewPerson) :-
    OldPerson = person(Name, _, City),
    NewPerson = person(Name, NewAge, City).

% Or using arg/3 for generic field access:
get_field(Record, FieldNum, Value) :-
    compound(Record),
    arg(FieldNum, Record, Value).

?- get_field(person(john, 25, london), 3, City).
City = london.
```

=../2 (univ)

Purpose: Converts between a compound term and a list (name followed by arguments).

When to use: Use for generic term manipulation, creating terms dynamically, or converting between representations.

ISO §8.5.3 - handles atoms, numbers, and compound terms. Numbers are 0-ary atomic terms: `42 =.. []` (v2.8.2). Construction with numeric functor and arity > 0 throws `type_error(atom, _)`.

v3.6.0: full ISO error clauses (8.5.3.3) - both sides unbound raises `instantiation_error`, a non-list (or improper-tail) list side raises `type_error(list, L)`, `X =.. []` raises `domain_error(non_empty_list, [])`, a compound head with extra arguments raises `type_error(atom, Head)`, `X =.. [f(a)]` raises `type_error(atomic, f(a))`. Works on non-ground terms in both directions: `f(Q) =.. L` decomposes and `X =.. [f, Y]` constructs with unbound arguments (both previously raised `instantiation_error`).

```
% Example: Convert term to list and back
?- person(john, 25, london) =.. List.
List = [person, john, 25, london].

?- 42 =.. L.
L = [42].

?- X =.. [42, a, b]. % Error: numeric functor not allowed with args
% throws type_error(atom, 42)

?- Term =.. [student, mary, 20, 'computer science'].
Term = student(mary, 20, 'computer science').

% Practical example: Add one more argument to any term
add_argument(OldTerm, NewArg, NewTerm) :-
    OldTerm =.. [Functor|OldArgs],
    append(OldArgs, [NewArg], NewArgs),
    NewTerm =.. [Functor|NewArgs].

?- add_argument(point(3, 4), 5, Result).
Result = point(3, 4, 5).

% Generic term builder
```

```
build_term(Functor, Args, Term) :-
    Term =.. [Functor|Args].

?- build_term(employee, [john, manager, 75000], T).
T = employee(john, manager, 75000).
```

copy_term/2

Purpose: Creates a copy of a term with fresh (renamed) variables.

When to use: Use when you need an independent copy of a term with its own variables, often in meta-programming or template processing.

```
% Example: Template-based fact generation
person_template(person(Name, Age, _City)) :-
    atom(Name),
    number(Age),
    Age > 0.

create_person_facts :-
    Template = person(_, _, london),
    % Create multiple facts from template
    copy_term(Template, person(john, 25, london)),
    copy_term(Template, person(mary, 30, london)),
    copy_term(Template, person(bob, 35, london)).

% Example of variable independence:
?- Template = foo(X, X), copy_term(Template, Copy).
Template = foo(X, X),
Copy = foo(_G1, _G1). % New variables but same binding pattern

% Practical use: Avoid variable conflicts in meta-predicates
apply_template(Template, Values, Result) :-
    copy_term(Template, Result),
    Result =.. [Functor|Values].
```

Attributed variables and coroutining

v3.11.0, engine v4 (ISS-2025-0457..0462): these predicates are native (`put_attr/3`, `get_attr/3`, `del_attr/2`, `attvar/1`, `term_attvars/2`) or prelude Prolog (`freeze/2`, `frozen/2`, `when/2`, `dif/2`, `?=/2`) over a real wake queue - binding an attributed variable runs its suspended goals on the machine's goal stack, so **the bindings a woken goal makes propagate**, it is traced through the four ports, the inference budget and Stop can abort it, and an exception it throws reaches the enclosing `catch/3`. (The older Java implementations, which lost a `when/2`-woken goal's bindings - ISS-2025-0336 - belonged to the engine deleted in 4.1.0 and are gone with it.)

One more deliberate behaviour: **a query's variables die with the query**. A goal still suspended when a query ends never fires in a later one (before 4.0.0 it did - session-scoped attributed variables, v2.9.4).

Predicate	Purpose
<code>put_attr(-Var, +Module, +Value)</code>	Attach/replace <code>Module</code> 's attribute on an unbound variable. Backtrackable. <code>type_error(variable, T)</code> on a non-variable.
<code>get_attr(+Var, +Module, ?Value)</code>	Unify <code>Value</code> with the attribute; fails when there is none.
<code>del_attr(+Var, +Module)</code>	Remove the attribute; succeeds when there is none. Backtrackable.
<code>attvar(@Term)</code>	True when <code>Term</code> is an unbound variable carrying at least one attribute.
<code>term_attvars(+Term, -AttVars)</code>	The attributed variables of <code>Term</code> , in depth-first order.
<code>copy_term(+Term, -Copy, -Goals)</code>	Copy with the attributes stripped ; <code>Goals</code> are the residual goals that would restore them, expressed over <code>Copy</code> 's variables.

<code>unifiable(@X, @Y, -Unifier)</code>	The bindings <code>X = Y</code> would make, as a list of <code>Var = Value</code> , without making them . Fails when the terms do not unify.
<code>freeze(?Var, :Goal)</code>	Run <code>Goal</code> as soon as <code>Var</code> is bound (immediately when it already is). Several frozen goals on one variable aggregate into a conjunction.
<code>frozen(@Var, -Goal)</code>	The goal (or conjunction) delayed on <code>Var</code> , or true.
<code>when(+Condition, :Goal)</code>	Run <code>Goal</code> once <code>Condition</code> holds. Conditions: <code>nonvar/1</code> , <code>ground/1</code> , <code>?=/2</code> , <code>(C1, C2)</code> , <code>(C1 ; C2)</code> . <code>instantiation_error</code> for an unbound condition, <code>domain_error(when_condition, C)</code> for anything else. A disjunctive condition fires the goal exactly once.
<code>dif(@X, @Y)</code>	<code>X</code> and <code>Y</code> can never become identical. Decided immediately when they are identical (fail) or cannot unify (succeed); otherwise it suspends on the variables of the remaining unifier and is re-checked as they are bound.
<code>?=(@X, @Y)</code>	True when <code>X</code> and <code>Y</code> are already identical or already cannot unify - i.e. their (dis)equality is decided.

``Module:attr_unify_hook(AttValue, Other)`` - user-definable in Prolog. The machine calls it, through the normal goal stack, for every attribute of a variable that has just been bound; `AttValue` is that module's attribute and `Other` is what the variable was bound to (possibly another variable). Failing the hook fails the unification. An attribute of a module with no hook is inert data.

```
% A one-module constraint library: the variable may only ever be bound to an even integer.
even:attr_unify_hook(_, Other) :-
    (   var(Other) -> put_attr(Other, even, true)
    ;   integer(Other), 0 is Other mod 2
    ).

even(X) :- put_attr(X, even, true).

?- even(X), X = 4.
X = 4.

?- even(X), X = 5.
false.

% Coroutining
?- freeze(X, format("X became ~w~n", [X])), X = hello.
X became hello
X = hello.

?- when(ground(X-Y), Z is X + Y), X = 1, Y = 2.
X = 1, Y = 2, Z = 3.

?- dif(f(X), f(Y)), X = 1, Y = 1.
false.

?- dif(f(X), f(Y)), X = 1, Y = 2.
X = 1, Y = 2.

% Residual goals of a copy
?- freeze(X, foo(X)), copy_term(X, Y, Goals).
Goals = [freeze(Y, foo(Y))].

?- unifiable(f(X, b), f(a, Y), U).
U = [X = a, Y = b].
```

From Java, `Prolog.residualGoals(solution)` returns the same residual goals for the variables of an answer (`freeze/2`, `when/2`, `dif/2`, `CLP(FD)` `in/2`, `put_attr/3`). Nothing prints them yet - the CLI and the IDE start showing them in wave W7 of the v4 design.

compare/3

Purpose: Three-way comparison of terms using standard ordering.

When to use: Use for sorting, searching, or implementing ordered data structures.

v3.6.0: a pre-bound order argument is validated (ISO 8.4.2.3) - a non-atom raises `type_error(atom, Order)` and an atom other than `<`, `=`, `>` raises `domain_error(order, Order)`; valid pre-bound orders still verify by unification.

```
% Standard term ordering: variables < numbers < atoms < compound terms
% Within each category, there's a specific ordering

% Example: Compare any two terms
?- compare(Order, 3, 5).
Order = (<). % 3 is less than 5

?- compare(Order, apple, banana).
Order = (<). % Alphabetical ordering for atoms

?- compare(Order, foo(1), foo(2)).
Order = (<). % Compound terms compared by arguments

% Practical example: Insert into sorted list
insert_sorted(X, [], [X]).
insert_sorted(X, [H|T], [X,H|T]) :-
    compare(<, X, H), !.
insert_sorted(X, [H|T], [H|Rest]) :-
    insert_sorted(X, T, Rest).

?- insert_sorted(3, [1, 2, 4, 5], Result).
Result = [1, 2, 3, 4, 5].

% Binary search tree insertion
insert_bst(X, nil, tree(X, nil, nil)).
insert_bst(X, tree(V, L, R), tree(V, NewL, R)) :-
    compare(<, X, V),
    insert_bst(X, L, NewL).
insert_bst(X, tree(V, L, R), tree(V, L, NewR)) :-
    compare(>, X, V),
    insert_bst(X, R, NewR).
insert_bst(X, tree(X, L, R), tree(X, L, R)). % Already exists
```

term_variables/2

Purpose: Extracts all variables from a term into a list.

When to use: Use to find all uninstantiated variables in complex terms.

```
% Example: Find all variables in a term
?- term_variables(foo(X, bar(Y, X, 3), Z), Vars).
Vars = [X, Y, Z].

% Practical example: Check if a rule has unbound variables
check_rule_complete(Rule) :-
    term_variables(Rule, Vars),
    ( Vars = []
    -> write('Rule is fully instantiated')
    ; length(Vars, N),
      write('Rule has '), write(N), write(' unbound variables')
    ).

?- check_rule_complete(parent(john, mary)).
Rule is fully instantiated
```

```

true.

?- check_rule_complete(parent(X, mary)).
Rule has 1 unbound variables
true.

```

subsumes_term/2

Purpose: Checks if one term is more general than another.

When to use: Use for pattern matching and generalization checking.

```

% A term T1 subsumes T2 if T1 can be made identical to T2 by binding variables

% Examples:
?- subsumes_term(foo(X, Y), foo(a, b)).
true. % foo(X,Y) is more general than foo(a,b)

?- subsumes_term(foo(X, X), foo(a, b)).
false. % Would need X=a and X=b simultaneously

?- subsumes_term(foo(a, b), foo(X, Y)).
false. % foo(a,b) is more specific, not more general

% Practical example: Find matching templates
find_matching_template(Data, Templates, Match) :-
    member(Match, Templates),
    subsumes_term(Match, Data).

% With templates:
templates([
    person(_, _, _),           % Any person
    person(john, _, _),        % John specifically
    person(_, Age, _) :- Age > 18 % Adults
]).

?- find_matching_template(person(john, 25, london),
    [person(_, _, paris), person(john, _, _)],
    Match).
Match = person(john, _, _).

```

term_to_atom/2

Purpose: Convert between a term and its atom representation. Bidirectional.

v2.8.2: term->atomdirection uses operator-aware formatter for proper roundtrip (1+2 is written as 1+2, not +(1,2)).

v3.6.0: works on non-ground terms - term_to_atom(foo(X, bar), A) formats the variable (A = 'foo(_G1, bar)') instead of failing; a bound atom side keeps parse-and-unify semantics (term_to_atom(foo(Z), 'foo(bar)') binds Z = bar).

```

?- term_to_atom(f(a, b), X).
X = 'f(a, b)'.

?- term_to_atom(T, 'f(a, b)').
T = f(a, b).

?- term_to_atom(1+2*3, A).
A = '1+2*3'. % operator notation preserved

```

term_string/2

Purpose: Convert between a term and its **string** representation. The SWI string twin of term_to_atom/2, bidirectional and with the same rules. (*added v4.2.0*)

```
?- term_string(f(x), S).
S = "f(x)".

?- term_string(T, "foo(a, B)").
T = foo(a, B).

?- term_string(1+2*3, S).
S = "1+2*3".    % operator notation preserved
```

The string side also accepts an atom, so term_string(T, 'f(a)') parses; the term side is written with quoted(true), exactly as term_to_atom/2 writes it.

atom_to_term/3

Purpose: Parse an atom as a Prolog term, returning the term plus a list of variable bindings (Name=Var pairs). (*v2.8.2+*)

```
?- atom_to_term('foo(X, Y)', T, B).
T = foo(_42, _43),
B = ['X'=_42, 'Y'=_43].
```

Use case: reading and post-processing user input that may contain variables.

numbervars/3

Purpose: Number unbound variables in a term with \$VAR(N) terms.

```
?- numbervars(f(X, Y, X), 0, End).
X = '$VAR'(0), Y = '$VAR'(1), End = 2.
```

3. List Operations

Lists are fundamental data structures in Prolog. These predicates provide essential list manipulation capabilities.

Understanding Prolog Lists

A list in Prolog is either:

- Empty: `[]`
- Non-empty: `[Head|Tail]` where Head is an element and Tail is a list

Examples:

- `[1, 2, 3]` is syntactic sugar for `[1|[2|[3|[]]]]`
- `[a, b|Rest]` matches a list starting with a, b, with Rest as the remaining list

append/3

Purpose: Concatenates two lists or splits a list into parts.

When to use: Use for joining lists, finding prefixes/suffixes, or generating list partitions.

v3.10.0, engine v4 (ISS-2025-0453): `member/2`, `memberchk/2`, `append/3`, `select/3`, `selectchk/3`, `nth0/3`, `nth1/3`, `last/2`, `reverse/2`, `length/2`, `msort/2`, `sort/2`, `sum_list/2`, `numlist/3`, `copy_term/2` and `clause/2` are native to the v4 machine, and the nondeterministic ones are **lazy generators**: one alternative per redo, O(1) memory, and the enumeration stops the moment the caller cuts, instead of every solution being materialised before the first is used. Modes, solution order and ISO error terms are unchanged - including two deliberate parity points: `append(X, Y, Z)` with all three arguments open still yields only the single standard solution `X = [], Z = Y`, and `member(X, PartialList)` does not extend the open tail.

```
% Mode 1: Concatenate two lists (inputs: +List1, +List2, output: -List3)
?- append([1, 2], [3, 4], Result).
Result = [1, 2, 3, 4].

% Mode 2: Find all ways to split a list (input: +List3, outputs: -List1, -List2)
?- append(Left, Right, [a, b, c]).
Left = [], Right = [a, b, c] ;
Left = [a], Right = [b, c] ;
Left = [a, b], Right = [c] ;
Left = [a, b, c], Right = [].

% Practical example: Insert element in middle of list
insert_after(Element, After, List, NewList) :-
    append(Prefix, [After|Suffix], List),
    append(Prefix, [After, Element|Suffix], NewList).

?- insert_after(new, b, [a, b, c, d], Result).
Result = [a, b, new, c, d].

% Find if one list is contained in another
sublist(Sub, List) :-
    append(_, Rest, List),
    append(Sub, _, Rest).

?- sublist([b, c], [a, b, c, d]).
true.
```

member/2

Purpose: Checks membership or generates elements of a list.

When to use: Use to test if an element is in a list or to iterate through list elements.

```
% Mode 1: Check if element is in list
?- member(b, [a, b, c]).
true.

?- member(x, [a, b, c]).
false.

% Mode 2: Generate all elements of a list
?- member(X, [red, green, blue]).
X = red ;
X = green ;
X = blue.

% Practical example: Find common elements in two lists
common_elements(List1, List2, Common) :-
    findall(X, (member(X, List1), member(X, List2)), CommonList),
    list_to_set(CommonList, Common). % Remove duplicates

?- common_elements([1, 2, 3, 4], [3, 4, 5, 6], Common).
Common = [3, 4].

% Validate input from a set of options
validate_option(Option, ValidOptions) :-
    member(Option, ValidOptions),
    !.
validate_option(Option, ValidOptions) :-
    write('Invalid option: '), write(Option),
    write('. Valid options are: '), write(ValidOptions), nl,
    fail.
```

length/2

Purpose: Relates a list to its length.

When to use: Use to count elements, create lists of specific length, or constrain list size.

v3.5.0: accepts proper lists containing unbound elements (e.g. `length([A, B, C], N)` gives `N = 3`) - only the list skeleton must be proper.

v3.6.1 (ISS-2025-0425): the **generative** mode works. When the length is unbound and the list's spine ends in an unbound tail, `length/2` enumerates `N = Prefix, Prefix+1, ..` on backtracking instead of failing, so `length(L, N)`, `N >= 3`, `!` gives `N = 3`, `L = [_ , _ , _]` and `length([a|T], N)` enumerates `N = 1, 2, 3, ..` Like SWI-Prolog, an **unguarded** `length(L, N)` with both arguments unbound is therefore a non-terminating generator - bound it with a cut, a comparison, or a known length.

```
% Mode 1: Find length of a list
?- length([a, b, c, d], Len).
Len = 4.

% Mode 2: Create a list of specific length with unbound variables
?- length(List, 3).
List = [_G1, _G2, _G3].

% Mode 3: Check if list has specific length
?- length([a, b, c], 3).
true.

% Practical example: Pad a list to specific length
```

```

pad_list(List, TargetLen, PadValue, PaddedList) :-
    length(List, CurrentLen),
    PadCount is TargetLen - CurrentLen,
    (   PadCount <= 0
    ->   PaddedList = List
    ;   length(Padding, PadCount),
        maplist(=(PadValue), Padding),
        append(List, Padding, PaddedList)
    ).

?- pad_list([a, b], 5, x, Result).
Result = [a, b, x, x, x].

% Generate lists of increasing length
generate_lists(Max) :-
    between(0, Max, N),
    length(List, N),
    write('List of length '), write(N), write(': '), write(List), nl,
    fail.
generate_lists(_).

```

reverse/2

Purpose: Reverses the order of elements in a list.

When to use: Use for reversing sequences, implementing stacks, or palindrome checking.

v3.5.0: the inverse mode `reverse(-List, +Reversed)` works, and lists may contain unbound elements (`reverse([A, B], R)` gives `R = [B, A]`).

```

% Basic usage
?- reverse([1, 2, 3, 4], Rev).
Rev = [4, 3, 2, 1].

% Inverse mode (v3.5.0+)
?- reverse(X, [1, 2, 3]).
X = [3, 2, 1].

% Check if list is palindrome
is_palindrome(List) :-
    reverse(List, List).

?- is_palindrome([a, b, c, b, a]).
true.

?- is_palindrome([a, b, c]).
false.

% Practical example: Process list in reverse order without modifying original
process_reverse(List) :-
    reverse(List, Reversed),
    process_items(Reversed).

% Build a string in reverse (useful for efficiency)
build_string_reverse([], Acc, Result) :-
    reverse(Acc, Result).
build_string_reverse([H|T], Acc, Result) :-
    process_char(H, Processed),
    build_string_reverse(T, [Processed|Acc], Result).

```

nth0/3 and nth1/3

Purpose: Access or find the position of an element in a list (0-based or 1-based indexing).

When to use: Use for indexed access to list elements or finding positions.

```
% nth0/3 - Zero-based indexing (like arrays in most languages)
?- nth0(0, [a, b, c, d], Element).
Element = a.

?- nth0(2, [a, b, c, d], Element).
Element = c.

% nth1/3 - One-based indexing (more natural for humans)
?- nth1(1, [a, b, c, d], Element).
Element = a.

?- nth1(3, [a, b, c, d], Element).
Element = c.

% Find position of element
?- nth0(Position, [apple, banana, cherry], banana).
Position = 1.

?- nth1(Position, [apple, banana, cherry], banana).
Position = 2.

% Practical example: Replace element at position
replace_nth(N, List, NewElement, NewList) :-
    nth0(N, List, OldElement, Rest),
    nth0(N, NewList, NewElement, Rest).

?- replace_nth(1, [a, b, c, d], x, Result).
Result = [a, x, c, d].

% Get multiple elements by indices
get_elements_at([], _, []).
get_elements_at([Index|Indices], List, [Element|Elements]) :-
    nth0(Index, List, Element),
    get_elements_at(Indices, List, Elements).

?- get_elements_at([0, 2, 3], [a, b, c, d, e], Elements).
Elements = [a, c, d].
```

permutation/2

Purpose: True when one list is a permutation (reordering) of the other.

When to use: Use to enumerate all orderings of a list or to check that two lists contain the same elements.

```
% Enumerate permutations
?- permutation([a, b, c], P).
P = [a, b, c] ;
P = [a, c, b] ;
P = [b, a, c] ;
P = [b, c, a] ;
P = [c, a, b] ;
P = [c, b, a].

% Check a specific reordering
?- permutation([1, 2, 3], [3, 1, 2]).
true.

% Inverse mode (v3.5.0+)
```

```
?- permutation(P, [1, 2]).
P = [1, 2] ;
P = [2, 1].
```

v3.5.0: the inverse mode `permutation(-List, +Permutation)` works, and lists may contain unbound elements.

select/3

Purpose: Selects an element from a list, returning the element and the rest.

When to use: Use for removing elements, permutations, or non-deterministic selection.

v3.5.0: the insertion mode `select(+Elem, -List, +Rest)` (shown below) works, and lists may contain unbound elements.

```
% Remove an element from a list
?- select(b, [a, b, c, d], Rest).
Rest = [a, c, d].

% Non-deterministic selection - try all possibilities
?- select(X, [1, 2, 3], Rest).
X = 1, Rest = [2, 3] ;
X = 2, Rest = [1, 3] ;
X = 3, Rest = [1, 2].

% Insert an element (using select/3 backwards)
?- select(x, Result, [a, b, c]).
Result = [x, a, b, c] ;
Result = [a, x, b, c] ;
Result = [a, b, x, c] ;
Result = [a, b, c, x].

% Practical example: Remove all occurrences of an element
remove_all(_, [], []).
remove_all(X, List, Result) :-
    select(X, List, Rest),
    !,
    remove_all(X, Rest, Result).
remove_all(_, List, List).

?- remove_all(a, [a, b, a, c, a, d], Result).
Result = [b, c, d].

% Generate permutations
permutation([], []).
permutation(List, [H|Perm]) :-
    select(H, List, Rest),
    permutation(Rest, Perm).

?- permutation([a, b, c], P).
P = [a, b, c] ;
P = [a, c, b] ;
P = [b, a, c] ;
P = [b, c, a] ;
P = [c, a, b] ;
P = [c, b, a].
```

sort/2, sort/4 and msort/2

Purpose: Sort lists in standard order.

- `sort/2` removes duplicates
- `msort/2` keeps duplicates

- `sort/4` (v2.8.1+) accepts key index + order operator: `sort(+Key, +Order, +List, -Sorted)`

```
% sort/4 with custom order
?- sort(0, @>, [3, 1, 2, 1], L).      % descending, dedup
L = [3, 2, 1].

?- sort(0, @=<=, [3, 1, 2, 1], L).    % ascending, KEEP dups
L = [1, 1, 2, 3].

% sort by 2nd arg of compound
?- sort(2, @<, [pair(a, 30), pair(b, 10), pair(c, 20)], L).
L = [pair(b, 10), pair(c, 20), pair(a, 30)].
```

Key = 0 means compare whole terms; Key = N (N>=1) extracts N-th argument of compound. Order operators: `@<`, `@=<=`, `@>`, `@=>=` (the `=</>=` variants keep duplicates).

v3.5.0: ISO error handling - an unbound first argument or a partial list (e.g. `[a|_]`) raises `instantiation_error`, a non-list raises `type_error(list, Culprit)` (previously these failed silently). Lists may contain unbound variables, which sort first in the standard order.

v3.6.0: `sort/4` validates keys and options with ISO error terms - with Key > 0 every element must be a compound of arity >= Key (`type_error(compound, Elem)`, `domain_error(argument_index, Key)` otherwise); a bad Key raises `type_error(integer, K)` / `domain_error(not_less_than_zero, K)` and a bad Order raises `type_error(atom, 0)` / `domain_error(order, 0)`. `predsort/3` accepts non-ground lists (`predsort(compare, [X, Y], L)` works), fails when the comparison predicate fails or yields a non-order, and raises `instantiation_error` / `type_error(callable, Pred)` on a bad predicate.

When to use: Use for ordering data, removing duplicates, or preparing data for efficient searching.

```
% sort/2 - Sort and remove duplicates
?- sort([3, 1, 4, 1, 5, 9, 2, 6], Sorted).
Sorted = [1, 2, 3, 4, 5, 6, 9]. % Note: only one 1

% msort/2 - Sort but keep duplicates
?- msort([3, 1, 4, 1, 5, 9, 2, 6], Sorted).
Sorted = [1, 1, 2, 3, 4, 5, 6, 9]. % Both 1s kept

% Sort complex terms (sorted by standard term ordering)
?- sort([person(john, 30), person(alice, 25), person(bob, 25)], Sorted).
Sorted = [person(alice, 25), person(bob, 25), person(john, 30)].

% Practical example: Find unique elements
unique_elements(List, Unique) :-
    sort(List, Unique).

?- unique_elements([a, b, a, c, b, d], Unique).
Unique = [a, b, c, d].

% Count occurrences after sorting
count_occurrences(List, Counts) :-
    msort(List, Sorted),
    count_consecutive(Sorted, Counts).

count_consecutive([], []).
count_consecutive([H|T], [H-Count|Rest]) :-
    count_same(H, T, 1, Count, Remaining),
    count_consecutive(Remaining, Rest).

count_same(X, [X|T], Acc, Count, Rest) :-
    !,
    Acc1 is Acc + 1,
    count_same(X, T, Acc1, Count, Rest).
count_same(_, List, Count, Count, List).
```

keysort/2

Purpose: Sorts a list of Key-Value pairs by key in standard order, keeping duplicates (stable sort).

When to use: Use to order tagged data by key while preserving the relative order of values with equal keys.

```
?- keysort([b-2, a-1, c-3, a-0], Sorted).
Sorted = [a-1, a-0, b-2, c-3].    % stable: a-1 stays before a-0
```

v3.5.0: accepts non-ground pairs (`keysort([K-V, b-2], S)` works - its primary use case), compares keys only (values are never compared), and raises ISO errors - `instantiation_error` on an unbound argument or partial list, `type_error(list, _)` on a non-list, `type_error(pair, _)` on an element that is not a Key-Value pair.

last/2

Purpose: True if Elem is the last element of List.

```
?- last([1, 2, 3], X).
X = 3.
```

flatten/2

Purpose: Flatten a nested list structure into a single flat list.

```
?- flatten([1, [2, [3, 4]], 5], X).
X = [1, 2, 3, 4, 5].
```

numlist/3

Purpose: Generate a list of consecutive integers from Low to High.

```
?- numlist(1, 5, X).
X = [1, 2, 3, 4, 5].
```

v3.5.0: raises ISO errors on bad bounds - `instantiation_error` if Low or High is unbound, `type_error(integer, Culprit)` if a bound is not an integer (previously failed silently).

sum_list/2, sumlist/2

Purpose: Sum all numeric elements of a list.

```
?- sum_list([1, 2, 3, 4], X).
X = 10.
```

max_list/2, min_list/2

Purpose: Find the maximum/minimum numeric element in a list.

```
?- max_list([3, 1, 4, 1, 5], X).
X = 5.

?- min_list([3, 1, 4, 1, 5], X).
X = 1.
```

delete/3

Purpose: Remove all occurrences of an element from a list.

```
?- delete([1, 2, 1, 3, 1], 1, X).
X = [2, 3].
```

subtract/3

Purpose: Remove from Set all elements present in Delete.

```
?- subtract([1, 2, 3, 4], [2, 4], X).
X = [1, 3].
```

intersection/3

Purpose: Elements present in both sets.

```
?- intersection([1, 2, 3], [2, 3, 4], X).
X = [2, 3].
```

union/3

Purpose: Set1 plus elements from Set2 not already in Set1.

```
?- union([1, 2, 3], [2, 3, 4], X).
X = [1, 2, 3, 4].
```

maplist/2, maplist/3, maplist/4, maplist/5

Purpose: Apply a goal to each element of a list (or each tuple across multiple lists). Higher-order predicate.

```
?- maplist(atom, [a, b, c]).
true.

?- maplist(succ, [1, 2, 3], Result).
Result = [2, 3, 4].

?- maplist(plus, [1,2,3], [10,20,30], R).
R = [11, 22, 33].
```

Added in v2.8.1: maplist/5 arity for call(Goal, E1, E2, E3, E4) across 4 lists.

v3.10.0, engine v4 only (-Djprolog.engine=v4, ISS-2025-0454): maplist/2..7 are **Prolog clauses** loaded from the v4 prelude (prelude/apply.pl) instead of a Java built-in. They are linear rather than quadratic with an output list, lazy (re-satisfiable through the mapped goal), traceable through the four ports and interruptible; maplist/6 and maplist/7 exist only there. **A program that defines its own `maplist/3` overrides the library definition** - the prelude is consulted only when the knowledge base has no clause for that indicator.

include/3, exclude/3

Purpose: Filter a list by keeping (include) or removing (exclude) elements where Goal succeeds.

```
?- include(atom, [a, 1, b, 2], X).
X = [a, b].
```

```
?- exclude(atom, [a, 1, b, 2], X).
X = [1, 2].
```

partition/4, partition/5

Purpose: Split a list in one pass. `partition(:Pred, +List, ?Included, ?Excluded)` puts each element where `call(Pred, X)` succeeds into `Included` and the rest into `Excluded`; `partition(:Pred, +List, ?Less, ?Equal, ?Greater)` uses `call(Pred, X, Order)` with `Order` one of `<, =, >`.

Availability: since v3.10.0 (ISS-2025-0454), as prelude Prolog clauses; before that a call raised `existence_error(procedure, partition/4)` and programs defined their own. That still works: **a user definition of `partition/4` overrides the library one** (the quicksort in `examples/test_16_sorting.pl` relies on this, and its `partition/4` takes a pivot rather than a goal).

```
?- partition([X]>>(X > 2), [1,2,3,4], Big, Small).
Big = [3, 4], Small = [1, 2].
```

foldl/4, foldl/5, foldl/6, foldl/7

Purpose: Left fold over a list with an accumulator.

```
add(X, Y, Z) :- Z is X + Y.
?- foldl(add, [1, 2, 3], 0, Sum).
Sum = 6.
```

v3.10.0, the default (v4) engine only: `foldl/4..7` are prelude Prolog clauses (see `maplist`), so they fold over four parallel lists and are linear and interruptible.

Lambda expressions - `library(yall)`

Purpose: Write an anonymous predicate inline instead of naming a helper.

Availability: since v3.10.0 (ISS-2025-0455); before that a lambda raised `existence_error(procedure, >>/4)`.

Form	Meaning
<code>Params>>Body</code>	<code>Params</code> is a list of formal parameters, e.g. <code>[X,Y]>>(Y is X*2)</code>
<code>Free/Params>>Body</code>	the variables of <code>Free</code> are shared with the caller instead of renamed apart
<code>\X1^...^Xn^Body</code>	the <code>library(lambda)</code> spelling of the same thing
<code>Free/\X^Body</code>	... with shared free variables

The lambda is copied before **every** call, so one lambda serves every element of a `maplist`. A variable that is already bound outside is copied as its value, so it needs no `/`; use `Free/` only to share an *unbound* variable.

```
?- maplist([X,Y]>>(Y is X*2), [1,2,3], L).
L = [2, 4, 6].

?- foldl([X,A0,A]>>(A is A0+X), [1,2,3,4], 0, S).
S = 10.

?- N = 10, maplist(N/[X,Y]>>(Y is X*N), [1,2], L).
L = [10, 20].
```

4. Arithmetic Predicates

Arithmetic predicates handle mathematical calculations and comparisons. Prolog uses special evaluation rules for arithmetic.

Understanding Arithmetic in Prolog

Prolog treats arithmetic expressions differently from other terms:

- `X = 2 + 3` unifies `X` with the term `+(2, 3)`, NOT with 5
- `X is 2 + 3` evaluates the expression and unifies `X` with 5
- Arithmetic comparisons (`<`, `>`, etc.) automatically evaluate their arguments

is/2

Purpose: Evaluates an arithmetic expression and unifies the result.

When to use: Use for all arithmetic calculations where you need the computed value.

Available evaluable functors

Arithmetic (+, -, *, /, // integer div, mod, rem, div, rdiv (*rational*), ** (ISO float power - ALWAYS float, e.g. `2*3 := 8.0`), `^` integer power (v2.8.1), unary `-`, `+`, `abs`, `sign`, `min`, `max`, `gcd` (v2.8.0*))

Float-only (sqrt, sin, cos, tan, asin, acos, atan, atan2, exp, log, log/2 (v2.8.1), cot (v2.8.1), acot (v2.8.1), cbrt (v2.8.1), sinh, cosh, tanh, asinh, acosh, atanh (*all* v2.8.1))

Float manipulation (truncate, round, floor, ceiling, float, integer (v2.8.1), float_integer_part, float_fractional_part, rational / rationalize (v2.8.1))

Bitwise integer (/ \, \/, xor, \ (NOT), <<, >>, msb, lsb, popcount)

Constants (pi, e, inf, nan, epsilon (v2.8.1), max_tagged_integer (v2.8.1), min_tagged_integer (v2.8.1))

v3.5.0: ISO error behavior - a computed float overflow raises `evaluation_error(float_overflow)` and a NaN result raises `evaluation_error(undefined)` (the `inf/nan` constants and their propagation still work); `0 ^ -1` raises `evaluation_error(zero_divisor)`; huge `^<</>>` operands raise a catchable ISO error instead of an unhandled Java exception.

v3.6.0: `float_integer_part` and `float_fractional_part` are correct beyond $\pm 2^{63}$ `float_integer_part(1.0e20)` gives `1.0e20` (results no longer saturate at the 64-bit integer range); truncate-toward-zero semantics for small and negative values are unchanged.

```
% Basic arithmetic
?- X is 2 + 3.
X = 5.

?- Y is 10 * (3 + 4).
Y = 70.

% Using variables (right side must be instantiated)
?- A = 5, B = 3, C is A * B.
A = 5, B = 3, C = 15.

% Common arithmetic functions
?- X is sqrt(16).
X = 4.0.

?- Y is sin(0).
Y = 0.0.

?- Z is max(5, 3).
```

```

Z = 5.

% Integer power
?- X is 2 ^ 10.
X = 1024.

% Truncating cast
?- X is integer(3.7).
X = 3.

?- X is integer(-3.7).
X = -3.

% Hyperbolic
?- X is sinh(0.0).
X = 0.0.

% Base-N logarithm
?- X is log(10, 100).
X = 2.0.

% GCD
?- X is gcd(12, 18).
X = 6.

% Epsilon
?- X is epsilon.
X = 2.220446049250313e-16.

% Practical example: Calculate compound interest
compound_interest(Principal, Rate, Time, Amount) :-
    Amount is Principal * (1 + Rate/100) ** Time.

?- compound_interest(1000, 5, 3, Amount).
Amount = 1157.625.

% Factorial calculation
factorial(0, 1) :- !.
factorial(N, Result) :-
    N > 0,
    N1 is N - 1,
    factorial(N1, R1),
    Result is N * R1.

?- factorial(5, R).
R = 120.

```

Arithmetic Comparison Operators

These operators automatically evaluate arithmetic expressions before comparing.

```

% Less than: <
?- 3 < 5.
true.

?- 2 + 3 < 4 + 2. % Evaluates to: 5 < 6
true.

% Less than or equal: <= (Note: not <= like other languages!)
?- 5 <= 5.
true.

% Greater than: >
?- 10 > 3.
true.

```

```

% Greater than or equal: >=
?- 7 >= 7.
true.

% Arithmetic equality: ==
?- 2 + 3 == 5.
true.

?- 2 + 3 == 4 + 1.
true.

% Arithmetic inequality: \=
?- 2 + 3 \= 6.
true.

% Practical example: Temperature converter with validation
celsius_to_fahrenheit(C, F) :-
    C >= -273.15, % Absolute zero check
    F is C * 9/5 + 32.

?- celsius_to_fahrenheit(100, F).
F = 212.0.

?- celsius_to_fahrenheit(-300, F).
false. % Below absolute zero

% Finding values in range
in_range(X, Min, Max) :-
    X >= Min,
    X <= Max.

?- in_range(5, 1, 10).
true.

% Maximum of three numbers
max_of_three(A, B, C, Max) :-
    ( A >= B, A >= C -> Max = A
    ; B >= A, B >= C -> Max = B
    ; Max = C
    ).

```

between/3

Purpose: Generates or tests integers within a range.

When to use: Use for generating sequences, validating ranges, or iteration.

v3.7.0 (ISS-2025-0432): the generative mode is **lazy** on the default engine - one integer per backtrack instead of materialising the whole range up front, so `between(1, 2000000, X), X >= 2000000, !` runs in constant memory. `between(Low, inf, X)` / `between(Low, infinite, X)` genuinely enumerate without an upper bound (they were silently capped at a million solutions).

```

% Mode 1: Check if number is in range
?- between(1, 10, 5).
true.

?- between(1, 10, 15).
false.

% Mode 2: Generate all integers in range
?- between(1, 5, X).
X = 1 ;
X = 2 ;
X = 3 ;

```

```

X = 4 ;
X = 5.

% Practical example: Generate multiplication table
multiplication_table(N) :-
    between(1, 10, I),
    Result is N * I,
    format('~w x ~w = ~w~n', [N, I, Result]),
    fail.
multiplication_table(_).

?- multiplication_table(7).
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
...
7 x 10 = 70
true.

% Find perfect squares in range
perfect_squares(Min, Max, Squares) :-
    findall(X, (between(Min, Max, X),
        Sqrt is sqrt(X),
        Sqrt := floor(Sqrt)),
        Squares).

?- perfect_squares(1, 20, S).
S = [1, 4, 9, 16].

```

succ/2

Purpose: Relates consecutive integers (successor relation).

When to use: Use for increment/decrement operations or defining natural number sequences.

```

% Find successor
?- succ(5, X).
X = 6.

% Find predecessor
?- succ(X, 10).
X = 9.

% Check consecutive numbers
?- succ(7, 8).
true.

% Practical example: Count down
countdown(0) :- write('Blast off!'), nl, !.
countdown(N) :-
    N > 0,
    write(N), nl,
    succ(N1, N),
    countdown(N1).

?- countdown(3).
3
2
1
Blast off!
true.

```

plus/3

Purpose: Relates three integers where the third is the sum of the first two.

When to use: Use for addition with multiple unknown values.

```
% Normal addition
?- plus(3, 4, X).
X = 7.

% Find what to add
?- plus(3, X, 10).
X = 7.

% Find first addend
?- plus(X, 5, 12).
X = 7.

% Practical example: Calculate remaining budget
remaining_budget(Total, Spent, Remaining) :-
    plus(Spent, Remaining, Total).

?- remaining_budget(1000, 750, R).
R = 250.

?- remaining_budget(1000, S, 400).
S = 600.
```

5. Control Flow

Control flow predicates determine how Prolog executes programs, including conditionals, loops, and backtracking control.

! (cut)

Purpose: Prevents backtracking past this point.

When to use: Use to commit to a choice, improve efficiency, or implement deterministic predicates.

```
% Example: Deterministic max/3 using cut
max(X, Y, X) :- X >= Y, !.
max(X, Y, Y).

% Without cut, max/3 might give multiple solutions
% With cut, once X >= Y succeeds, we commit to that answer

?- max(5, 3, M).
M = 5. % Only one solution

% Practical example: Grade classification
grade(Score, Grade) :-
    Score >= 90, !,
    Grade = 'A'.
grade(Score, Grade) :-
    Score >= 80, !,
    Grade = 'B'.
grade(Score, Grade) :-
    Score >= 70, !,
    Grade = 'C'.
grade(Score, Grade) :-
    Score >= 60, !,
    Grade = 'D'.
grade(_, 'F').

?- grade(85, G).
G = 'B'. % Only one answer, no backtracking

% Red cut vs Green cut
% Green cut: Doesn't change meaning, just efficiency
% Red cut: Changes program meaning

% Green cut example:
member_check(X, [X|_]) :- !. % Found it, no need to search more
member_check(X, [_|T]) :- member_check(X, T).

% Red cut example (dangerous):
min(X, Y, X) :- X < Y, !.
min(_, Y, Y). % Assumes X >= Y without checking
```

-> (if-then) and ; (else)

Purpose: Implements conditional execution (if-then-else).

When to use: Use for conditional logic where you need different actions based on conditions.

```
% Basic if-then syntax: (Condition -> ThenPart)
test_sign(X) :-
    ( X > 0
    -> write('Positive')
    ).
```

```
% If-then-else syntax: (Condition -> ThenPart ; ElsePart)
sign(X, Sign) :-
  (   X > 0
  -> Sign = positive
  ;   X < 0
  -> Sign = negative
  ;   Sign = zero
  ).

?- sign(5, S).
S = positive.

?- sign(-3, S).
S = negative.

?- sign(0, S).
S = zero.

% Practical example: Safe division
safe_divide(X, Y, Result) :-
  (   Y == 0
  -> write('Error: Division by zero'), nl,
      fail
  ;   Result is X / Y
  ).

?- safe_divide(10, 2, R).
R = 5.0.

?- safe_divide(10, 0, R).
Error: Division by zero
false.

% Nested conditions
classify_age(Age, Category) :-
  (   Age < 0
  -> Category = invalid
  ;   Age < 13
  -> Category = child
  ;   Age < 20
  -> Category = teenager
  ;   Age < 60
  -> Category = adult
  ;   Category = senior
  ).
```

\+ (negation as failure)

Purpose: Succeeds if the goal fails (negation by failure).

When to use: Use to test that something is NOT true.

```
% Basic negation
?- \+ member(x, [a, b, c]).
true. % x is not a member

?- \+ member(b, [a, b, c]).
false. % b is a member, so negation fails

% Practical example: Find elements not in a list
not_in_list([], _).
not_in_list([H|T], Exclude) :-
  \+ member(H, Exclude),
  not_in_list(T, Exclude).
```

```
?- not_in_list([a, b, c], [b, d, e]).
false. % b is in the exclude list

?- not_in_list([a, c], [b, d, e]).
true. % neither a nor c is in exclude list

% Check uniqueness
all_different([]).
all_different([H|T]) :-
    \+ member(H, T),
    all_different(T).

?- all_different([a, b, c]).
true.

?- all_different([a, b, a]).
false.

% Important: Negation by failure has limitations
% \+ X = 5 fails if X is unbound (can't prove X is not 5)
% Use with ground terms for predictable behavior
```

once/1

Purpose: Succeeds at most once (finds first solution only).

When to use: Use to make non-deterministic predicates deterministic.

v3.7.0 (ISS-2025-0431): `once/1`, `ignore/1` and `forall/2` run natively on the default engine (as `(G -> true), (G -> true ; true)` and `\+ (C, \+ A)`) and every other meta-call built-in - `aggregate_all/3`, `bagof/3`, `setof/3`, `setup_call_cleanup/3`, `with_output_to/2`, `maplist/2..5`, `foldl/4..6`, `include/3`, `exclude/3`, `partition/4`, `predsort/3` - runs its sub-goal on the iterative machine instead of the (since 4.0.0 deleted) recursive solver. Visible effects: they are one to two orders of magnitude faster, they no longer hit a recursion limit on long lists, and (importantly for embedders) the inference budget set by `Prolog.setInferenceBudget` and the Stop interrupt now apply **inside** them. Semantics - cut opacity, determinism, ISO error terms - are unchanged.

v3.6.0: a non-callable goal raises `type_error(callable, Goal)` (`once(1)` used to fail silently); an unbound goal raises `instantiation_error`.

```
% Without once/1 - multiple solutions
?- member(X, [a, b, c]).
X = a ;
X = b ;
X = c.

% With once/1 - first solution only
?- once(member(X, [a, b, c])).
X = a.

% Practical example: Find first matching record
find_first_adult(People, Adult) :-
    once((member(person(Name, Age), People), Age >= 18)),
    Adult = Name.

?- find_first_adult([person(tom, 12), person(jane, 20), person(bob, 25)], A).
A = jane. % Only returns first adult, not all

% Optimization: Prevent unnecessary backtracking
expensive_check(X) :-
    once(complex_computation(X, Result)),
    Result > threshold.

% Ensure deterministic behavior
```

```
get_default(Key, Value) :-
    once((lookup(Key, Value) ; Value = default)).
```

repeat/0

Purpose: Always succeeds and provides infinite choice points.

When to use: Use to create loops that retry on failure.

v3.6.1 (ISS-2025-0423): genuinely **infinite** on the default engine. It previously produced exactly 1000 solutions, so a repeat, ... Done, ! driver loop that needed more than 1000 iterations failed silently. (The 1000-solution bound of the pre-3.6.1 recursive engine is history: that engine was deleted in 4.0.0.)

```
% repeat/0 always succeeds and creates a choice point
% On backtracking, it succeeds again (infinitely)

% Interactive menu example
menu :-
    repeat,
    write('1. Option A'), nl,
    write('2. Option B'), nl,
    write('3. Quit'), nl,
    write('Enter choice: '),
    read(Choice),
    process_choice(Choice),
    Choice = 3, % Exit condition
    !.

process_choice(1) :- write('You chose A'), nl.
process_choice(2) :- write('You chose B'), nl.
process_choice(3) :- write('Goodbye'), nl.
process_choice(_) :- write('Invalid choice'), nl.

% Read until valid input
get_positive_number(N) :-
    repeat,
    write('Enter a positive number: '),
    read(N),
    (   number(N), N > 0
    -> ! % Cut to stop repeating
    ;   write('Invalid input, try again'), nl,
        fail % Force backtrack to repeat
    ).
```

forall/2

Purpose: Succeeds if Action succeeds for all solutions of Condition.

When to use: Use to verify universal conditions or perform actions on all solutions.

v3.6.0: a non-callable Condition or Action raises `type_error(callable, G)` (`forall(1, true)` and `forall(true, 1)` used to succeed silently); an unbound goal raises `instantiation_error`.

```
% Syntax: forall(Condition, Action)
% Succeeds if Action succeeds for every solution of Condition

% Check if all elements satisfy a condition
?- forall(member(X, [2, 4, 6, 8]), X mod 2 == 0).
true. % All are even

?- forall(member(X, [2, 4, 5, 8]), X mod 2 == 0).
false. % 5 is not even

% Practical example: Validate all fields in a form
```

```

validate_form(Form) :-
    forall(member(field(Name, Value), Form),
           validate_field(Name, Value)).

validate_field(age, Value) :- number(Value), Value > 0, Value < 150.
validate_field(name, Value) :- atom(Value), Value \= ''.
validate_field(email, Value) :- atom(Value), sub_atom(Value, _, _, _, '@').

% Print all solutions
print_all_solutions(Goal) :-
    forall(Goal, (write(Goal), nl)).

?- print_all_solutions(between(1, 5, X)).
between(1, 5, 1)
between(1, 5, 2)
between(1, 5, 3)
between(1, 5, 4)
between(1, 5, 5)
true.

% Check database consistency
check_parent_child_consistency :-
    forall(parent(P, C), child(C, P)).

```

ignore/1

Purpose: Always succeeds, whether the goal succeeds or fails.

When to use: Use for optional operations that shouldn't stop execution if they fail.

v3.6.0: a non-callable goal raises `type_error(callable, Goal)` (`ignore(1)` used to succeed silently); an unbound goal raises `instantiation_error`.

```

% ignore/1 tries to execute the goal but always succeeds

% Example: Optional logging
process_data(Data) :-
    validate(Data),
    ignore(log_to_file(Data)), % Don't fail if logging fails
    compute_result(Data, Result),
    display(Result).

% Optional cleanup
cleanup_resources :-
    ignore(close_file(F1)),
    ignore(close_connection(C1)),
    ignore(free_memory(M1)).

% Try to set optional configuration
initialize :-
    set_required_config,
    ignore(set_optional_feature(feature1)),
    ignore(set_optional_feature(feature2)),
    start_system.

```

setup_call_cleanup/3

Purpose: `setup_call_cleanup(:Setup, :Goal, :Cleanup)` - runs `Setup` once, then `Goal`, and runs `Cleanup` exactly once when `Goal` finishes (all solutions exhausted, failure, or an exception). If `Setup` fails or raises, `Cleanup` is not run.

When to use: Guaranteed resource cleanup (closing files/streams/connections) regardless of how the goal terminates.

```
% Cleanup runs whether the goal succeeds, fails, or raises:
read_first_line(File, Line) :-
    setup_call_cleanup(
        open(File, read, S),      % Setup: acquire the stream
        read_line_to_string(S, Line), % Goal
        close(S)).                % Cleanup: always closes S

?- setup_call_cleanup(true, member(X,[1,2]), writeln(done)).
done
X = 1 ;
X = 2.
```

Note: in this engine's eager-solution model, `Cleanup` runs after all of `Goal`'s solutions have been produced (or as soon as `Goal` fails/raises).

call_cleanup/2

Purpose: `call_cleanup(:Goal, :Cleanup)` - equivalent to `setup_call_cleanup(true, Goal, Cleanup)`.

```
?- call_cleanup(member(X,[a,b]), writeln(cleaned)).
cleaned
X = a ;
X = b.
```

6. Meta-Predicates

Meta-predicates operate on other predicates, enabling powerful programming techniques like finding all solutions, applying predicates to collections, and dynamic execution.

call/1

Purpose: Executes a goal constructed at runtime.

When to use: Use for dynamic predicate execution, higher-order programming, or when the predicate to execute is determined at runtime.

```
% Basic usage: Execute a goal
?- call(append([1], [2], X)).
X = [1, 2].

% Call with a variable goal
?- Goal = member(X, [a, b, c]), call(Goal).
Goal = member(a, [a, b, c]), X = a ;
Goal = member(b, [a, b, c]), X = b ;
Goal = member(c, [a, b, c]), X = c.

% Practical example: Generic filter predicate
filter([], _, []).
filter([H|T], Predicate, [H|Filtered]) :-
    call(Predicate, H),
    !,
    filter(T, Predicate, Filtered).
filter([_|T], Predicate, Filtered) :-
    filter(T, Predicate, Filtered).

% Define some test predicates
positive(X) :- number(X), X > 0.
even(X) :- number(X), 0 is X mod 2.

?- filter([1, -2, 3, -4, 5], positive, Result).
Result = [1, 3, 5].

?- filter([1, 2, 3, 4, 5, 6], even, Result).
Result = [2, 4, 6].

% Higher-order predicate: apply operation to list
map([], _, []).
map([H|T], Operation, [R|Results]) :-
    call(Operation, H, R),
    map(T, Operation, Results).

double(X, Y) :- Y is X * 2.
square(X, Y) :- Y is X * X.

?- map([1, 2, 3], double, Result).
Result = [2, 4, 6].

?- map([1, 2, 3, 4], square, Result).
Result = [1, 4, 9, 16].
```

$\wedge/2$ (existential quantification / goal call)

Purpose: $\wedge$ Goal is callable as an ordinary goal, equivalent to `call(Goal)` - the $\wedge$ prefix is simply ignored outside `bagof/3`/`setof/3` (SWI/SICStus/YAP-compatible). (v3.6.0)

When to use: Normally written inside `bagof/3`/`setof/3` to existentially quantify variables; direct calls matter when such a goal is built dynamically or passed around as data and eventually invoked.

```
% As an ordinary goal: V^Goal = call(Goal)
?- X^member(X, [a, b]).
X = a ;
X = b.

% Unchanged inside bagof/setof: ^ marks variables to ignore for grouping
?- bagof(Child, Parent^parent(Parent, Child), Children).
Children = [bob, liz, ann, pat, jim].

% The arithmetic ^ (integer power) is a separate evaluable functor:
?- X is 2 ^ 3.
X = 8.
```

An unbound `Goal` raises `instantiation_error`; a non-callable one raises `type_error(callable, Goal)`. Like `call/1`, the called goal is opaque to `cut`.

findall/3

Purpose: Collects all solutions to a goal into a list.

When to use: Use when you need all possible solutions collected together.

v3.6.0: the result argument is type-checked (ISO 8.10.1.3) - `findall(X, fail, a)` raises `type_error(list, a)`; variables, partial lists and proper lists remain legal.

```
% Syntax: findall(Template, Goal, List)
% Finds all solutions where Goal succeeds and collects Template values

% Basic example
?- findall(X, member(X, [a, b, c]), List).
List = [a, b, c].

% Collect specific information
?- findall(Name, person(Name, Age, _), Names).
Names = [john, mary, bob]. % Assuming those facts exist

% Practical example: Database queries
% Assume we have facts: employee(Name, Department, Salary)
employee(john, sales, 50000).
employee(mary, it, 60000).
employee(bob, sales, 55000).
employee(alice, it, 65000).

% Find all employees in IT
it_employees(Employees) :-
    findall(Name, employee(Name, it, _), Employees).

?- it_employees(E).
E = [mary, alice].

% Collect complex terms
high_earners(Earners) :-
    findall(
        emp(Name, Salary),
        (employee(Name, _, Salary), Salary > 55000),
        Earners
    ).

?- high_earners(E).
E = [emp(mary, 60000), emp(alice, 65000)].

% Calculate statistics
average_salary(Avg) :-
    findall(Salary, employee(_, _, Salary), Salaries),
    sum_list(Salaries, Total),
```

```

length(Salaries, Count),
Avg is Total / Count.

% Important: findall/3 returns [] if no solutions found
?- findall(X, member(X, []), List).
List = []. % Empty list, not failure

```

findall/4

Purpose: `findall(+Template, :Goal, -List, +Tail)` - like `findall/3`, but the collected list ends in `Tail` instead of `[]` (a difference list). (added v4.2.0)

When to use: to append the solutions of several goals without a second `append/3` pass.

```

?- findall(X, member(X, [1, 2]), L, t).
L = [1, 2|t].

?- findall(X, member(X, [1, 2]), L, [3]).
L = [1, 2, 3].

?- findall(X, fail, L, t).
L = t.

% Concatenating two collections in one pass
?- findall(X, member(X, [a, b]), L, T), findall(Y, member(Y, [c, d]), T, []).
L = [a, b, c, d].

```

An unbound `Goal` raises `instantiation_error`; a non-callable one raises `type_error(callable, Goal)`.

bagof/3

Purpose: Collects solutions like `findall/3` but respects variable bindings.

When to use: Use when you want solutions grouped by free variables.

v3.5.0: each collected solution is a renamed-apart fresh copy (result lists no longer alias caller variables); an unbound goal raises `instantiation_error` and a non-callable goal raises `type_error(callable, Goal)`.

v3.10.0, engine v4 (ISS-2025-0452): `bagof/3` and `setof/3` are native - the goal runs on the machine and the witness groups are handed out lazily, one per redo. The `^` handling, the variant-witness grouping and `setof/3`'s standard-order group enumeration are unchanged.

v3.6.0: witness grouping follows ISO 8.10.2.1 - solutions whose witness tuples are variants of each other merge into a single group (e.g. fresh clause variables in the witness no longer split groups), with the member tuples unified against the witness variables on emission.

```

% bagof/3 is like findall/3 but treats free variables differently

% With these facts:
parent(tom, bob).
parent(tom, liz).
parent(bob, ann).
parent(bob, pat).
parent(pat, jim).

% findall/3 collects all solutions
?- findall(Child, parent(_, Child), Children).
Children = [bob, liz, ann, pat, jim].

% bagof/3 groups by free variables
?- bagof(Child, parent(Parent, Child), Children).

```

```

Parent = bob, Children = [ann, pat] ;
Parent = pat, Children = [jim] ;
Parent = tom, Children = [bob, liz].

% Use ^ to existentially quantify variables (ignore them)
?- bagof(Child, Parent^parent(Parent, Child), Children).
Children = [bob, liz, ann, pat, jim]. % Like findall now

% Practical example: Group products by category
product(electronics, laptop, 1000).
product(electronics, phone, 500).
product(clothing, shirt, 30).
product(clothing, pants, 50).
product(food, apple, 2).
product(food, bread, 3).

products_by_category(Category, Products) :-
    bagof(Product, product(Category, Product, _), Products).

?- products_by_category(Cat, Prods).
Cat = electronics, Prods = [laptop, phone] ;
Cat = clothing, Prods = [shirt, pants] ;
Cat = food, Prods = [apple, bread].

```

setof/3

Purpose: Like bagof/3 but removes duplicates and sorts results.

When to use: Use when you want unique, sorted solutions.

v3.5.0: each collected solution is a renamed-apart fresh copy (result lists no longer alias caller variables); an unbound goal raises `instantiation_error` and a non-callable goal raises `type_error(callable, Goal)`.

v3.6.0: witness groups are enumerated in the standard order of terms (not in textual order) - `setof(X, member(X-Y, [a-10, b-2]), L)` yields `Y = 2, L = [b]` before `Y = 10, L = [a]` - and variant witnesses merge into a single group (ISO 8.10.2.1, also in bagof/3).

```

% setof/3 = bagof/3 + sort + remove duplicates

% With duplicate data:
likes(mary, food).
likes(mary, wine).
likes(john, wine).
likes(john, wine). % Duplicate
likes(bob, food).

% bagof/3 keeps duplicates
?- bagof(X, likes(john, X), Things).
Things = [wine, wine].

% setof/3 removes duplicates and sorts
?- setof(X, likes(john, X), Things).
Things = [wine].

% Collect all unique items liked
?- setof(Item, Person^likes(Person, Item), AllItems).
AllItems = [food, wine]. % Sorted and unique

% Practical example: Find unique skills
has_skill(john, python).
has_skill(john, java).
has_skill(mary, python).
has_skill(mary, python). % Duplicate
has_skill(bob, javascript).

```

```

has_skill(bob, java).

unique_skills(Skills) :-
    setof(Skill, Person^has_skill(Person, Skill), Skills).

?- unique_skills(S).
S = [java, javascript, python]. % Alphabetically sorted

% Find people with common skills
people_with_skill(Skill, People) :-
    setof(Person, has_skill(Person, Skill), People).

?- people_with_skill(python, P).
P = [john, mary]. % Sorted list of people

```

aggregate_all/3

Purpose: Aggregates values from all solutions to a goal using a specified aggregate template.

When to use: Use when you need to compute aggregate statistics (sum, count, max, min, bag, set) over all solutions to a goal in a single call.

v3.5.0: ISO error balls raised by Goal propagate unchanged (no longer wrapped or swallowed); an unbound or non-callable goal raises `instantiation_error` / `type_error(callable, _)`.

v3.10.0, *engine v4* (ISS-2025-0452): native over the machine's `findall`. New template forms `max(Value-Witness)` and `min(Value-Witness)`, which compare the numeric left-hand side and answer with the winning pair: `aggregate_all(max(V-W), member(V-W, [1-a, 3-b]), 3-b)`.

v3.6.0: `max(Expr)/min(Expr)` fail when the goal has no solutions and raise `type_error(number, T)` on a non-numeric solution (previously skipped silently); `sum(Expr)` accumulates integers exactly (BigInteger - no 64-bit overflow), with float contagion producing a genuine float (`sum` over `[1.5, 2.5]` is `4.0`); the empty sum is the integer `0`.

```

% Syntax: aggregate_all(Template, Goal, Result)
% Template can be: count, sum(Expr), max(Expr), min(Expr), bag(Expr), set(Expr)

% Count solutions
?- aggregate_all(count, member(_, [a, b, c]), Count).
Count = 3.

% Sum values
employee(john, 50000).
employee(mary, 60000).
employee(bob, 55000).

?- aggregate_all(sum(S), employee(_, S), Total).
Total = 165000.

% Collect into a sorted set
?- aggregate_all(set(X), member(X, [b, a, c, a, b]), Sorted).
Sorted = [a, b, c].

% Find maximum
?- aggregate_all(max(S), employee(_, S), Highest).
Highest = 60000.

% Practical example: Generate summary statistics
salary_report(Count, Total, Max, Min) :-
    aggregate_all(count, employee(_, _), Count),
    aggregate_all(sum(S), employee(_, S), Total),
    aggregate_all(max(S), employee(_, S), Max),
    aggregate_all(min(S), employee(_, S), Min).

```

```
?- salary_report(Count, Total, Max, Min).  
Count = 3, Total = 165000, Max = 60000, Min = 50000.
```

7. Input/Output

I/O predicates handle reading from and writing to files and streams.

Basic Output

write/1-2 and writeln/1-2

Purpose: Output terms to the current output stream.

When to use: Use for displaying results, debugging, or user interaction.

v2.8.2: Output is now **operator-aware** - consults the operator table for infix/prefix/postfix notation, list notation, curly braces, and '\$VAR'(N) rendering when numbervars(true). Output of `write(1+2)` is `1+2` (was `+(1,2)`); lists print as `[a,b,c]`; precedence-aware parens added when needed.

v3.5.0: stream-argument forms `write(Stream, Term)` and `writeln(Stream, Term)` added; `write/1` and `writeln/1` imply `numbervars(true)`, so '\$VAR'(0) prints as `A` (ISO 8.14.2); floats print in ISO syntax - lowercase exponent (`1.0e10`) and `inf/-inf/nan` spellings - so output re-reads as the same term.

```
% write/1 - Output without newline
?- write('Hello'), write(' '), write('World').
Hello World
true.

% writeln/1 - Output with newline
?- writeln('Hello World').
Hello World
true.

% Writing different types of terms
?- write(42), nl, write([a, b, c]), nl, write(person(john, 25)).
42
[a, b, c]
person(john, 25)
true.

% Practical example: Formatted output
display_person(person(Name, Age, City)) :-
    write('Name: '), writeln(Name),
    write('Age: '), writeln(Age),
    write('City: '), writeln(City).

?- display_person(person(john, 25, london)).
Name: john
Age: 25
City: london
true.

% Building formatted messages
error_message(Code, Message) :-
    write('ERROR '), write(Code), write(': '), writeln(Message).

?- error_message(404, 'File not found').
ERROR 404: File not found
true.
```

nl/0-1

Purpose: Outputs a newline character.

When to use: Use to control line breaks in output.

v3.5.0: the stream form `nl(Stream)` writes the newline to the given stream.

```
% Basic usage
?- write('Line 1'), nl, write('Line 2'), nl.
Line 1
Line 2
true.

% Creating formatted reports
print_header :-
    writeln('=' * 40), % Note: This would need special handling
    writeln('      REPORT TITLE'),
    writeln('=' * 40),
    nl.

% Spacing output
print_list([]).
print_list([H|T]) :-
    write(' - '), writeln(H),
    print_list(T).

?- print_list([apple, banana, cherry]).
- apple
- banana
- cherry
true.
```

print/1-2

Purpose: Output a term with `write` semantics (unquoted, operators honoured) and `numbervars(true)`. (added v3.5.0)

When to use: Use as the conventional "user-friendly output" predicate; equivalent to `write_term(Term, [numbervars(true)])`. The `portray/1` hook is not supported.

```
?- print(hello), nl.
hello
true.

?- print('$VAR'(0)), nl.
A
true.

% print/2 writes to a specific stream
?- open('out.txt', write, S), print(S, foo(1)), close(S).
true.
```

writeq/1-2

Purpose: Output a term with quoting - atoms that need quotes are quoted so the output can be read back.

```
?- writeq('hello world').
'hello world'
true.

?- writeq(Stream, Term). % stream form
```

v4.2.0: `writeq/2` resolves its stream exactly like `write/2` - it is captured by `with_output_to/2` and by the IDE console (it used to write straight to the process stdout through a static stream map), and it reports the ISO stream errors (`instantiation_error`, `domain_error(stream_or_alias, S)`, `existence_error(stream, S)`) instead of a bare evaluation error.

v3.5.0: quoting/spacing fixes - `writeq(-(1))` prints `- 1`, which re-reads as the same compound (it used to

print -1, a number); ', ', ' .' and comment-opening symbolic atoms such as '/*' are quoted; numbervars(true) is implied ('\$VAR'(0) prints as A); floats print in ISO syntax (1.0e10, inf, -inf, nan).

format/2 and format/3

Purpose: Formatted output driven by a directive string (~w, ~a, ~d, ~q, ~n, ~2f, ...), similar to C's printf.

When to use: Use for readable, formatted output instead of chains of write/1 calls.

```
?- format("Hello ~w, you are ~w years old~n", [john, 25]).
Hello john, you are 25 years old
true.

% format/3 with an output stream
?- open('out.txt', write, S), format(S, "~w~n", [data]), close(S).
true.

% format/3 with a capture sink
?- format(atom(A), '~2f', [3.14159]).
A = '3.14'.
```

v3.5.0: format/2, 3 now succeed as goals (output used to be produced with the goal then failing, killing any conjunction containing it); the format string may be an atom, a double-quoted string (the spelling produced by the default double_quotes=string flag), or a code/char list; format/3 honours its first argument - a stream alias/handle, or a capture sink atom(A) / string(S) / codes(C) / chars(C). ~w/ ~q imply numbervars(true).

v3.10.0 (ISS-2025-0452): new directive `~@` - the argument is a goal; it is called and everything it writes is spliced in at that point. ~p (portray) likewise calls portray/1. Both sub-goals run on the resolution engine, so on the v4 engine they honour the inference budget and the Stop interrupt.

```
?- format("[~@]~n", [write(inner)]).
[inner]
true.
```

v3.6.0: argument mismatches raise errors instead of being papered over - too few arguments for the directives raise a format error, ~d with a non-integer raises type_error(integer, Arg), and an unknown directive raises an error (previously echoed literally). The atom [] in the argument position is the **empty argument list** ([[]] passes the atom [] as a single argument; a non-list term still counts as one argument, SWI-style).

Basic Input

read/1, read/2

Purpose: Reads a Prolog term from input (must end with period).

- read/1 reads from current_input (stdin by default)
- read/2 reads from given stream: read(+Stream, -Term) (added v2.8.0)

```
% From file stream
?- open('data.pl', read, S), read(S, T), close(S).
T = foo(1, 2).
```

On end-of-file, both bind the term to the atom end_of_file.

v3.6.0: input is consumed up to the ISO **end token** (. followed by layout) rather than one line at a time - terms spanning several lines, several terms on one line, leading % and /* */ comments, and dots inside quotes/escapes/floats/graphic tokens are all handled; the stream position is preserved between calls, so the next read resumes right after the end token. Resolves LIM-029.

When to use: Use for reading structured Prolog data.

```
% Interactive reading
get_user_fact :-
    write('Enter a fact (end with .): '),
    read(Fact),
    assertz(Fact),
    write('Fact added: '), writeln(Fact).

% Example interaction:
?- get_user_fact.
Enter a fact (end with .): likes(john, pizza).
Fact added: likes(john, pizza)
true.

% Read and process commands
command_loop :-
    repeat,
    write('Command> '),
    read(Cmd),
    process_command(Cmd),
    Cmd = quit,
    !.

process_command(quit) :- writeln('Goodbye!').
process_command(help) :- writeln('Available commands: help, list, quit').
process_command(list) :- listing.
process_command(_) :- writeln('Unknown command. Type help for assistance.').
```

read_term/2-3

Purpose: Read a term with control options.

- `read_term(-Term, +Options)` reads from the current input
- `read_term(+Stream, -Term)` reads from a stream with default options
- `read_term(+Stream, -Term, +Options)` - the primary ISO 8.14.1 form (*added v3.5.0*)

Supported options: `variables(Vars)` (all variables of the term), `variable_names(Pairs)` (Name=Var pairs for the named variables), `singletons(Pairs)` (Name=Var pairs for singleton variables), `syntax_errors(error|fail|quiet)`, `term_position(Pos)` (v3.14.0: the '\$stream_position'(CharCount, LineCount, LinePosition, ByteCount) term for the FIRST character of the term just read - feed it to `stream_position_data/3`).

```
?- read_term(T, [variable_names(Vs)]).
|: foo(X, Y, X).
T = foo(_A, _B, _A),
Vs = ['X'=_A, 'Y'=_B].

% From a stream (v3.5.0+)
?- open('data.pl', read, S), read_term(S, T, []), close(S).
T = foo(1, 2).
```

v3.5.0: `read_term/2, 3` succeed as goals (previously the read happened but the goal failed). On end-of-file, Term is bound to the atom `end_of_file`.

v3.6.0: like `read/1, 2`, reads up to the ISO end token instead of one line - multi-line terms, several terms per line, and leading comments all work, with the stream position preserved between calls.

v3.14.0: reads through the stream's own decoder, so `read_term/2, 3`, `get_char/2` and `seek/4` all agree about where the stream is (before this each of `read/1, 2`, `read_term/2, 3` and `get_char/2` kept its own buffer). Input nested more than 1000 levels deep raises error(`resource_error(parser_nesting)`, _) rather than blowing the Java stack or failing silently.

Character I/O

get_char/1-2 and put_char/1-2

Purpose: Read or write single characters.

When to use: Use for character-by-character processing.

v3.5.0: the stream form `put_char(Stream, Char)` writes the character to the given stream; `get_char/1` honours the current input set by `set_input/1`.

```
% Read a single character
?- get_char(C).
% User types: a
C = 'a'.

% Write a single character
?- put_char('X').
X
true.

% Read password with echo off (conceptual example)
read_password(Password) :-
    read_password_chars([], Password).

read_password_chars(Acc, Password) :-
    get_char(C),
    (   C = '\n'
    -> reverse(Acc, Password)
    ;   put_char('*'), % Echo asterisk instead
        read_password_chars([C|Acc], Password)
    ).
```

get_code/1-2, peek_char/1-2 and peek_code/1-2

Purpose: Read the next character code (`get_code`), or look ahead at the next character (`peek_char`) / character code (`peek_code`) without consuming it.

```
?- get_code(C).           % from current input; C is the character code
?- get_code(Stream, C).  % from a specific stream (v3.5.0+)

?- peek_char(C).         % C is the next character (an atom), not consumed
?- peek_char(Stream, C).

?- peek_code(C).         % C is the next character code, not consumed
?- peek_code(Stream, C).
```

On end-of-file, `get_code/peek_code` unify the code with `-1` and `peek_char` unifies the character with `end_of_file`. (*stream-argument forms added v3.5.0*)

Byte I/O

get_byte/1-2 and put_byte/1-2

Purpose: Read or write single bytes for binary stream operations.

```
% Read a byte from current input
?- get_byte(B).

% Read a byte from a specific stream
```

```
?- get_byte(Stream, B).

% Write a byte to current output
?- put_byte(65). % Writes 'A'

% Write a byte to a specific stream
?- put_byte(Stream, 65).
```

peek_byte/1-2

Purpose: Non-consuming byte lookahead.

```
?- peek_byte(B). % Peek from current input
?- peek_byte(Stream, B). % Peek from specific stream
```

at_end_of_stream/0-1

Purpose: Test whether end of stream has been reached.

```
?- at_end_of_stream. % Current input
?- at_end_of_stream(Stream). % Specific stream
```

write_canonical/1-2

Purpose: Write term in canonical (functor) notation that can be read back.

```
?- write_canonical(1+2).
% Outputs: '+'(1,2)

?- write_canonical(Stream, Term).
```

write_term/2-3

Purpose: Write a term under explicit control options.

- `write_term(+Term, +Options)` writes to the current output
- `write_term(+Stream, +Term)` writes to a stream with default options
- `write_term(+Stream, +Term, +Options)` - the primary ISO 8.14.2 form (*added v3.5.0*)

Supported options (*complete since v3.14.0*):

Option	Meaning
<code>quoted(Bool)</code>	quote atoms and strings so the output re-reads as the same term
<code>ignore_ops(Bool)</code>	never use operator notation: <code>1-2</code> prints as <code>-(1,2)</code>
<code>numbervars(Bool)</code>	render '\$VAR' (N) as A, B, ..., Z, A1, ... (default false for <code>write_term</code> , true for <code>write/1</code> and <code>print/1</code>)
<code>max_depth(N)</code>	print at most N nesting levels; deeper structure prints as ... and a longer list as <code>[E1, E2 ...]</code> . 0 means unlimited
<code>portray(Bool)</code>	call the user's <code>portray/1</code> on every subterm; what it writes replaces the subterm
<code>cycles(Bool)</code>	render a rational tree as <code>@(Template, Substitutions) - X = f(X)</code> prints as <code>@(_S1, [_S1=f(_S1)])</code> . With <code>cycles(false)</code> a back edge prints as ..., so a cyclic term always terminates
<code>variable_names(Pairs)</code>	a list of Name=Var pairs; each listed variable prints as its name
<code>spacing(standard next_argument)</code>	<code>next_argument</code> puts a space after every argument separator: <code>f(a, b, c)</code>

An unrecognised option raises error(domain_error(write_option, Option), _).

```
?- write_term('hello world', [quoted(true)]).
'hello world'
true.

?- write_term('$VAR'(0), [numbervars(true)]).
A
true.

?- write_term(Stream, 1+2, [ignore_ops(true)]).
% Writes: +(1,2)
```

v3.5.0: write_term/2 succeeds as a goal (previously it printed and then failed); the three-argument stream form was added.

v3.14.0: the whole family (write/1,2, writeln/1,2, writeq/1,2, print/1,2, write_canonical/1,2, write_term/2,3, portray_clause/1,2, format ~w/~q/~p) renders through one writer, core.engine.v4.Writer. It is iterative (a 1 000 000-element list and a 1 000 000-deep structure print at the default JVM stack), cycle-safe, and reads the **engine's own** operator table, so an operator declared by :- op/3 in a consulted file is honoured when the term is written back.

```
?- write_term([1,2,3,4,5], [max_depth(3)]).
[1,2|...]

?- write_term(f(X,Y), [variable_names(['Foo'=X, 'Bar'=Y)])).
f(Foo,Bar)

?- X = f(X), write_term(X, [cycles(true)]).
@(_S1,[_S1=f(_S1)])
```

portray_clause/1-2

Purpose: Write a clause the way listing/1 does - quoted, variables numbered A, B, ..., one body goal per indented line, terminated by a full stop and a newline. (*added v3.14.0*)

```
?- portray_clause((p(X,Y) :- q(X), r(Y))).
p(A,B) :-
    q(A),
    r(B).
true.

?- portray_clause(fact).
fact.
true.
```

portray_clause(+Stream, +Clause) writes to Stream.

print_message/2

Purpose: print_message(+Kind, +Message) - report a message. (*added v3.14.0*)

An ISO error(Formal, Context) ball is rendered readably; anything else is written with quoted(true). error and warning go to user_error, silent prints nothing, every other kind goes to the current output. A user-defined message_hook/3 is **not** consulted (JProlog has no message-catalogue layer).

```
?- print_message(error, error(type_error(integer, abc), foo/1)).
ERROR: Type error: `integer' expected, found `abc' (foo/1)
true.

?- print_message(informational, hello).
% hello
```

```
true.
```

Stream properties and positions

stream_property/2

Purpose: `stream_property(?Stream, ?Property)` relates an open stream of **this engine** to its properties. Both arguments may be unbound; an unbound `Stream` enumerates every open stream.

Properties (*complete since v3.14.0*): `file_name(F)`, `mode(read|write|append)`, `input`, `output`, `alias(A)` (one solution per alias), `position(P)`, `end_of_stream(not|at|past)`, `eof_action(error|eof_code|reset)`, `reposition(true|false)`, `type(text|binary)`, `encoding(E)`, `line_count(N)`.

`P` is the opaque term `'$stream_position'` (`CharCount`, `LineCount`, `LinePosition`, `ByteCount`); take it apart with `stream_position_data/3` and feed it back to `set_stream_position/2`.

```
?- open('data.txt', read, S, [alias(input)]), stream_property(input, mode(M)).
M = read.
```

set_stream/2

Purpose: `set_stream(+Stream, +Property)` changes a property of an open stream. (*added v3.14.0*)

Accepted: `alias(A)` (adds another name for the stream), `type(text|binary)`, `eof_action(error|eof_code|reset)`, `encoding(E)`. Anything else raises `domain_error(stream_property, P)`.

stream_position_data/3

Purpose: `stream_position_data(+Field, +Position, ?Data)` extracts one field of a `'$stream_position'/4` term: `char_count`, `line_count`, `line_position` or `byte_count`. (*added v3.14.0*)

character_count/2, line_count/2, line_position/2

Purpose: The three counters of an open stream, without going through `stream_property/2`. (*added v3.14.0*)

`line_count/2` is 1-based (a freshly opened stream is on line 1); `line_position/2` is the 0-based column; `character_count/2` counts characters read or written so far. All three are exact on **text** streams, because the stream decodes through its own buffer.

```
?- open('two_lines.txt', read, S), get_char(S,_), get_char(S,_), get_char(S,_),
   get_char(S,_), get_char(S,_), get_char(S,_), get_char(S,C),
   line_count(S, L), line_position(S, LP).
C = w, L = 2, LP = 1.
```

current_stream/3

Purpose: `current_stream(?File, ?Mode, ?Stream)` enumerates the open **file** streams of this engine. (*added v3.14.0*)

seek/4, stream_position/2 and set_stream_position/2

`seek(+Stream, +Offset, +Method, -NewLocation)` with Method one of `bof`, `current`, `eof`; `stream_position(+Stream, -ByteOffset)`; `set_stream_position(+Stream, +Position)` where Position is a byte offset or a '\$stream_position'/4 term.

v3.14.0: repositioning now works on text streams. Before this release the stream handed out a `PushbackReader` over the raw file with its own 8 KB buffer, so a seek moved the file channel but not the reader - `get_char(S,C1)`, `seek(S,0,bof,_)`, `get_char(S,C2)` answered `C2 = e` after `C1 = h`. A reposition now flushes the stream's decode buffer and resets the decoder, and it recomputes the character/line counters.

Streams are per engine

v3.14.0: the stream table, its aliases and `current_input/current_output` belong to the Prolog instance (and, for the current streams, to the calling thread). A stream opened by one engine is invisible to another, and `set_output/1` on one thread does not redirect another thread's output. `open/3,4` unifies Stream with the canonical term '\$stream'(N); every stream argument also accepts an atom alias, the `stream_<id>` handle, and the reserved `user_input/user_output/user_error/current_input/current_output`.

char_conversion/2 and current_char_conversion/2

Purpose: Manage character conversion table used during term reading.

```
% Define conversion: 'A' reads as 'a'
?- char_conversion('A', a).

% Query current conversion
?- current_char_conversion('A', X).
X = a.

% Remove conversion (convert to self)
?- char_conversion('A', 'A').
```

File I/O

open/3 and close/1

Purpose: Open and close file streams.

When to use: Use for file-based I/O operations.

v3.5.0: failures raise `ISO_error/2` terms instead of plain-atom exceptions - opening a missing file raises `existence_error(source_sink, File)`, an invalid stream or alias raises `domain_error(stream_or_alias, S) / existence_error(stream, S)`. Also applies to `open/4`, which takes an options list (e.g. `alias(Name)`).

```
% Open file for reading
read_file_terms(Filename, Terms) :-
    open(Filename, read, Stream),
    read_all_terms(Stream, Terms),
    close(Stream).

read_all_terms(Stream, []) :-
    at_end_of_stream(Stream), !.
read_all_terms(Stream, [Term|Terms]) :-
    read(Stream, Term),
    read_all_terms(Stream, Terms).
```

```
% Open file for writing
write_to_file(Filename, Data) :-
    open(Filename, write, Stream),
    write(Stream, Data),
    write(Stream, '.'), % Add period for Prolog term
    nl(Stream),
    close(Stream).

% Append to file
append_to_log(Message) :-
    open('log.txt', append, Stream),
    get_time(Time),
    write(Stream, Time), write(Stream, ': '),
    writeln(Stream, Message),
    close(Stream).
```

set_input/1 and set_output/1

Purpose: Make a stream the current input / current output.

```
% Redirect input: read/1, get_char/1, get_code/1, ... now read from the stream
?- open('data.pl', read, S), set_input(S), read(T).
T = foo(1, 2).

% Redirect output: write/1, nl/0, format/2, ... now write to the stream
?- open('out.txt', write, S), set_output(S), writeln(hello).
true.
```

v3.5.0: these predicates actually redirect - the arity-1 input predicates honour the current input, and the output built-ins honour the current output (previously the setting was recorded but ignored).

tab/1-2

Purpose: Write N space characters to current output (tab/1) or to a given stream (tab(Stream, N), added v3.5.0).

```
?- write(hello), tab(5), write(world).
hello      world
```

with_output_to/2

Purpose: Execute Goal once, capturing everything it writes into Sink.

Sinks: atom(A) on every engine; string(S), codes(C) and chars(C) in addition on the v4 engine (v3.10.0, ISS-2025-0452), where a target that is none of those raises domain_error(output_sink, T). Goal runs as once/1: only the first solution's output is captured, and with_output_to/2 fails if Goal fails.

```
?- with_output_to(atom(X), write(hello)).
X = hello.

?- with_output_to(codes(C), write(ab)).      % v4
C = [97, 98].
```

v3.10.0, engine v4: the capture uses the thread-local output stream and restores whatever was installed (the IDE installs one per background solve), instead of swapping JVM-wide System.out and clearing it afterwards. See LIM-025.

8. Database Operations

Database operations allow dynamic modification of the Prolog knowledge base at runtime.

Understanding the Dynamic Database

Prolog's database can be modified during program execution:

- Add new facts and rules with `assert` family
- Remove facts and rules with `retract` family
- Query what's currently in the database with `listing`

`assert/1`, `asserta/1`, `assertz/1`

Purpose: Add facts or rules to the database.

- `assert/1` and `assertz/1` add at the end
- `asserta/1` adds at the beginning

When to use: Use to store runtime data, learn new information, or build dynamic knowledge bases.

v3.5.0: ISO validation - asserting an unbound term raises `instantiation_error`; a non-callable clause (`1, (1 :- true), (foo :- 7)`) raises `type_error(callable, _)`; asserting over a built-in raises `permission_error(modify, static_procedure, Name/Arity)`. Asserting a predicate implies declaring it dynamic (see `dynamic/1`).

```
% Add a simple fact
?- assertz(likes(john, pizza)).
true.

?- likes(john, X).
X = pizza.

% Add a rule
?- assertz((parent(X, Y) :- father(X, Y))).
true.

% Difference between asserta and assertz
?- assertz(color(red)).
true.
?- assertz(color(blue)).
true.
?- asserta(color(green)). % Added at beginning
true.

?- color(X).
X = green ; % First (added with asserta)
X = red ; % Second
X = blue. % Third

% Practical example: Learning system
learn_fact :-
    write('What did you learn? '),
    read(Fact),
    ( \+ Fact % Check if not already known
    -> assertz(Fact),
        writeln('I learned something new!')
    ; writeln('I already knew that!')
    ).

% Cache computation results
fibonacci_cached(N, Result) :-
    ( fib_cache(N, Result) % Check cache first
```

```
-> true
; calculate_fibonacci(N, Result),
  assertz(fib_cache(N, Result)) % Cache the result
).
```

retract/1 and retractall/1

Purpose: Remove facts or rules from the database.

- `retract/1` removes first matching clause
- `retractall/1` removes all matching clauses

When to use: Use to remove outdated information, clean up temporary data, or implement undo functionality.

v3.5.0: ISO validation - `retract/1` validates its argument (`instantiation_error` for an unbound term, `type_error(callable, _)` for a non-callable one, instead of an internal error); retracting clauses of a built-in raises `permission_error(modify, static_procedure, Name/Arity)`; `retractall/1` validates its argument the same way and implies declaring the predicate dynamic.

v3.6.0: `retract/1` is **re-executable on backtracking** on the v4 and v2 engines (ISO 8.9.3) - each redo retracts the next matching clause, so `findall(X, retract(p(X)), L)` drains the predicate one clause per solution; retractions of earlier solutions persist across backtracking. (The recursive engine, which retracted all matching clauses eagerly on the first call, was deleted in 4.0.0.)

```
% Remove a specific fact
?- assertz(temp(1)), assertz(temp(2)), assertz(temp(3)).
true.

?- retract(temp(2)).
true.

?- temp(X).
X = 1 ;
X = 3.

% Remove all matching facts
?- retractall(temp(_)).
true.

?- temp(X).
false. % All removed

% Practical example: Session management
login(User) :-
    retractall(current_user(_)), % Remove any existing login
    assertz(current_user(User)),
    write('Logged in as: '), writeln(User).

logout :-
    retractall(current_user(_)),
    writeln('Logged out').

% Update a fact (retract old, assert new)
update_score(Player, NewScore) :-
    retractall(score(Player, _)),
    assertz(score(Player, NewScore)).

% Temporary facts with cleanup
with_temp_fact(Fact, Goal) :-
    assertz(Fact),
    call(Goal),
    retract(Fact).
```

abolish/1

Purpose: Removes all clauses of a predicate.

When to use: Use to completely remove a predicate definition.

v3.5.0: abolishing a built-in raises `permission_error(modify, static_procedure, Name/Arity)`.

```
% Remove entire predicate
?- assertz(test(1)), assertz(test(2)), assertz((test(X) :- X > 10)).
true.

?- abolish(test/1). % Remove all clauses of test/1
true.

?- test(X).
ERROR: Undefined predicate: test/1

% Practical example: Reset game state
reset_game :-
    abolish(player_position/2),
    abolish(player_score/2),
    abolish(game_object/3),
    initialize_game.

% Clear all temporary predicates
cleanup_temp :-
    abolish(temp/1),
    abolish(cache/2),
    abolish(session/1).
```

dynamic/1

Purpose: Declares procedures dynamic, so calling them while they have no clauses fails silently instead of raising `existence_error(procedure, Name/Arity)`. (*callable as a goal since v3.5.0*)

When to use: Declare every predicate you plan to assert/retract at runtime, so querying it before any clause exists fails instead of raising an error (with the `unknown` flag at its default error).

```
% Directive form (in a consulted file)
:- dynamic(counter/1).

% Goal form (v3.5.0+): a predicate indicator ...
?- dynamic(score/2).
true.

?- score(X, Y).
false. % no existence_error: score/2 is dynamic

% ... a ','-sequence of indicators, or a list of indicators
?- dynamic((foo/1, bar/2)).
true.

?- dynamic([baz/0, quux/3]).
true.
```

Note: `assert/retractall` on a predicate imply declaring it dynamic; the `:- dynamic` directive is honoured during `consult` (it was a no-op before v3.5.0).

listing/0 and listing/1

Purpose: Display current database contents.

When to use: Use for debugging, inspecting dynamic predicates, or showing current state.

v4.2.0: **`listing/1` works.** Every earlier release registered one implementation for the name `listing`, the arity-0 one, which rejected any argument - `listing(foo/1)` raised `listing/0` takes no arguments despite being documented here since v2. Both arities are now separate v4 natives; a bare name lists every arity, `Name/Arity` lists one, and both print through the current output (so `with_output_to/2` and the IDE console capture them) with one full stop per clause instead of two.

```
% List everything
?- listing.
% Shows all user-defined predicates

% List specific predicate
?- listing(likes/2).
likes(john, pizza).
likes(mary, wine).

% List predicates by name (all arities)
?- listing(parent).
parent(tom, bob).
parent(bob, ann).
parent(X, Y) :- father(X, Y).
parent(X, Y) :- mother(X, Y).

% Practical example: Show current configuration
show_config :-
    writeln('Current Configuration:'),
    writeln('====='),
    listing(config/2),
    listing(option/1).
```

current_predicate/1

Purpose: Check or enumerate defined predicates.

When to use: Use to check if predicates exist or list available predicates.

```
% Check if predicate exists
?- current_predicate(member/2).
true.

?- current_predicate(nonexistent/3).
false.

% Enumerate predicates
?- current_predicate(Name/2).
Name = append ;
Name = member ;
Name = select ;
...

% Find all predicates with specific arity
?- findall(P, current_predicate(P/3), Predicates).
Predicates = [append/3, select/3, nth0/3, ...].

% Practical example: Safe predicate call
safe_call_predicate(Name, Arity, Args) :-
    current_predicate(Name/Arity),
    length(Args, Arity),
    Goal =.. [Name|Args],
```

```
    call(Goal).
safe_call_predicate(Name, Arity, _) :-
    \+ current_predicate(Name/Arity),
    write('Undefined predicate: '), write(Name/Arity), nl,
    fail.
```

9. Atom and String Operations

These predicates manipulate atoms (symbolic constants) and strings.

v3.6.0 (SWI-style text interop): the `atom_*` predicates accept strings (`atom_length("abc", 3)`, `atom_codes(X, "abc")` gives the atom `abc`), and `string_length/2`, `string_chars/2`, `string_concat/3` accept atoms (`string_concat(a, b, S)` gives the string `S = "ab"` - the result stays a string).

atom_length/2

Purpose: Determines the length of an atom.

When to use: Use for validation, formatting, or string processing.

v3.6.0: ISO error terms - an unbound atom raises `instantiation_error`, a non-atom raises `type_error(atom, A)`, a non-integer length raises `type_error(integer, L)` and a negative length raises `domain_error(not_less_than_zero, L)`. Also accepts a string first argument (SWI interop).

```
% Get length of atom
?- atom_length(hello, Len).
Len = 5.

?- atom_length('Hello World', Len).
Len = 11.

% Validate input length
validate_username(Username) :-
    atom(Username),
    atom_length(Username, Len),
    Len >= 3,
    Len <= 20,
    !.
validate_username(_) :-
    writeln('Username must be 3-20 characters'),
    fail.

% Pad atom to specific length
pad_atom(Atom, TargetLen, PadChar, Padded) :-
    atom_length(Atom, CurrentLen),
    PadCount is TargetLen - CurrentLen,
    (   PadCount <= 0
    -> Padded = Atom
    ;   create_padding(PadCount, PadChar, Padding),
        atom_concat(Atom, Padding, Padded)
    ).
```

atom_concat/3

Purpose: Concatenates atoms or splits an atom.

When to use: Use for building identifiers, messages, or parsing.

v3.6.0: ISO error terms - all arguments unbound raises `instantiation_error`, a non-atomic argument raises a proper error (`type_error(atom, Culprit)`, `_`) ball naming the culprit (was a plain-text exception). Also accepts string arguments (SWI interop).

```
% Concatenate atoms
?- atom_concat(hello, world, Result).
Result = helloworld.

?- atom_concat('Hello ', 'World', Result).
```

```

Result = 'Hello World'.

% Split atom (finding possible splits)
?- atom_concat(Prefix, Suffix, helloworld).
Prefix = '', Suffix = helloworld ;
Prefix = h, Suffix = elloworld ;
Prefix = he, Suffix = lloworld ;
...

% Find specific prefix/suffix
?- atom_concat(hello, Suffix, helloworld).
Suffix = world.

?- atom_concat(Prefix, world, helloworld).
Prefix = hello.

% Practical example: Build file paths
build_path(Dir, File, Path) :-
    atom_concat(Dir, '/', Temp),
    atom_concat(Temp, File, Path).

?- build_path('/home/user', 'file.txt', Path).
Path = '/home/user/file.txt'.

% Generate unique identifiers
generate_id(Base, Counter, ID) :-
    atom_number(CounterAtom, Counter),
    atom_concat(Base, '_', Temp),
    atom_concat(Temp, CounterAtom, ID).

?- generate_id(user, 42, ID).
ID = user_42.

```

sub_atom/5

Purpose: Extracts substrings from atoms.

When to use: Use for parsing, pattern matching, or string manipulation.

v3.10.0, engine v4 (ISS-2025-0453): `sub_atom/5` and `sub_string/5` are lazy generators - one candidate per redo instead of the full $O(n^2)$ cross-product materialised up front, so `once(sub_atom(LongAtom, _, _, _, S))` is cheap. This also fixes a hang: with an **empty** `SubAtom` (`sub_atom(abc, B, L, A, '')`) the default engine loops forever, because `String.indexOf("", Idx)` stops advancing past the end of the atom; on v4 the four positions are enumerated once.

```

% Syntax: sub_atom(+Atom, ?Before, ?Length, ?After, ?SubAtom)
% Before: characters before the subatom
% Length: length of the subatom
% After: characters after the subatom

% Extract substring
?- sub_atom(helloworld, 5, 5, 0, Sub).
Sub = world.

?- sub_atom(helloworld, 0, 5, _, Sub).
Sub = hello.

% Find position of substring
?- sub_atom(helloworld, Before, 5, After, world).
Before = 5, After = 0.

% Check if atom contains substring
contains_substring(Atom, Sub) :-
    sub_atom(Atom, _, _, _, Sub).

```

```
?- contains_substring('hello world', world).
true.

% Practical example: Extract file extension
get_extension(Filename, Ext) :-
    sub_atom(Filename, Before, 1, After, '.'),
    After > 0, % Ensure dot is not at the end
    !,
    sub_atom(Filename, _, After, 0, Ext).

?- get_extension('document.pdf', Ext).
Ext = pdf.

% Parse email address
parse_email(Email, User, Domain) :-
    sub_atom(Email, Before, 1, After, '@'),
    sub_atom(Email, 0, Before, _, User),
    sub_atom(Email, _, After, 0, Domain).

?- parse_email('john@example.com', U, D).
U = john, D = 'example.com'.
```

atom_chars/2 and atom_codes/2

Purpose: Convert between atoms and character/code lists.

When to use: Use for character-level processing or encoding conversions.

v3.6.0: ISO error terms - with the atom side unbound, a partial list or a list with an unbound element raises `instantiation_error`; a bad element raises `type_error(character, E)` (`atom_chars`) or `representation_error(character_code)` (`atom_codes`). A number first argument stringifies (`atom_chars(42, L)` gives `L = ['4', '2']`, SWI/GNU behavior). Both also accept a string first argument (SWI interop).

```
% atom_chars/2 - Convert to/from character list
?- atom_chars(hello, Chars).
Chars = [h, e, l, l, o].

?- atom_chars(Atom, [h, e, l, l, o]).
Atom = hello.

% atom_codes/2 - Convert to/from ASCII codes
?- atom_codes(hello, Codes).
Codes = [104, 101, 108, 108, 111].

?- atom_codes(Atom, [72, 69, 76, 76, 79]).
Atom = 'HELLO'.

% Practical example: Reverse an atom
reverse_atom(Atom, Reversed) :-
    atom_chars(Atom, Chars),
    reverse(Chars, RevChars),
    atom_chars(Reversed, RevChars).

?- reverse_atom(hello, Rev).
Rev = olleh.

% Simple encryption (Caesar cipher)
caesar_cipher(Text, Shift, Encrypted) :-
    atom_codes(Text, Codes),
    maplist(shift_code(Shift), Codes, ShiftedCodes),
    atom_codes(Encrypted, ShiftedCodes).
```

```

shift_code(Shift, Code, Shifted) :-
    Shifted is Code + Shift.

?- caesar_cipher(abc, 1, Encrypted).
Encrypted = bcd.

```

number_chars/2 and number_codes/2

Purpose: Convert between numbers and character/code lists.

```

?- number_chars(42, Chars).
Chars = ['4', '2'].

?- number_chars(N, ['3', '.', '1', '4']).
N = 3.14.

?- number_codes(255, Codes).
Codes = [50, 53, 53].

```

v3.5.0: arbitrarily large integers round-trip exactly (values beyond 64-bit precision were previously corrupted silently).

v3.6.0: type-faithful floats - float-syntax text yields a float (number_chars(X, ['1', '.', '0'])) gives X = 1.0, not the integer 1) and floats keep float syntax on output (number_codes(1.0, L) gives "1.0"). ISO 0x/0o/0b radix notation and 0'c char-code constants are accepted; Java-only spellings (Infinity, NaN, 1f, '.5', '3.') raise syntax_error(illegal_number). ISO error terms otherwise: both sides unbound raises instantiation_error, a non-number first argument raises type_error(number, N).

atom_number/2

Purpose: Converts between atoms and numbers.

When to use: Use for parsing numeric input or formatting numbers.

v2.8.2: accepts hex (0xFF), binary (0b1010), octal (0o77) prefixes when parsing atom->number.

v3.5.0: arbitrarily large integers convert exactly in both directions (no more silent 64-bit corruption).

v3.6.0: float syntax is preserved in the number->atomdirection - atom_number(A, 123.0) gives A = '123.0' (previously '123'); the atom->numberdirection is type-faithful (atom_number('1.0', X) gives the float 1.0) and rejects Java-only spellings ('Infinity', 'NaN', '1f') with syntax_error(illegal_number).

```

% Convert atom to number
?- atom_number('42', N).
N = 42.

?- atom_number('3.14', N).
N = 3.14.

% Hex / binary / octal (v2.8.2+)
?- atom_number('0xFF', N).
N = 255.

?- atom_number('0b1010', N).
N = 10.

?- atom_number('0o77', N).
N = 63.

% Convert number to atom
?- atom_number(A, 42).
A = '42'.

```

```
% Practical example: Parse numeric input
parse_number(Input, Number) :-
    atom(Input),
    atom_number(Input, Number),
    !.
parse_number(Input, _) :-
    write('Invalid number: '), write(Input), nl,
    fail.

% Calculate from string expression
calculate_string(Expr, Result) :-
    atom_number(Expr, Result). % Works for simple numbers

?- calculate_string('123', R).
R = 123.

% Format numbers with specific precision (conceptual)
format_currency(Amount, Formatted) :-
    Round is round(Amount * 100) / 100,
    atom_number(Amount, Round),
    atom_concat('$', AtomAmount, Formatted).

?- format_currency(42.3456, F).
F = '$42.35'.
```

String Processing

split_string/4

Purpose: Splits strings by separators with padding removal.

When to use: Use for parsing CSV, processing user input, or tokenization.

```
% Syntax: split_string(+String, +Separators, +PadChars, -SubStrings)

% Basic splitting
?- split_string("apple,banana,cherry", ",", "", L).
L = [apple, banana, cherry].

% Multiple separators
?- split_string("apple;banana,cherry:date", ";,:", "", L).
L = [apple, banana, cherry, date].

% Remove padding (spaces)
?- split_string(" apple , banana , cherry ", ",", " ", L).
L = [apple, banana, cherry].

% Practical example: Parse CSV line
parse_csv_line(Line, Fields) :-
    split_string(Line, ",", "\t", Fields).

?- parse_csv_line("John, 25, London", Fields).
Fields = ['John', '25', 'London'].

% Parse configuration line
parse_config(Line, Key, Value) :-
    split_string(Line, "=", " ", [Key, Value]).

?- parse_config("username = john_doe", K, V).
K = username, V = john_doe.

% Tokenize sentence
tokenize(Sentence, Tokens) :-
```

```
split_string(Sentence, " .,!?", " ", Tokens).

?- tokenize("Hello, world! How are you?", T).
T = ['Hello', 'world', 'How', 'are', 'you'].
```

atomic_list_concat/2, atomic_list_concat/3

Purpose: Joins atoms (optionally with separator) or splits by separator.

- `atomic_list_concat/2` (v2.8.2+): joins without separator
- `atomic_list_concat/3`: joins with separator; reverse mode splits

When to use: Use for building formatted strings or parsing.

```
% Join atoms with separator
?- atomic_list_concat([hello, world], ' ', Result).
Result = 'hello world'.

?- atomic_list_concat([one, two, three], '-', Result).
Result = 'one-two-three'.

% Split by separator
?- atomic_list_concat(List, '-', 'one-two-three').
List = [one, two, three].

% Join without separator (use atomic_list_concat/2)
?- atomic_list_concat([hello, world], Result).
Result = helloworld.

% Practical example: Build SQL query
build_insert(Table, Values, Query) :-
    atomic_list_concat(Values, ', ', ValueString),
    atomic_list_concat(['INSERT INTO ', Table, ' VALUES (', ValueString, ')'], Query).

?- build_insert(users, ['John', 25, 'London'], Q).
Q = 'INSERT INTO users VALUES (John, 25, London)'.

% Create formatted message
format_message(Template, Args, Message) :-
    atomic_list_concat(Args, ', ', ArgString),
    atomic_list_concat([Template, ': ', ArgString], Message).

?- format_message('Error', [404, 'Not Found'], Msg).
Msg = 'Error: 404, Not Found'.
```

string_to_atom/2

Purpose: Convert between string and atom representations. Bidirectional.

```
?- string_to_atom(hello, X).
X = hello.

% (-, +) mode binds a string (v3.6.0)
?- string_to_atom(S, foo).
S = "foo".
```

v3.6.0: the `(-, +)` mode binds a **string** - `string_to_atom(S, foo)` gives `S = "foo"` with `string(S)` true and `atom(S)` false (it used to bind an atom, making the mode a no-op). The `(+, -)` direction is unchanged.

number_to_atom/2, atom_to_number/2

Purpose: Convert between number and atom representations.

```
?- number_to_atom(42, X).  
X = '42'.  
  
?- atom_to_number('3.14', X).  
X = 3.14.
```

string_code/3

Purpose: Get the character code at a 1-based index in a string/atom.

```
?- string_code(1, hello, X).  
X = 104. % ASCII code for 'h'
```

10. Character Processing

Character processing predicates work with individual characters and their properties.

char_code/2

Purpose: Converts between characters and their numeric codes.

When to use: Use for character encoding, ASCII operations, or character arithmetic.

v3.6.0: ISO error terms (8.16.6.3) - both arguments unbound raises `instantiation_error`, a first argument that is not a one-char atom raises `type_error(character, C)`, a non-integer code raises `type_error(integer, Code)`, and an integer outside the Unicode range raises `representation_error(character_code)` (previously these failed silently).

```
% Character to code
?- char_code(a, Code).
Code = 97.

?- char_code('A', Code).
Code = 65.

% Code to character
?- char_code(Char, 65).
Char = 'A'.

?- char_code(Char, 97).
Char = a.

% Practical example: Check character type
is_uppercase(Char) :-
    char_code(Char, Code),
    Code >= 65,
    Code <= 90.

is_lowercase(Char) :-
    char_code(Char, Code),
    Code >= 97,
    Code <= 122.

is_digit(Char) :-
    char_code(Char, Code),
    Code >= 48,
    Code <= 57.

?- is_uppercase('A').
true.

?- is_digit('5').
true.

% Convert case
to_uppercase(Lower, Upper) :-
    is_lowercase(Lower),
    char_code(Lower, LowerCode),
    UpperCode is LowerCode - 32,
    char_code(Upper, UpperCode).

?- to_uppercase(a, U).
U = 'A'.
```

char_type/2

Purpose: Classifies characters into categories.

When to use: Use for parsing, validation, or text processing.

```
% Check character type
?- char_type(a, alpha).
true.

?- char_type('5', digit).
true.

?- char_type(' ', space).
true.

% Find all types of a character
?- char_type('A', Type).
Type = alnum ;    % Alphanumeric
Type = alpha ;   % Alphabetic
Type = ascii ;   % ASCII character
Type = upper ;   % Uppercase
...

% Character type categories:
% - alnum: alphanumeric
% - alpha: alphabetic
% - ascii: ASCII character
% - cntrl: control character
% - digit: decimal digit
% - graph: graphical character
% - lower: lowercase letter
% - print: printable character
% - punct: punctuation
% - space: whitespace
% - upper: uppercase letter
% - xdigit: hexadecimal digit

% Practical example: Validate password
validate_password(Password) :-
    atom_chars(Password, Chars),
    length(Chars, Len),
    Len >= 8,
    member(Upper, Chars), char_type(Upper, upper),
    member(Lower, Chars), char_type(Lower, lower),
    member(Digit, Chars), char_type(Digit, digit),
    !.

% Extract specific character types
extract_digits(Text, Digits) :-
    atom_chars(Text, Chars),
    findall(D, (member(D, Chars), char_type(D, digit)), Digits).

?- extract_digits('abc123def456', D).
D = ['1', '2', '3', '4', '5', '6'].

% Remove non-alphabetic characters
clean_text(Input, Cleaned) :-
    atom_chars(Input, Chars),
    findall(C, (member(C, Chars), char_type(C, alpha)), CleanChars),
    atom_chars(Cleaned, CleanChars).

?- clean_text('hello123world!', C).
C = helloworld.
```

upcase_atom/2 and downcase_atom/2

Purpose: Convert atom case.

When to use: Use for normalization, case-insensitive comparisons, or formatting.

```
% Convert to uppercase
?- upcase_atom(hello, Upper).
Upper = 'HELLO'.

?- upcase_atom('Hello World', Upper).
Upper = 'HELLO WORLD'.

% Convert to lowercase
?- downcase_atom('HELLO', Lower).
Lower = hello.

?- downcase_atom('Hello World', Lower).
Lower = 'hello world'.

% Practical example: Case-insensitive comparison
equal_ignore_case(Atom1, Atom2) :-
    downcase_atom(Atom1, Lower1),
    downcase_atom(Atom2, Lower2),
    Lower1 = Lower2.

?- equal_ignore_case('Hello', 'HELLO').
true.

?- equal_ignore_case('Hello', 'World').
false.

% Normalize input
normalize_command(Input, Command) :-
    downcase_atom(Input, Lower),
    atom_string(Lower, TrimmedString),
    atom_string(Command, TrimmedString).

% Create formatted headers
format_header(Text, Header) :-
    upcase_atom(Text, Upper),
    atom_concat('== ', Upper, Temp),
    atom_concat(Temp, ' ==', Header).

?- format_header('section title', H).
H = '== SECTION TITLE =='.
```

11. DCG (Grammar) Predicates

Definite Clause Grammars provide a high-level notation for parsing and generating sequences.

Understanding DCGs

DCGs are a syntactic extension for writing parsers and generators:

- DCG rules use `-->` instead of `:-`
- Automatically handle list processing with difference lists
- Can embed Prolog goals using `{}`

```
% Simple DCG rule
noun --> [cat].
noun --> [dog].

% Is translated internally to:
% noun([cat|Rest], Rest).
% noun([dog|Rest], Rest).
```

phrase/2 and phrase/3

Purpose: Executes DCG rules for parsing or generation. `phrase(Body, List)` parses the whole list; `phrase(Body, List, Rest)` unifies `Rest` with the unparsed remainder.

When to use: Use to parse input with grammar rules or generate valid sequences.

v3.5.0: the first argument may be any DCG body, not just a non-terminal - `(A, B)`, `(A ; B)`, `(A -> B)`, `\+ A`, `!, {Goal}`, terminal lists `[a, b]` and `[]` are all translated correctly; a non-list second/third argument raises `type_error(list, _)` and a non-callable body raises `type_error(callable, _)` (previously these failed silently).

v3.10.0, engine v4 (ISS-2025-0451): `phrase/2, 3` is native - the grammar body is translated and the resulting goal is pushed onto the machine's goal stack instead of being solved by a nested recursive sub-solver. A DCG over a **1 000 000-token list** parses at the default JVM stack (the default engine raises `resource_error(stack_overflow)` on the same query even with `-Xss4m`), and the inference budget and the Stop interrupt now fire *inside* a parse. The ISO 13211-3 error clauses are unchanged.

v3.6.0: `phrase/3` with two free variables (e.g. `phrase(nt, [a|T], R)`) no longer raises a spurious `representation_error(cyclic_term)` on the default engine - it now answers with the expected var-var binding; real cyclic-term (rational-tree) protection is unaffected.

```
% Define a simple grammar
article --> [the].
article --> [a].
noun --> [cat].
noun --> [dog].
verb --> [chases].
verb --> [sees].

noun_phrase --> article, noun.
verb_phrase --> verb, noun_phrase.
sentence --> noun_phrase, verb_phrase.

% Parse with phrase/2
?- phrase(noun, [cat]).
true.

?- phrase(noun_phrase, [the, dog]).
true.
```

```

?- phrase(sentence, [the, cat, chases, a, dog]).
true.

% Generate valid sentences
?- phrase(sentence, S).
S = [the, cat, chases, the, cat] ;
S = [the, cat, chases, the, dog] ;
S = [the, cat, chases, a, cat] ;
...

% Practical example: Number parser
digit(D) --> [D], { char_type(D, digit) }.

digits([D|Ds]) --> digit(D), digits(Ds).
digits([D]) --> digit(D).

number(N) -->
    digits(Ds),
    { atom_chars(Atom, Ds), atom_number(Atom, N) }.

?- phrase(number(N), ['1', '2', '3']).
N = 123.

% phrase/3 returns the remainder; the body can be any DCG body (v3.5.0+)
?- phrase((article, noun), [the, dog, runs], Rest).
Rest = [runs].

```

enhanced_phrase/2 and enhanced_phrase/3

Purpose: Enhanced DCG parsing with ISO/IEC DTS 13211-3 compliance.

When to use: Use for advanced parsing with complex grammars.

```

% Enhanced parsing with complex control structures
expression --> term, expression_rest.
expression_rest --> ['+'], term, expression_rest.
expression_rest --> ['-'], term, expression_rest.
expression_rest --> [].

term --> factor, term_rest.
term_rest --> ['*'], factor, term_rest.
term_rest --> ['/'], factor, term_rest.
term_rest --> [].

factor --> ['('], expression, [')'].
factor --> number.

% Parse arithmetic expression
?- enhanced_phrase(expression, ['2', '*', '(', '3', '+', '4', ')']).
true.

% With remainder extraction
?- enhanced_phrase(expression, ['2', '+', '3', ';', 'rest'], Rest).
Rest = [';', 'rest'].

% Practical example: Configuration parser
config_line -->
    key(K),
    spaces,
    ['='],
    spaces,
    value(V),
    { assertz(config(K, V)) }.

key(K) --> identifier(K).

```

```

value(V) --> quoted_string(V).
value(V) --> identifier(V).
value(V) --> number(V).

spaces --> [' '], spaces.
spaces --> [].

identifier(ID) -->
    [C],
    { char_type(C, alpha) },
    id_rest(Cs),
    { atom_chars(ID, [C|Cs]) }.

id_rest([C|Cs]) -->
    [C],
    { char_type(C, alnum) },
    id_rest(Cs).
id_rest([]) --> [].

```

phrase_with_options/4

Purpose: DCG parsing with advanced control options.

When to use: Use when you need parsing with error handling, debugging, or depth limits.

```

% Parse with options
Options = [
    syntax_errors(fail),    % How to handle syntax errors
    max_depth(100),        % Maximum recursion depth
    debug(true),           % Enable debug output
    trace(false),          % Disable tracing
].

% Safe parsing with depth limit
safe_parse(Rule, Input, Result) :-
    phrase_with_options(
        Rule,
        Input,
        [],
        [max_depth(1000), syntax_errors(fail)]
    ),
    Result = success.
safe_parse(_, _, failed).

% Practical example: JSON parser with error handling
json_value --> json_object.
json_value --> json_array.
json_value --> json_string.
json_value --> json_number.
json_value --> json_boolean.
json_value --> json_null.

parse_json(Input, Result) :-
    phrase_with_options(
        json_value,
        Input,
        [],
        [syntax_errors(error), max_depth(100)]
    ),
    Result = valid_json.

% Debug parsing issues
debug_parse(Grammar, Input) :-
    phrase_with_options(
        Grammar,

```

```

    Input,
    Rest,
    [debug(true), trace(true)]
),
write('Remaining input: '), writeln(Rest).

```

call_dcg/3

Purpose: Calls DCG rules with explicit difference lists.

When to use: Use for meta-programming with DCGs or custom parsing control.

```

% Direct DCG call with explicit lists
?- call_dcg(noun, [cat, runs], Rest).
Rest = [runs].

% Compose DCG rules dynamically
parse_sequence([], Input, Input).
parse_sequence([Rule|Rules], Input, Output) :-
    call_dcg(Rule, Input, Temp),
    parse_sequence(Rules, Temp, Output).

?- parse_sequence([article, noun, verb], [the, cat, runs], Rest).
Rest = [].

% Practical example: Dynamic grammar selection
parse_by_type(number, Input, Rest) :-
    call_dcg(number_parser, Input, Rest).
parse_by_type(word, Input, Rest) :-
    call_dcg(word_parser, Input, Rest).
parse_by_type(symbol, Input, Rest) :-
    call_dcg(symbol_parser, Input, Rest).

% Meta-DCG: Apply DCG rule multiple times
repeat_dcg(_, 0, Input, Input) :- !.
repeat_dcg(Rule, N, Input, Output) :-
    N > 0,
    call_dcg(Rule, Input, Temp),
    N1 is N - 1,
    repeat_dcg(Rule, N1, Temp, Output).

?- repeat_dcg(digit, 3, ['1', '2', '3', '4'], Rest).
Rest = ['4'].

```

dcg_translate_rule/2

Purpose: Translates DCG rules to standard Prolog clauses.

When to use: Use for understanding DCG transformation or meta-programming.

```

% See how DCG rules are translated
?- dcg_translate_rule((noun --> [cat]), Clause).
Clause = (noun([cat|S], S)).

?- dcg_translate_rule((noun_phrase --> article, noun), Clause).
Clause = (noun_phrase(S0, S) :- article(S0, S1), noun(S1, S)).

% Complex translation with embedded goals
?- dcg_translate_rule(
    (number(N) --> digits(D), { atom_number(D, N) })),
    Clause
).
Clause = (number(N, S0, S) :-
    digits(D, S0, S),

```

```
    atom_number(D, N)
).

% Practical example: Generate parser predicates
generate_parser(Grammar, Predicates) :-
    findall(Pred,
        (member(Rule, Grammar),
         dcg_translate_rule(Rule, Pred)),
        Predicates).

% Analyze grammar complexity
analyze_dcg(Rule) :-
    dcg_translate_rule(Rule, Clause),
    Clause = (Head :- Body),
    functor(Head, Name, Arity),
    write('DCG rule: '), write(Name),
    write(' with arity '), write(Arity), nl,
    write('Translates to: '), write(Clause), nl.
```

12. Exception Handling

Exception handling provides robust error management and recovery mechanisms.

throw/1

Purpose: Throws an exception to be caught by an enclosing catch/3.

When to use: Use to signal errors or exceptional conditions.

```
% Throw simple exception
divide(_, 0, _) :-
    throw(division_by_zero).
divide(X, Y, Result) :-
    Result is X / Y.

?- divide(10, 0, R).
ERROR: Uncaught exception: division_by_zero

% Throw ISO error terms
validate_positive(X) :-
    (   var(X)
    -> throw(error(instantiation_error, validate_positive/1))
    ;   \+ number(X)
    -> throw(error(type_error(number, X), validate_positive/1))
    ;   X <= 0
    -> throw(error(domain_error(positive_number, X), validate_positive/1))
    ;   true
    ).

% Practical example: File operations with error handling
read_file_safe(Filename, Content) :-
    (   \+ exists_file(Filename)
    -> throw(error(existence_error(file, Filename), read_file_safe/2))
    ;   \+ access_file(Filename, read)
    -> throw(error(permission_error(read, file, Filename), read_file_safe/2))
    ;   read_file_to_string(Filename, Content)
    ).

% Custom error types
process_data(Data) :-
    (   validate_format(Data)
    -> transform_data(Data)
    ;   throw(custom_error(invalid_format, Data))
    ).
```

catch/3

Purpose: Catches exceptions thrown by goals.

When to use: Use to handle errors gracefully and implement recovery strategies.

```
% Syntax: catch(+Goal, +Catcher, +Recovery)

% Basic exception catching
safe_divide(X, Y, Result) :-
    catch(
        Result is X / Y,
        error(evaluation_error(zero_divisor), _),
        (write('Division by zero!'), nl, fail)
    ).

?- safe_divide(10, 0, R).
```

```

Division by zero!
false.

% Catch specific exceptions
process_with_retry(Data, Result) :-
    catch(
        process_data(Data, Result),
        network_error(timeout),
        (   writeln('Network timeout, retrying...'),
            sleep(1),
            process_with_retry(Data, Result)
        )
    ).

% Multiple exception types
robust_operation(Input, Output) :-
    catch(
        dangerous_operation(Input, Output),
        Error,
        handle_error(Error)
    ).

handle_error(error(type_error(_, _), _)) :-
    writeln('Type error: Invalid input type').
handle_error(error(domain_error(_, _), _)) :-
    writeln('Domain error: Value out of range').
handle_error(error(existence_error(_, _), _)) :-
    writeln('Existence error: Resource not found').
handle_error(Error) :-
    write('Unexpected error: '), writeln(Error).

% Practical example: Transaction with rollback
transaction(Actions, Result) :-
    catch(
        (   begin_transaction,
            perform_actions(Actions),
            commit_transaction,
            Result = success
        ),
        Error,
        (   rollback_transaction,
            write('Transaction failed: '), writeln(Error),
            Result = failed(Error)
        )
    ).

% Cleanup with exception safety
with_resource(Resource, Goal) :-
    acquire_resource(Resource),
    catch(
        call(Goal),
        Error,
        (release_resource(Resource), throw(Error))
    ),
    release_resource(Resource).

```

halt/0 and halt/1

Purpose: Terminates the Prolog processor. `halt/0` is equivalent to `halt(0)`; `halt(Code)` exits with the given exit code.

When to use: Use to exit the program, typically after fatal errors or completion.

v3.5.0: `halt/0`, `halt/1` actually terminate the processor - the CLI process exits with the given exit code, :- `halt` aborts a consult, and the IDE ends the run session gracefully (previously `halt` was effectively a

no-op).

```
% Exit with success
?- halt(0).
% Program terminates with exit code 0

% Exit with error code
fatal_error(Message) :-
    write('FATAL ERROR: '), writeln(Message),
    halt(1).

% Practical example: Command-line application
main :-
    catch(
        run_application,
        Error,
        (   print_error(Error),
            halt(1)
        )
    ),
    halt(0).

run_application :-
    parse_arguments(Args),
    validate_arguments(Args),
    process_command(Args).

% Conditional exit
check_requirements :-
    (   missing_requirement(Req)
    -> write('Missing requirement: '), writeln(Req),
        halt(2)
    ;   true
    ).
```

13. System Predicates

System predicates provide access to Prolog system features and configuration.

current_prolog_flag/2

Purpose: Queries system flags and configuration.

When to use: Use to check system capabilities or configuration.

```
% Query specific flag
?- current_prolog_flag(version, V).
V = '2.0.15'.

?- current_prolog_flag(bounded, B).
B = true. % Integers are bounded

% Enumerate all flags
?- current_prolog_flag(Flag, Value).
Flag = bounded, Value = true ;
Flag = max_integer, Value = 9223372036854775807 ;
Flag = min_integer, Value = -9223372036854775808 ;
...

% Check system capabilities
check_unicode_support :-
    current_prolog_flag(encoding, Encoding),
    (   Encoding = utf8
    ->  writeln('Unicode supported')
    ;   writeln('Limited character support')
    ).

% Practical example: Adjust behavior based on flags
get_max_int(Max) :-
    (   current_prolog_flag(bounded, true)
    ->  current_prolog_flag(max_integer, Max)
    ;   Max = inf
    ).

% Check debug mode
is_debug_mode :-
    current_prolog_flag(debug, on).

debug_print(Message) :-
    (   is_debug_mode
    ->  write('[DEBUG] '), writeln(Message)
    ;   true
    ).
```

set_prolog_flag/2

Purpose: Sets system flags (where allowed).

When to use: Use to configure system behavior.

v3.5.0: the `unknown` flag is enforced - calling an undefined procedure raises `existence_error(procedure, Name/Arity)` when the flag is `error` (the default), prints a warning and fails when `warning`, and fails silently when `fail`. Procedures declared `dynamic` (via the `:- dynamic` directive, the `dynamic/1` goal, or implied by `assert/retractall`) fail silently instead of raising the error.

v3.9.0 (ISS-2025-0441): `occurs_check` accepts a **third** value, `error`, as ISO 7.11.2.4 requires (`true`, `false`, `error`) - it was rejected before. `error` makes a unification that would build a cyclic term raise `representation_error(cyclic_term)` instead of succeeding with a rational tree.

v3.8.0 (ISS-2025-0437): flags are **per engine**. `set_prolog_flag/2` now changes only the Prolog instance that runs the goal - previously the flag store was a process-wide static, so `set_prolog_flag(unknown, fail)` (or `double_quotes`, or `occurs_check`) in one engine silently reconfigured every other engine in the JVM. The same applies to `trace/0` / `notrace/0`. Embedders can reach a specific engine's store with `Prolog.getFlags()` and toggle tracing from another thread with `Prolog.setTracing(boolean)`.

v3.14.0 (ISS-2025-0472/0474/0477): the last of the process-global state follows. The **stream table**, the **operator store** (`op/3` / `current_op/3`), the **spy points** and the **profiler counters** belong to the Prolog instance, reached with `Prolog.getStreams()`, `Prolog.getOps()` and `Prolog.getEngineState()`. Two engines in one JVM no longer see each other's streams, aliases, operators, spy points or profile numbers. LIM-034 is closed.

```
% Enable debug mode
?- set_prolog_flag(debug, on).
true.

% Set unknown predicate behavior
?- set_prolog_flag(unknown, fail). % Fail on undefined predicates
true.

% Practical example: Configure application
initialize_app :-
    set_prolog_flag(debug, off),
    set_prolog_flag(unknown, error),
    set_prolog_flag(double_quotes, codes),
    writeln('Application configured').

% Toggle debug mode
toggle_debug :-
    current_prolog_flag(debug, Current),
    (   Current = on
    -> set_prolog_flag(debug, off),
        writeln('Debug mode disabled')
    ;   set_prolog_flag(debug, on),
        writeln('Debug mode enabled')
    ).

% Set optimization level
set_optimization(Level) :-
    (   member(Level, [0, 1, 2, 3])
    -> set_prolog_flag(optimize, Level)
    ;   writeln('Invalid optimization level')
    ).
```

statistics/2

Purpose: Queries system statistics and performance metrics.

When to use: Use for performance monitoring and optimization.

```
% Query runtime
?- statistics(runtime, [Total, Since]).
Total = 1234, % Total milliseconds since start
Since = 10.  % Milliseconds since last call

% Memory usage
?- statistics(memory, [Used, Free]).
Used = 5242880, % Bytes used
Free = 10485760. % Bytes free

% Practical example: Measure execution time
time_goal(Goal, Time) :-
```

```

    statistics(runtime, [Start, _]),
    call(Goal),
    statistics(runtime, [End, _]),
    Time is End - Start.

?- time_goal(sleep(1), T).
T = 1000. % Milliseconds

% Performance profiling
profile(Goal) :-
    statistics(runtime, [T0, _]),
    statistics(memory, [M0, _]),
    call(Goal),
    statistics(runtime, [T1, _]),
    statistics(memory, [M1, _]),
    Time is T1 - T0,
    Memory is M1 - M0,
    format('Execution time: ~w ms~n', [Time]),
    format('Memory used: ~w bytes~n', [Memory]).

% Monitor resource usage
monitor_resources :-
    repeat,
    statistics(memory, [Used, Free]),
    Total is Used + Free,
    Percentage is (Used * 100) / Total,
    format('Memory usage: ~2f%~n', [Percentage]),
    sleep(5),
    fail.

```

<!-- START_CHANGE: ISS-2025-0179 - Add leash/1 debug predicate -->

leash/1

Purpose: Controls which ports (call, exit, redo, fail) the debugger pauses at during tracing.

When to use: Use to fine-tune debugger behavior so it only stops at ports you are interested in.

```

% Syntax: leash(+Ports)
% Ports is a list of port names: call, exit, redo, fail
?- leash([call, fail]).
true.
% Debugger will now only pause at call and fail ports

```

<!-- END_CHANGE: ISS-2025-0179 -->

14. Cryptographic Predicates

Cryptographic predicates provide hashing, encoding, and secure random generation capabilities for data integrity, authentication, and security use cases.

md5_hash/2

Purpose: Computes the MD5 hash of an atom, returning a hex string.

When to use: Use for checksums and non-security fingerprinting of data.

```
% Syntax: md5_hash(+Input, -Hash)
?- md5_hash('hello world', Hash).
Hash = '5eb63bbbe01eeed093cb22bb8f5acdc3'.

% Verify data integrity
verify_integrity(Data, ExpectedHash) :-
    md5_hash(Data, ActualHash),
    ActualHash = ExpectedHash.
```

sha256_hash/2

Purpose: Computes the SHA-256 hash of an atom, returning a hex string.

When to use: Use for secure hashing where collision resistance is needed.

```
% Syntax: sha256_hash(+Input, -Hash)
?- sha256_hash('hello world', Hash).
Hash = 'b94d27b9934d3e08a52e52d7da7dabfac484efe37a5380ee9088f7ace2efcde9'.

% Hash a password (for demonstration; use proper KDF in production)
store_password(User, Password) :-
    sha256_hash(Password, Hash),
    assertz(user_hash(User, Hash)).
```

sha512_hash/2

Purpose: Computes the SHA-512 hash of an atom, returning a hex string.

When to use: Use when maximum hash length and security margin are required.

```
% Syntax: sha512_hash(+Input, -Hash)
?- sha512_hash('hello world', Hash).
Hash = '309ecc489c12d6eb4cc40f50c902f2b4d0ed77ee511a7c7a9bcd3ca86d4cd86f989dd35bc5ff499670da34255b45b0cfd830e81f605dcf7dc5542e93ae9cd76f'.
```

crypto_hash/3

Purpose: Computes a hash using a specified algorithm.

When to use: Use when you need flexibility in choosing the hash algorithm at runtime.

```
% Syntax: crypto_hash(+Algorithm, +Input, -Hash)
?- crypto_hash(sha256, 'test data', Hash).
Hash = '916f0027a575074ce72a331777c3478d6513f786a591bd892da1a577bf2335f9'.

?- crypto_hash(md5, 'test data', Hash).
Hash = 'eb733a00c0c9d336e65691a37ab54293'.

% Choose algorithm based on requirements
```

```
secure_hash(Input, Hash) :-
    crypto_hash(sha256, Input, Hash).
```

hmac/4

Purpose: Computes an HMAC (Hash-based Message Authentication Code) using a key and algorithm.

When to use: Use for message authentication and integrity verification with a shared secret.

```
% Syntax: hmac(+Algorithm, +Key, +Message, -MAC)
?- hmac(sha256, 'secret_key', 'my message', MAC).
MAC = 'a1b2c3d4e5f6...'.

% Verify a received message
verify_message(Key, Message, ExpectedMAC) :-
    hmac(sha256, Key, Message, ComputedMAC),
    ComputedMAC = ExpectedMAC.
```

base64_encode/2

Purpose: Encodes an atom to its Base64 representation.

When to use: Use for encoding binary or text data for safe transmission over text-based protocols.

```
% Syntax: base64_encode(+Plain, -Encoded)
?- base64_encode('Hello, World!', Encoded).
Encoded = 'SGVsbG8sIFdvcmxkIQ=='.

% Encode credentials for HTTP basic auth
basic_auth_header(User, Password, Header) :-
    atom_concat(User, ':', Temp),
    atom_concat(Temp, Password, Credentials),
    base64_encode(Credentials, Encoded),
    atom_concat('Basic ', Encoded, Header).
```

base64_decode/2

Purpose: Decodes a Base64-encoded atom back to its original form.

When to use: Use to decode data received in Base64 format.

```
% Syntax: base64_decode(+Encoded, -Decoded)
?- base64_decode('SGVsbG8sIFdvcmxkIQ==', Decoded).
Decoded = 'Hello, World!'.

% Round-trip encoding
?- base64_encode('test', E), base64_decode(E, D).
E = 'dGVzdA==', D = 'test'.
```

uuid/1

Purpose: Generates a new UUID (version 4, random).

When to use: Use to create unique identifiers for records, sessions, or transactions.

```
% Syntax: uuid(-UUID)
?- uuid(Id).
Id = '550e8400-e29b-41d4-a716-446655440000'.

% Create a unique record
create_record(Name, Record) :-
    uuid(Id),
```

```
Record = record(Id, Name).
```

random_token/2

Purpose: Generates a cryptographically secure random hex token of the specified byte length.

When to use: Use for generating session tokens, API keys, or nonces.

```
% Syntax: random_token(+ByteLength, -Token)
?- random_token(16, Token).
Token = 'a3f2b8c1d4e5f6a7b8c9d0e1f2a3b4c5'.

% Generate an API key
generate_api_key(Key) :-
    random_token(32, Key).
```

crypto_random_int/3

Purpose: Generates a cryptographically secure random integer within a range.

When to use: Use when you need unbiased, unpredictable random numbers for security-sensitive applications.

```
% Syntax: crypto_random_int(+Low, +High, -Value)
% Generates a random integer N where Low <= N < High
?- crypto_random_int(1, 100, N).
N = 42.

% Generate a 6-digit OTP
generate_otp(OTP) :-
    crypto_random_int(100000, 1000000, OTP).
```

```
<!-- START_CHANGE: ISS-2025-0179 - Add AES and password hashing predicates -->
```

crypto_aes_encrypt/4

Purpose: Encrypts plaintext using AES symmetric encryption.

```
% Syntax: crypto_aes_encrypt(+Key, +Plaintext, +Options, -Ciphertext)
?- crypto_aes_encrypt('my_secret_key', 'hello world', [], Cipher).
```

crypto_aes_decrypt/4

Purpose: Decrypts AES-encrypted ciphertext back to plaintext.

```
% Syntax: crypto_aes_decrypt(+Key, +Ciphertext, +Options, -Plaintext)
?- crypto_aes_decrypt('my_secret_key', Cipher, [], Plain).
```

crypto_hash_password/2

Purpose: Hashes a password using PBKDF2 with a random salt for secure storage.

```
% Syntax: crypto_hash_password(+Password, -Hash)
?- crypto_hash_password('my_password', Hash).
Hash = 'pbkdf2:sha256:...'.

```

crypto_verify_password/2

Purpose: Verifies a password against a previously hashed value.

```
% Syntax: crypto_verify_password(+Password, +Hash)
?- crypto_hash_password('secret', H), crypto_verify_password('secret', H).
true.
```

<!-- END_CHANGE: ISS-2025-0179 -->

15. JSON Predicates

JSON predicates provide parsing, serialization, and manipulation of JSON data, enabling integration with web services and configuration files.

json_parse/2

Purpose: Parses a JSON string into a Prolog term representation.

When to use: Use to convert JSON data received from external sources into Prolog terms for processing.

```
% Syntax: json_parse(+JsonAtom, -Term)
?- json_parse('{"name":"John","age":30}', Term).
Term = json([name='John', age=30]).

?- json_parse('[1, 2, 3]', Term).
Term = [1, 2, 3].

% Parse and extract data
process_json(JsonString, Name) :-
    json_parse(JsonString, json(Pairs)),
    member(name=Name, Pairs).
```

json_serialize/2

Purpose: Serializes a Prolog term into a JSON string.

When to use: Use to produce JSON output for APIs, files, or inter-process communication.

```
% Syntax: json_serialize(+Term, -JsonAtom)
?- json_serialize(json([name='John', age=30]), Json).
Json = '{"name":"John","age":30}'.

?- json_serialize([1, 2, 3], Json).
Json = '[1,2,3]'.

% Build and serialize a response
build_response(Status, Message, Json) :-
    json_serialize(json([status=Status, message=Message]), Json).

?- build_response(ok, 'Operation completed', Json).
Json = '{"status":"ok","message":"Operation completed"}'.
```

json_get/3

Purpose: Retrieves a value from a parsed JSON term by key.

When to use: Use to access specific fields from a JSON object without manual member lookup.

```
% Syntax: json_get(+JsonTerm, +Key, -Value)
?- json_parse('{"name":"John","age":30}', J), json_get(J, name, V).
V = 'John'.

% Nested access with dot notation
?- json_parse('{"user":{"name":"John"}}', J), json_get(J, 'user.name', V).
V = 'John'.
```

json_set/4

Purpose: Sets or updates a key-value pair in a JSON term, producing a new term.

When to use: Use to modify JSON data immutably before serialization.

```
% Syntax: json_set(+JsonTerm, +Key, +Value, -NewJsonTerm)
?- json_parse('{ "name": "John", "age": 30}', J),
   json_set(J, age, 31, J2),
   json_serialize(J2, Out).
Out = '{ "name": "John", "age": 31}'.
```

json_keys/2

Purpose: Retrieves all keys from a JSON object as a list.

When to use: Use to enumerate the fields of a JSON object.

```
% Syntax: json_keys(+JsonTerm, -Keys)
?- json_parse('{ "name": "John", "age": 30, "city": "NYC"}', J), json_keys(J, Keys).
Keys = [name, age, city].
```

json_member/3

Purpose: Non-deterministically enumerates key-value pairs in a JSON object.

When to use: Use to iterate over all fields of a JSON object via backtracking.

```
% Syntax: json_member(+JsonTerm, ?Key, ?Value)
?- json_parse('{ "a": 1, "b": 2}', J), json_member(J, K, V).
K = a, V = 1 ;
K = b, V = 2.

% Find keys with numeric values
numeric_keys(Json, Keys) :-
    findall(K, (json_member(Json, K, V), number(V)), Keys).
```

16. Date/Time Predicates

Date and time predicates provide access to the system clock and operations for date arithmetic, formatting, and parsing.

get_time/1

Purpose: Returns the current time as a Unix timestamp (seconds since epoch) as a float.

When to use: Use for precise timing, benchmarking, or recording event timestamps.

```
% Syntax: get_time(-Timestamp)
?- get_time(T).
T = 1.7110592e+09.

% Measure elapsed time
elapsed(Goal, Seconds) :-
    get_time(T0),
    call(Goal),
    get_time(T1),
    Seconds is T1 - T0.
```

now/1

Purpose: Returns the current date and time as a `datetime(Y, M, D, H, Min, S)` term.

When to use: Use when you need the current date and time as structured components.

```
% Syntax: now(-DateTime)
?- now(DT).
DT = datetime(2026, 3, 21, 14, 30, 45).
```

today/1

Purpose: Returns the current date as a `date(Y, M, D)` term.

When to use: Use when you only need the current date without time components.

```
% Syntax: today(-Date)
?- today(D).
D = date(2026, 3, 21).
```

date_add/4

Purpose: Adds a duration to a date, producing a new date.

When to use: Use for date arithmetic such as computing deadlines or future dates.

```
% Syntax: date_add(+Date, +Duration, +Unit, -NewDate)
% Unit is one of: days, months, years
?- date_add(date(2026, 3, 21), 10, days, NewDate).
NewDate = date(2026, 3, 31).

?- date_add(date(2026, 3, 21), 2, months, NewDate).
NewDate = date(2026, 5, 21).

% Calculate a deadline
deadline(StartDate, DaysAllowed, Deadline) :-
    date_add(StartDate, DaysAllowed, days, Deadline).
```

date_diff/4

Purpose: Computes the difference between two dates in the specified unit.

When to use: Use to determine the number of days, months, or years between two dates.

```
% Syntax: date_diff(+Date1, +Date2, +Unit, -Difference)
?- date_diff(date(2026, 1, 1), date(2026, 3, 21), days, Diff).
Diff = 79.

% Calculate age in years
age(BirthDate, Age) :-
    today(Today),
    date_diff(BirthDate, Today, years, Age).
```

day_of_week/2

Purpose: Determines the day of the week for a given date.

When to use: Use for scheduling or display purposes.

```
% Syntax: day_of_week(+Date, -DayName)
?- day_of_week(date(2026, 3, 21), Day).
Day = saturday.

% Check if a date is a weekend
is_weekend(Date) :-
    day_of_week(Date, Day),
    member(Day, [saturday, sunday]).
```

date_parts/4

Purpose: Decomposes a date term into its year, month, and day components.

When to use: Use to extract individual components from a date term.

```
% Syntax: date_parts(+Date, -Year, -Month, -Day)
?- date_parts(date(2026, 3, 21), Y, M, D).
Y = 2026, M = 3, D = 21.

% Extract just the year
get_year(Date, Year) :-
    date_parts(Date, Year, _, _).
```

time_parts/4

Purpose: Decomposes a datetime term into its hour, minute, and second components.

When to use: Use to extract time components from a datetime term.

```
% Syntax: time_parts(+DateTime, -Hour, -Minute, -Second)
?- now(DT), time_parts(DT, H, M, S).
H = 14, M = 30, S = 45.
```

format_date/3

Purpose: Formats a date or datetime term into a string according to a format pattern.

When to use: Use to produce human-readable or standardized date strings.

```
% Syntax: format_date(+Format, +Date, -Formatted)
?- format_date('~Y-~m-~d', date(2026, 3, 21), S).
S = '2026-03-21'.

?- format_date('~d/~m/~Y', date(2026, 3, 21), S).
S = '21/03/2026'.

% ISO 8601 formatting
iso_date(Date, IsoString) :-
    format_date('~Y-~m-~d', Date, IsoString).
```

parse_date/3

Purpose: Parses a date string into a date term according to a format pattern.

When to use: Use to convert user input or file data into structured date terms.

```
% Syntax: parse_date(+Format, +String, -Date)
?- parse_date('~Y-~m-~d', '2026-03-21', D).
D = date(2026, 3, 21).

?- parse_date('~d/~m/~Y', '21/03/2026', D).
D = date(2026, 3, 21).
```

17. Filesystem Predicates

Filesystem predicates provide operations for querying and manipulating files and directories on the host system.

file_exists/1

Purpose: Checks whether a file exists at the given path.

When to use: Use to verify a file is present before attempting to read or process it.

```
% Syntax: file_exists(+Path)
?- file_exists('data.pl').
true.

?- file_exists('nonexistent.pl').
false.

% Safe file loading
safe_consult(File) :-
    (   file_exists(File)
    -> consult(File)
    ;   write('File not found: '), writeln(File)
    ).
```

directory_exists/1

Purpose: Checks whether a directory exists at the given path.

When to use: Use to verify a directory is present before listing or writing into it.

```
% Syntax: directory_exists(+Path)
?- directory_exists('/tmp').
true.

?- directory_exists('/nonexistent').
false.
```

delete_file/1

Purpose: Deletes a file from the filesystem.

When to use: Use to clean up temporary files or remove obsolete data.

```
% Syntax: delete_file(+Path)
?- delete_file('temp_output.txt').
true.
```

rename_file/2

Purpose: Renames or moves a file.

When to use: Use to rename files or move them between directories.

```
% Syntax: rename_file(+OldPath, +NewPath)
?- rename_file('old_name.pl', 'new_name.pl').
true.
```

copy_file/2

Purpose: Copies a file to a new location.

When to use: Use to create backups or duplicate files.

```
% Syntax: copy_file(+Source, +Destination)
?- copy_file('data.pl', 'data_backup.pl').
true.

% Backup before modifying
safe_modify(File) :-
    atom_concat(File, '.bak', Backup),
    copy_file(File, Backup).
```

file_size/2

Purpose: Returns the size of a file in bytes.

When to use: Use to check file sizes before processing or to report storage usage.

```
% Syntax: file_size(+Path, -Size)
?- file_size('data.pl', Size).
Size = 4096.
```

read_file_to_atom/2

Purpose: Reads the entire contents of a file into an atom.

When to use: Use for reading small files or configuration data in one operation.

```
% Syntax: read_file_to_atom(+Path, -Content)
?- read_file_to_atom('config.txt', Content).
Content = 'key1=value1\nkey2=value2\n'.
```

write_atom_to_file/2

Purpose: Writes an atom as the entire contents of a file, overwriting any existing content.

When to use: Use for writing small files or saving generated output.

```
% Syntax: write_atom_to_file(+Content, +Path)
?- write_atom_to_file('Hello, World!\n', 'output.txt').
true.
```

make_directory/1

Purpose: Creates a single directory.

When to use: Use when you need to create exactly one directory level.

```
% Syntax: make_directory(+Path)
?- make_directory('output').
true.
```

make_directory_path/1

Purpose: Creates a directory and all necessary parent directories.

When to use: Use when the full path may not exist and intermediate directories need to be created.

```
% Syntax: make_directory_path(+Path)
?- make_directory_path('output/reports/2026').
true.
```

absolute_file_name/2

Purpose: Resolves a relative or aliased file path to an absolute path.

When to use: Use to normalize file paths for consistent handling.

```
% Syntax: absolute_file_name(+Relative, -Absolute)
?- absolute_file_name('data.pl', Abs).
Abs = '/home/user/project/data.pl'.
```

directory_files/2

Purpose: Lists all files and directories in a given directory.

When to use: Use to enumerate directory contents for batch processing.

```
% Syntax: directory_files(+Directory, -Files)
?- directory_files('.', Files).
Files = ['file1.pl', 'file2.pl', 'subdir'].

% Process all Prolog files in a directory
consult_all(Dir) :-
    directory_files(Dir, Files),
    member(F, Files),
    file_extension(F, pl),
    atom_concat(Dir, '/', Temp),
    atom_concat(Temp, F, Path),
    consult(Path),
    fail ; true.
```

file_extension/2

Purpose: Extracts the file extension from a filename.

When to use: Use to filter files by type or determine how to process a file.

```
% Syntax: file_extension(+FileName, -Extension)
?- file_extension('report.pdf', Ext).
Ext = pdf.

?- file_extension('archive.tar.gz', Ext).
Ext = gz.
```

file_base_name/2

Purpose: Extracts the base filename (without directory path) from a full path.

When to use: Use to get just the filename from a full path.

```
% Syntax: file_base_name(+Path, -BaseName)
?- file_base_name('/home/user/data.pl', Base).
Base = 'data.pl'.
```

file_directory_name/2

Purpose: Extracts the directory portion from a full file path.

When to use: Use to determine which directory a file resides in.

```
% Syntax: file_directory_name(+Path, -Directory)
?- file_directory_name('/home/user/data.pl', Dir).
Dir = '/home/user'.
```

18. OS Predicates

OS predicates provide access to the operating system environment, process information, and shell command execution.

getenv/2

Purpose: Retrieves the value of an environment variable.

When to use: Use to read configuration from environment variables.

```
% Syntax: getenv(+VarName, -Value)
?- getenv('HOME', Home).
Home = '/home/user'.

?- getenv('PATH', Path).
Path = '/usr/local/bin:/usr/bin:/bin'.

% Configuration from environment
db_host(Host) :-
    (   getenv('DB_HOST', Host)
    -> true
    ;   Host = localhost
    ).
```

setenv/2

Purpose: Sets the value of an environment variable.

When to use: Use to configure the environment for child processes or shell commands.

```
% Syntax: setenv(+VarName, +Value)
?- setenv('MY_APP_MODE', 'production').
true.
```

system_time/1

Purpose: Returns the system time in milliseconds since epoch as an integer.

When to use: Use for timestamps, profiling, or unique identifier generation.

```
% Syntax: system_time(-Millis)
?- system_time(T).
T = 1711059200000.
```

os_name/1

Purpose: Returns the name of the operating system.

When to use: Use to adapt behavior based on the host platform.

```
% Syntax: os_name(-Name)
?- os_name(OS).
OS = 'Linux'.

% Platform-specific path separator
path_separator(Sep) :-
    os_name(OS),
    (   OS = 'Windows' -> Sep = '\\' ; Sep = '/' ).
```

cpu_count/1

Purpose: Returns the number of available CPU cores.

When to use: Use to size thread pools or determine parallelism levels.

```
% Syntax: cpu_count(-Count)
?- cpu_count(N).
N = 8.
```

free_memory/1

Purpose: Returns the amount of free memory available to the JVM in bytes.

When to use: Use to monitor resource consumption or decide whether to proceed with memory-intensive tasks.

```
% Syntax: free_memory(-Bytes)
?- free_memory(M).
M = 268435456.
```

total_memory/1

Purpose: Returns the total memory allocated to the JVM in bytes.

When to use: Use together with free_memory/1 for memory utilization reporting.

```
% Syntax: total_memory(-Bytes)
?- total_memory(T), free_memory(F), Used is T - F.
T = 536870912, F = 268435456, Used = 268435456.
```

pid/1

Purpose: Returns the process ID of the current JVM process.

When to use: Use for logging, lock files, or process identification.

```
% Syntax: pid(-PID)
?- pid(P).
P = 12345.
```

hostname/1

Purpose: Returns the hostname of the machine.

When to use: Use for logging or identifying which host a program is running on.

```
% Syntax: hostname(-Name)
?- hostname(H).
H = 'myserver.example.com'.
```

shell/1

Purpose: Executes a shell command and succeeds if the command exits with status 0.

When to use: Use to run system commands where you only need to know success or failure.

```
% Syntax: shell(+Command)
?- shell('ls /tmp').
```

```
true.

?- shell('test -f config.ini').
true. % File exists
```

shell/2

Purpose: Executes a shell command and unifies the second argument with the exit code.

When to use: Use when you need to inspect the exit status of a command.

```
% Syntax: shell(+Command, -ExitCode)
?- shell('grep -q pattern file.txt', Code).
Code = 0. % Pattern found

?- shell('false', Code).
Code = 1.
```

shell_output/3

Purpose: Executes a shell command, capturing its standard output and exit code.

When to use: Use when you need to process the output of a system command in Prolog.

```
% Syntax: shell_output(+Command, -Output, -ExitCode)
?- shell_output('date +%Y', Output, Code).
Output = '2026\n', Code = 0.

?- shell_output('whoami', User, _).
User = 'prolog_user\n'.
```

19. Regex Predicates

Regex predicates provide regular expression matching, substitution, and splitting, powered by Java's regex engine.

re_match/2

Purpose: Tests whether a string matches a regular expression.

When to use: Use for pattern validation such as checking email formats, identifiers, or input constraints.

```
% Syntax: re_match(+Pattern, +String)
?- re_match('[0-9]+', '12345').
true.

?- re_match('^[a-z]+$', 'Hello').
false.

% Validate an email address (simplified)
valid_email(Email) :-
    re_match('^[^@]+@[^@]+\.[^@]+$', Email).

?- valid_email('user@example.com').
true.
```

re_matchsub/3

Purpose: Matches a regex with capturing groups and returns a list of captured substrings.

When to use: Use to extract parts of a string that match specific sub-patterns.

```
% Syntax: re_matchsub(+Pattern, +String, -Captures)
?- re_matchsub('(\d{4})-(\d{2})-(\d{2})', '2026-03-21', Caps).
Caps = ['2026-03-21', '2026', '03', '21'].

% Parse a log line
parse_log(Line, Level, Message) :-
    re_matchsub('\[(\w+)\]\s+(.*)', Line, [_, Level, Message]).

?- parse_log('[ERROR] Connection failed', Level, Msg).
Level = 'ERROR', Msg = 'Connection failed'.
```

re_replace/4

Purpose: Replaces occurrences of a pattern in a string with a replacement.

When to use: Use for text transformation, sanitization, or templating.

```
% Syntax: re_replace(+Pattern, +Replacement, +Input, -Output)
?- re_replace('[0-9]+', 'NUM', 'Order 123 has 4 items', Out).
Out = 'Order NUM has NUM items'.

% Sanitize input
sanitize(Input, Clean) :-
    re_replace('[^a-zA-Z0-9 ]', '', Input, Clean).

?- sanitize('Hello <World>!', C).
C = 'Hello World'.
```

re_split/3

Purpose: Splits a string by a regex pattern into a list of substrings.

When to use: Use to tokenize or break apart strings using flexible delimiters.

```
% Syntax: re_split(+Pattern, +String, -Parts)
?- re_split(',', '\\s*', 'a, b, c, d', Parts).
Parts = ['a', 'b', 'c', 'd'].

?- re_split('\\\\s+', 'Hello  World  Test', Words).
Words = ['Hello', 'World', 'Test'].
```

re_findall/3

Purpose: Finds all non-overlapping matches of a pattern in a string.

When to use: Use to extract all occurrences of a pattern from text.

```
% Syntax: re_findall(+Pattern, +String, -Matches)
?- re_findall('[0-9]+', 'Order 123 has 4 items at $56', Nums).
Nums = ['123', '4', '56'].

% Extract all email addresses from text
extract_emails(Text, Emails) :-
    re_findall('[\\w.]+@[\\w.]+', Text, Emails).
```

<!-- START_CHANGE: ISS-2025-0179 - Add re_escape/2 -->

re_escape/2

Purpose: Escapes special regex characters in a string so it can be used as a literal pattern.

```
% Syntax: re_escape(+Input, -Escaped)
?- re_escape('hello.world', E).
E = 'hello\\.world'.

% Use to safely match user-provided literal text
safe_match(Literal, Text) :-
    re_escape(Literal, Pattern),
    re_match(Pattern, Text).
```

<!-- END_CHANGE: ISS-2025-0179 -->

20. XML Predicates

XML predicates provide parsing, serialization, and querying of XML documents.

xml_parse/2

Purpose: Parses an XML string into a Prolog term representation.

When to use: Use to process XML data from files, APIs, or configuration sources.

```
% Syntax: xml_parse(+XmlAtom, -Term)
?- xml_parse('<person name="John"><age>30</age></person>', Term).
Term = element(person, [name='John'], [element(age, [], ['30'])]).

% Parse and inspect structure
get_root_tag(Xml, Tag) :-
    xml_parse(Xml, element(Tag, _, _)).
```

xml_serialize/2

Purpose: Serializes a Prolog XML term into an XML string.

When to use: Use to generate XML output from Prolog data structures.

```
% Syntax: xml_serialize(+Term, -XmlAtom)
?- xml_serialize(element(item, [id='1'], ['Hello']), Xml).
Xml = '<item id="1">Hello</item>'.

% Build an XML document
build_person_xml(Name, Age, Xml) :-
    AgeAtom = element(age, [], [Age]),
    Person = element(person, [name=Name], [AgeAtom]),
    xml_serialize(Person, Xml).
```

xpath/3

Purpose: Queries an XML term using an XPath-like expression, returning matching elements.

When to use: Use to extract specific elements from parsed XML data.

```
% Syntax: xpath(+XmlTerm, +Path, -Result)
?- xml_parse('<root><item>A</item><item>B</item></root>', Doc),
    xpath(Doc, '//item', Item).
Item = element(item, [], ['A']) ;
Item = element(item, [], ['B']).

% Extract attribute values
get_attribute(Xml, Path, Attr, Value) :-
    xpath(Xml, Path, element(_, Attrs, _)),
    member(Attr=Value, Attrs).
```

21. Threading Predicates

Threading predicates enable concurrent execution with message passing, built on Java's threading model. Use these for parallelism and background task processing.

Since v4.0.0 (ISS-2025-0479) a thread really runs its goal, on its own resolution machine over the same engine. Before that, `thread_create/2` started a thread that slept briefly and recorded a synthetic status; the goal never ran and it had to be an atom. What a worker shares with its creator: the clause store (so `assertz/retract` are visible both ways), the flags, the operator table and the module system. What it does not share: the goal's variables - the goal is copy_term'd, so **a binding made by a worker never appears in the creator's query** - its current input/output streams, and its inference-budget counter (its own, with the same limit). A thread id is an integer; an `alias(Name)` option gives it a name usable wherever an id is.

Sandbox: `Prolog.enableSafeMode()` removes every predicate in this section and in section 40 (concurrent execution) - a JVM thread is a host resource, and each worker carries its own inference budget, so spawning threads would escape the engine's CPU limit.

thread_create/2

Purpose: Creates a new thread that executes a given goal.

When to use: Use to run a goal concurrently in the background.

```
% Syntax: thread_create(:Goal, -ThreadId)
?- thread_create(long_computation(Result), TId).
TId = 1.

% Start a background task
start_worker(Id) :-
    thread_create(worker_loop, Id).

% The goal really runs, and its database writes are visible afterwards
?- thread_create(assertz(done(yes)), T), thread_join(T, true), done(X).
X = yes.

% ... but its BINDINGS are not: the goal is copied
?- thread_create(X = 1, T), thread_join(T, true), var(X).
true.
```

thread_create/3

Purpose: Creates a thread with options.

When to use: Use when the thread needs a name (so other threads can send it messages) or must clean itself up without being joined.

```
% Syntax: thread_create(:Goal, -ThreadId, +Options)
% Options: alias(Name)      - a name usable wherever a thread id is
%          detached(Bool)   - true: the thread frees itself on completion and cannot be jo
ined
% Any other option is accepted and ignored.
?- thread_create(worker_loop, _, [alias(logger)]).
true.

?- thread_create(cleanup_task, T, [detached(true)]).
T = 4.
```

thread_join/2

Purpose: Waits for a thread to complete and unifies with its exit status.

When to use: Use to synchronize with a thread and retrieve its result.

The status is `true` when the goal succeeded, `false` when it failed, `exception(Ball)` when it threw (including `exception(inference_limit_exceeded)` when it exhausted the engine's inference budget), and `cancelled` when the thread was interrupted. The thread argument may be an id or an alias. A detached thread cannot be joined.

```
% Syntax: thread_join(+ThreadOrAlias, -Status)
?- thread_create(member(X, [a, b, c]), TId),
   thread_join(TId, Status).
Status = true.

?- thread_create(fail, T), thread_join(T, S).
S = false.

?- thread_create(throw(oops), T), thread_join(T, S).
S = exception(oops).

% Wait for a computation
run_and_wait(Goal, Status) :-
    thread_create(Goal, TId),
    thread_join(TId, Status).
```

thread_detach/1

Purpose: Detaches a thread so its resources are freed automatically upon completion.

When to use: Use for fire-and-forget tasks where you do not need the result.

```
% Syntax: thread_detach(+ThreadId)
?- thread_create(log_event(startup), TId), thread_detach(TId).
true.
```

thread_self/1

Purpose: Returns the identifier of the currently executing thread.

When to use: Use for logging or when threads need to identify themselves.

```
% Syntax: thread_self(-ThreadId)
?- thread_self(Id).
Id = main.
```

A thread that has an **alias** reports the alias, as in SWI-Prolog: the top-level thread is `main`, and a worker started with `thread_create(Goal, Id, [alias(w1)])` reports `w1`. A worker with no alias reports its integer Prolog id. Either form is accepted wherever a thread is named (`thread_join/2`, `thread_send_message/2`, `thread_is_alive/1`, ...).

thread_sleep/1

Purpose: Suspends the current thread for the specified number of seconds.

When to use: Use for delays, polling intervals, or rate limiting.

```
% Syntax: thread_sleep(+Seconds)
?- thread_sleep(2).
true. % Resumes after 2 seconds
```

```
% Retry with delay
retry_with_delay(Goal, Retries) :-
    Retries > 0,
    (   call(Goal) -> true
    ;   thread_sleep(1),
        R1 is Retries - 1,
        retry_with_delay(Goal, R1)
    ).
```

thread_is_alive/1

Purpose: Checks whether a thread is still running.

When to use: Use to poll thread status without blocking.

```
% Syntax: thread_is_alive(+ThreadId)
?- thread_create(thread_sleep(10), TId), thread_is_alive(TId).
true.
```

message_queue_create/1

Purpose: Creates a new message queue for inter-thread communication.

When to use: Use to set up communication channels between threads.

Since v4.0.0 a queue carries **arbitrary Prolog terms**, not just atoms, and every message is copied on the way in and on the way out so no variable is shared between the two threads. Every thread created by `thread_create/2,3` also owns a queue: `thread_send_message/2` accepts a queue id, a thread id or a thread alias, and `thread_get_message/1` reads the calling thread's own queue.

```
% Syntax: message_queue_create(-QueueId)
?- message_queue_create(Q).
Q = 1.
```

thread_get_message/1

Purpose: Reads a message from the calling thread's own queue, blocking until one arrives.

When to use: Use inside a worker started with `thread_create/3` and an `alias/1`, so other threads can address it by name.

```
% Syntax: thread_get_message(-Message)
?- thread_create((thread_get_message(M), assertz(got(M))), T, [alias(worker)]),
    thread_send_message(worker, hello(world)),
    thread_join(T, true),
    got(G).
G = hello(world).
```

thread_send_message/2

Purpose: Sends a message (any Prolog term) to a message queue.

When to use: Use to pass data to a consumer thread.

```
% Syntax: thread_send_message(+QueueOrThreadOrAlias, +Message)
?- message_queue_create(Q), thread_send_message(Q, hello).
true.

% Any term, not just an atom
?- message_queue_create(Q), thread_send_message(Q, point(1, [a,b])),
```

```

    thread_get_message(Q, M).
M = point(1,[a,b]).

% Producer pattern
produce(Queue, Items) :-
    member(Item, Items),
    thread_send_message(Queue, Item),
    fail ; true.

```

thread_get_message/2

Purpose: Retrieves a message from a queue, blocking until one is available.

When to use: Use in consumer threads to wait for and process incoming messages.

```

% Syntax: thread_get_message(+QueueOrThreadOrAlias, -Message)
?- message_queue_create(Q),
    thread_send_message(Q, world),
    thread_get_message(Q, Msg).
Msg = world.

% Consumer loop
consume_loop(Queue) :-
    thread_get_message(Queue, Msg),
    (   Msg = stop -> true
    ;   process(Msg),
        consume_loop(Queue)
    ).

```

thread_peek_message/2

Purpose: Checks if a message is available in a queue without removing it.

When to use: Use to inspect the queue non-destructively, or to check for messages without blocking.

```

% Syntax: thread_peek_message(+QueueId, -Message)
?- message_queue_create(Q),
    thread_send_message(Q, test),
    thread_peek_message(Q, Msg).
Msg = test.

% Check if there is work to do
has_work(Queue) :-
    thread_peek_message(Queue, _).

```

22. CSV Predicates

CSV predicates provide parsing, serialization, and file I/O for comma-separated value data, commonly used for data exchange with spreadsheets and databases.

csv_parse/2

Purpose: Parses a CSV string into a list of rows, where each row is a list of fields.

When to use: Use to process CSV data received as a string.

```
% Syntax: csv_parse(+CsvAtom, -Rows)
?- csv_parse('name,age\nJohn,30\nMary,25', Rows).
Rows = [['name', 'age'], ['John', '30'], ['Mary', '25']].

% Parse and skip header
csv_data(CsvString, Header, DataRows) :-
    csv_parse(CsvString, [Header|DataRows]).
```

csv_serialize/2

Purpose: Serializes a list of rows into a CSV string.

When to use: Use to produce CSV output from Prolog data.

```
% Syntax: csv_serialize(+Rows, -CsvAtom)
?- csv_serialize(['name', 'age'], ['John', '30'], Csv).
Csv = 'name,age\nJohn,30\n'.
```

csv_read_file/2

Purpose: Reads a CSV file and returns its contents as a list of rows.

When to use: Use to load tabular data directly from a file.

```
% Syntax: csv_read_file(+Path, -Rows)
?- csv_read_file('data.csv', Rows).
Rows = [['id', 'name', 'score'], ['1', 'Alice', '95'], ['2', 'Bob', '87']].

% Process file data
average_score(File, Avg) :-
    csv_read_file(File, [_Header|Rows]),
    findall(S, (member(Row, Rows), last(Row, SA), atom_number(SA, S)), Scores),
    sumlist(Scores, Total),
    length(Scores, N),
    Avg is Total / N.
```

csv_write_file/2

Purpose: Writes a list of rows to a CSV file.

When to use: Use to export Prolog data to CSV format for use in other tools.

```
% Syntax: csv_write_file(+Path, +Rows)
?- csv_write_file('output.csv', [['name', 'score'], ['Alice', '95'], ['Bob', '87']]).
true.

% Export query results
export_employees(File) :-
    findall([Name, Dept, Sal],
```

```
    employee(Name, Dept, Sal),  
    Rows),  
    csv_write_file(File, [['Name', 'Department', 'Salary']|Rows]).
```

23. Logging Predicates

Logging predicates provide structured, level-based logging for diagnostics and monitoring during program execution.

log_info/1

Purpose: Logs a message at the INFO level.

When to use: Use for general informational messages about normal program operation.

```
% Syntax: log_info(+Message)
?- log_info('Application started').
% Output: [INFO] Application started
true.
```

log_warning/1

Purpose: Logs a message at the WARNING level.

When to use: Use to report unexpected but non-fatal conditions.

```
% Syntax: log_warning(+Message)
?- log_warning('Configuration file missing, using defaults').
% Output: [WARNING] Configuration file missing, using defaults
true.
```

log_error/1

Purpose: Logs a message at the ERROR level.

When to use: Use to report errors that may affect program correctness.

```
% Syntax: log_error(+Message)
?- log_error('Failed to connect to database').
% Output: [ERROR] Failed to connect to database
true.

% Log errors in exception handlers
safe_process(Goal) :-
    catch(
        call(Goal),
        Error,
        (   term_to_atom(Error, Msg),
            log_error(Msg),
            fail
        )
    ).
```

log_debug/1

Purpose: Logs a message at the DEBUG level.

When to use: Use for detailed diagnostic information during development.

```
% Syntax: log_debug(+Message)
?- log_debug('Entering solve/3 with X = 42').
% Output: [DEBUG] Entering solve/3 with X = 42
true.
```

log_level/1

Purpose: Sets the minimum logging level. Messages below this level are suppressed.

When to use: Use to control verbosity at runtime.

```
% Syntax: log_level(+Level)
% Levels (from most to least verbose): debug, info, warning, error
?- log_level(warning).
true.

% Now only warning and error messages are displayed
?- log_info('This will be suppressed').
true.

?- log_warning('This will be shown').
% Output: [WARNING] This will be shown
true.
```

log_to_file/1

Purpose: Redirects log output to a file.

When to use: Use to capture logs for later analysis instead of writing to the console.

```
% Syntax: log_to_file(+FilePath)
?- log_to_file('app.log').
true.

?- log_info('This goes to the file').
true.

% Set up application logging
init_logging :-
    log_to_file('logs/app.log'),
    log_level(info),
    log_info('Logging initialized').
```

24. CLP(FD) Predicates

CLP(FD) (Constraint Logic Programming over Finite Domains) predicates allow you to declare constraints over integer variables and let the solver find valid assignments. JProlog v3.0.0 uses a clean-room interval-domain solver (builtin.clpfd.v2) by default: bounded interval domains, a per-query trail-backtracked constraint store, bound-consistency propagation (e.g. `real #=>` propagation, `Cmp/Sum/Mul/Abs/Alldifferent/Linear/Reified/Mod`), and sound first-fail labeling. The legacy AC-3 store remains available via `-Djprolog.clpfd=legacy`.

in/2

Purpose: Constrains a variable to a finite domain range.

When to use: Use to declare the possible values for a constraint variable before posting constraints.

v3.5.0: unification respects domains - `X in 1..3, X = 5` fails (it used to succeed unsoundly), while `X in 1..3, X = 2` succeeds; constraint posts are undone on backtracking.

```
% Syntax: X in +Low..+High
?- X in 1..10.
true.

% Constrain multiple variables
sudoku_vars(Vars) :-
    length(Vars, 9),
    maplist(clpfd:in_(1..9), Vars).
```

#=/2

Purpose: Posts an equality constraint between two arithmetic expressions.

When to use: Use instead of `is/2` when working with constraint variables that are not yet bound.

v3.5.0: when propagation narrows a variable to a singleton domain the variable is bound - `X #=> 2` gives `X = 2` without labeling.

v3.6.0: non-linear constraints propagate instead of failing silently - variable products (`X*Y`) and squares (`X*X #=> 16` narrows `X in 0..10` to `X = 4` before labeling) work via interval products, as do `abs/1`, `min/2`, `max/2` and `E mod M` (constant positive `M`); a genuinely unsupported functor raises `type_error(evaluable, F/N)` and a float coefficient (`2.5*X`) raises `type_error(integer, 2.5)` instead of truncating.

```
% Syntax: Expr1 #=> Expr2
?- X in 1..10, X #=> 3 + 4.
X = 7.

% Bidirectional reasoning
?- X in 1..10, Y in 1..10, X + Y #=> 10, X #=> 3.
X = 3, Y = 7.

% Non-linear products (v3.6.0)
?- X in 0..10, X*X #=> 16.
X = 4.

?- A in 1..5, B in 1..5, C in 1..5,
    A*A + B*B #=> C*C, label([A, B, C]).
A = 3, B = 4, C = 5 ;
A = 4, B = 3, C = 5.
```

#\=/2

Purpose: Posts a disequality (not-equal) constraint.

When to use: Use to declare that two expressions must have different values.

v3.5.0: multi-variable expressions work ($X \# \neq Y + 1$), and $X \# \neq X$ fails.

```
% Syntax: Expr1 #\= Expr2
?- X in 1..5, Y in 1..5, X #\= Y, X #= 3, label([Y]).
Y = 1 ; Y = 2 ; Y = 4 ; Y = 5.
```

#</2

Purpose: Posts a strict less-than constraint.

```
% Syntax: Expr1 #< Expr2
?- X in 1..10, Y in 1..10, X #< Y, X #= 8, label([Y]).
Y = 9 ; Y = 10.
```

#>/2

Purpose: Posts a strict greater-than constraint.

```
% Syntax: Expr1 #> Expr2
?- X in 1..10, X #> 7, label([X]).
X = 8 ; X = 9 ; X = 10.
```

#=</2

Purpose: Posts a less-than-or-equal constraint.

```
% Syntax: Expr1 #=< Expr2
?- X in 1..10, X #=< 3, label([X]).
X = 1 ; X = 2 ; X = 3.
```

#>=/2

Purpose: Posts a greater-than-or-equal constraint.

```
% Syntax: Expr1 #>= Expr2
?- X in 1..5, X #>= 4, label([X]).
X = 4 ; X = 5.
```

all_different/1, all_distinct/1

Purpose: Constrains all variables in a list to take pairwise different values. `all_distinct/1` is an accepted synonym (in SWI it is the stronger, domain-consistent propagator; JProlog implements the same pairwise constraint as `all_different/1` plus a pigeonhole check).

When to use: Use for problems like Sudoku, graph coloring, or any assignment problem requiring distinct values.

```
% Syntax: all_different(+Vars)
?- X in 1..3, Y in 1..3, Z in 1..3,
   all_different([X, Y, Z]),
   label([X, Y, Z]).
X = 1, Y = 2, Z = 3 ;
```

```
X = 1, Y = 3, Z = 2 ;
...
```

label/1

Purpose: Assigns concrete values to constraint variables by exhaustive search.

When to use: Use after posting all constraints to enumerate solutions.

v3.6.0: list elements that are neither variables nor integers raise `type_error(integer, T)` - `label([a])` no longer succeeds silently; ground integers remain legal.

```
% Syntax: label(+Vars)
?- X in 1..3, Y in 1..3, X #< Y, label([X, Y]).
X = 1, Y = 2 ;
X = 1, Y = 3 ;
X = 2, Y = 3.
```

labeling/2

Purpose: Labels variables with options controlling search strategy.

When to use: Use for fine-grained control over variable and value ordering.

v3.6.0: the options are honoured - `leftmost/ff/ffc/min/max` select the branching variable, `up/down` set the value enumeration order (down yields the largest value first), `min(Expr)/max(Expr)` order solutions optimum-first, and `step/enum` are accepted. An unknown option raises `domain_error(labeling_option, 0)`, an unbound option raises `instantiation_error`, and a non-list options argument raises `type_error(list, 0)`. Non-variable, non-integer list elements raise `type_error(integer, T)` (as for `label/1`).

```
% Syntax: labeling(+Options, +Vars)
% Options: leftmost, ff (first-fail), ffc, min, max, up, down, min(Expr), max(Expr)
?- X in 1..5, Y in 1..5, X + Y #= 6,
   labeling([ff], [X, Y]).
X = 1, Y = 5 ;
X = 2, Y = 4 ;
...

% Value order (v3.6.0)
?- X in 0..5, labeling([down], [X]).
X = 5 ;
X = 4 ;
...

?- X in 1..3, labeling([bogus], [X]).
% throws domain_error(labeling_option, bogus)
```

indomain/1

Purpose: Nondeterministically assigns a value from the domain of a single variable.

When to use: Use for custom labeling strategies where you want to control variable order manually.

```
% Syntax: indomain(+Var)
?- X in 1..3, indomain(X).
X = 1 ; X = 2 ; X = 3.
```

fd_dom/2

Purpose: Returns the current domain of a constraint variable.

When to use: Use to inspect the remaining possible values after constraint propagation.

```
% Syntax: fd_dom(+Var, -Domain)
?- X in 1..10, X #> 7, fd_dom(X, Dom).
Dom = 8..10.
```

fd_size/2

Purpose: Returns the number of elements in the current domain of a variable.

When to use: Use in custom search heuristics (e.g., first-fail selects the variable with smallest domain).

```
% Syntax: fd_size(+Var, -Size)
?- X in 1..10, X #> 7, fd_size(X, S).
S = 3.
```

25. Tabling Predicates

Tabling (memoization, tabled resolution) caches the answers of a tabled predicate per **call variant**, so a repeated call with the same argument pattern is answered from the table and a left-recursive or cyclic definition terminates instead of looping.

One implementation since 4.1.0

Tabling used to differ per engine. The engine that carried the bounded, sometimes-wrong implementation was deleted in 4.1.0, so what follows is simply how tabling works.

	the pre-4.1.0 fallback engines (deleted)	the engine (v4, an option since 3.12.0, the default since 4.0.0)
Algorithm	bounded re-evaluation: the goal was re-run at most 100 times over name-keyed answer maps	linear tabling with completion (SLD + iterative completion, B-Prolog/DRA style) in the machine's own choice points
Correctness	wrong answers for a left-recursive predicate over a long chain (LIM-038 / design limit L-03)	correct and complete for definite programs, left recursion included
Recursion depth	the pre-4.0.0 recursive solver's 2 000-deep Java cap	none (a tabled call is a choice point)
Inference budget / Stop	not enforced inside the fixpoint	enforced
Four-port trace / debugger	the whole tabled call was opaque	Call/Exit/Redo/Fail like any predicate
current_table/2	not available	available

The classic repro, which **succeeds today and failed on the deleted fallback**:

```

edge(I, J) :- between(1, 3000, I), J is I + 1.
:- table path/2.
path(X, Y) :- edge(X, Y).
path(X, Y) :- path(X, Z), edge(Z, Y).

?- path(1, 3001).                % true   (old fallback: fails)
?- path(1, 51).                 % true   (old fallback: fails)
?- findall(Y, path(1, Y), L), length(L, N). % N = 3000

```

Semantics

- A **variant table** is created per tabled subgoal. `path(1, Y)` and `path(1, 51)` are different variants and each gets its own table; both are answered correctly.
- The first call to a variant is its **generator**: it runs the predicate's clauses against a private copy of the call and records every answer. A call to a variant that is still being evaluated is a **consumer** over the answers found so far. When a generator that leads its strongly connected component exhausts its clauses, it re-runs them until a round produces no new answer anywhere in the component, and the whole component is then marked complete. There is no iteration cap; termination follows from the finite, deduplicated answer set.
- Answers are deduplicated by variant, and returned in the order they were first found.
- A **!** in a tabled clause body is **local to that body**: it prunes the body's own choice points and never truncates the table.
- An evaluation abandoned by an exception, a cut or the resource guard **discards its tables**, so the next call recomputes them rather than reading a partial answer set.
- Tabled calls work inside `findall/3`, `bagof/3`, `\+ /1`, `catch/3`, `once/1` and `forall/2`.
- Invalidation policy**: asserting to or retracting from a *tabled* predicate drops that predicate's tables. A change to a **non-tabled** predicate that a tabled one depends on is **not** tracked - call `abolish_all_tables/0` yourself (this is also what XSB requires). Tables persist across queries; two

safety caps (100 000 tables, 4 000 000 answers) drop the oldest completed tables at a query boundary so a long-lived engine cannot grow the store without bound.

- **`tnot/1` (tabled negation) is not implemented** on any engine: it raises `existence_error(procedure, tnot/1)`.

table/1

Purpose: Declares a predicate as tabled, enabling automatic memoization.

When to use: Use for predicates with overlapping subproblems (e.g., Fibonacci, transitive closure) or left-recursive definitions. Declare the predicate **before** it is first called.

```
% Syntax: :- table Predicate/Arity.      (also callable as a goal: table(Predicate/Arity)
%)
:- table fib/2.
fib(0, 0).
fib(1, 1).
fib(N, F) :-
    N > 1,
    N1 is N - 1, N2 is N - 2,
    fib(N1, F1), fib(N2, F2),
    F is F1 + F2.

% Without tabling: exponential time
% With tabling: linear time
?- fib(30, F).
F = 832040.
?- fib(1000, F).                % 209 digits, ~0.1 s on v4
```

Errors: fails silently if the argument is not a Name/Arity predicate indicator.

abolish_all_tables/0

Purpose: Clears all cached answers, forcing every tabled predicate to recompute on its next call. The table **declarations are kept**.

When to use: after `assert/retract` on facts a tabled predicate depends on, or between computation phases.

```
% Syntax: abolish_all_tables
?- fib(10, F).
F = 55.

?- abolish_all_tables.
true.

% Next call to fib/2 recomputes from scratch
```

Errors (v4): `permission_error(modify, table, ...)` if called from inside a running tabled evaluation.

abolish_table/1

Purpose: Clears the cached answers of one predicate **and un-declares it as tabled**.

When to use: to invalidate one predicate's table selectively, or to switch a predicate back to ordinary evaluation. Call `table(Name/Arity)` again to re-enable tabling for it.

```
% Syntax: abolish_table(+Name/Arity)
?- abolish_table(fib/2).
true.

% Only fib/2 is affected; other tabled predicates keep their tables and their declarations
```

Errors: `instantiation_error` for an unbound argument, `type_error(predicate_indicator, T)` for anything that is not `Name/Arity`, `permission_error(modify, table, ...)` from inside a running tabled evaluation. (On the engines deleted in 4.0.0/4.1.0 a malformed argument made the call fail silently.)

current_table/2

Purpose: Enumerates the tables that currently exist. Added with engine v4 (v3.12.0).

When to use: debugging a tabled program, or checking that a table was really discarded.

```
% Syntax: current_table(?Variant, ?Status)      Status = complete | incomplete
?- path(a, c), current_table(V, S).
V = path(a, c), S = complete ;
V = path(a, _), S = complete.
```

`Variant` is unified with each table's call pattern (so a partially instantiated `Variant` enumerates every table it matches, as `current_op/3` does); `Status` is `complete` for a table whose evaluation has finished and `incomplete` for one still being evaluated (only observable from inside a tabled computation).

26. HTTP Predicates

HTTP predicates provide both server-side and client-side HTTP capabilities, enabling JProlog programs to serve web APIs and consume external HTTP services.

http_server/2

Purpose: Starts an HTTP server on the specified port with given options.

When to use: Use to create web services or REST APIs from Prolog.

```
% Syntax: http_server(+Port, +Options)
?- http_server(8080, []).
true.

% Start server with registered handlers
start_api :-
    http_handler('/hello', handle_hello, []),
    http_server(8080, []).
```

http_stop/1

Purpose: Stops a running HTTP server on the specified port.

When to use: Use to gracefully shut down a server.

```
% Syntax: http_stop(+Port)
?- http_stop(8080).
true.
```

http_handler/3

Purpose: Registers a handler predicate for a URL path pattern.

When to use: Use to define routing for your HTTP server before starting it.

```
% Syntax: http_handler(+Path, +Handler, +Options)
?- http_handler('/api/users', handle_users, [method(get)]).
true.

handle_users(Request) :-
    http_reply(Request, 200, 'application/json', '{"users":[]}').
```

http_get_request/2

Purpose: Extracts request details (method, path, headers, body) from an HTTP request object.

When to use: Use inside handler predicates to inspect incoming requests.

```
% Syntax: http_get_request(+Request, -Details)
handle(Request) :-
    http_get_request(Request, Details),
    member(method(Method), Details),
    member(path(Path), Details).
```

http_reply/4

Purpose: Sends an HTTP response with a status code, content type, and body.

When to use: Use inside handler predicates to send responses.

```
% Syntax: http_reply(+Request, +StatusCode, +ContentType, +Body)
handle_hello(Request) :-
    http_reply(Request, 200, 'text/plain', 'Hello, World!').
```

http_reply_json/3

Purpose: Sends a JSON HTTP response (convenience wrapper around http_reply/4).

When to use: Use when building JSON APIs to avoid specifying content type manually.

```
% Syntax: http_reply_json(+Request, +StatusCode, +JsonTerm)
handle_user(Request) :-
    http_reply_json(Request, 200, json([name='Alice', age=30])).
```

http_client_get/2

Purpose: Performs an HTTP GET request and unifies with the response body.

When to use: Use to fetch data from external HTTP services.

```
% Syntax: http_client_get(+URL, -Response)
?- http_client_get('http://example.com/api/data', Response).
Response = '{"key":"value"}'.
```

http_client_post/3

Purpose: Performs an HTTP POST request with a body and unifies with the response.

When to use: Use to send data to external HTTP services.

```
% Syntax: http_client_post(+URL, +Body, -Response)
?- http_client_post('http://example.com/api/data', '{"key":"value"}', Response).
Response = '{"status":"ok"}'.
```

http_open/3

Purpose: Opens an HTTP connection as a stream for reading.

When to use: Use for streaming large HTTP responses or when you need fine-grained control over reading.

```
% Syntax: http_open(+URL, -Stream, +Options)
?- http_open('http://example.com/data.csv', Stream, []),
    read_stream_to_codes(Stream, Codes),
    close(Stream).
```

url_encode/2

Purpose: Percent-encodes a string for use in URLs.

When to use: Use when constructing URLs with user-supplied parameters.

```
% Syntax: url_encode(+Plain, -Encoded)
?- url_encode('hello world', E).
E = 'hello%20world'.
```

url_decode/2

Purpose: Decodes a percent-encoded URL string.

When to use: Use when parsing URL parameters from incoming requests.

```
% Syntax: url_decode(+Encoded, -Plain)
?- url_decode('hello%20world', D).
D = 'hello world'.
```

27. Persistence Predicates

Persistence predicates allow saving and loading the Prolog knowledge base to/from files, including support for JSON interchange format, predicate-level export, and snapshot/restore for transactional workflows.

db_save/1

Purpose: Saves the entire knowledge base to a Prolog-format file.

When to use: Use to persist all facts and rules to disk so they can be reloaded in a later session.

```
% Syntax: db_save(+File)
?- assert(person(alice, 30)),
   assert(person(bob, 25)),
   db_save('mydata.pl').
true.

% The file mydata.pl now contains:
% person(alice, 30).
% person(bob, 25).
```

db_load/1

Purpose: Loads facts and rules from a previously saved Prolog-format file into the knowledge base.

When to use: Use to restore a knowledge base saved with db_save/1 or any standard Prolog source file.

```
% Syntax: db_load(+File)
?- db_load('mydata.pl').
true.

?- person(alice, Age).
Age = 30.
```

db_save_predicate/2

Purpose: Saves only the clauses for a specific predicate to a file.

When to use: Use when you want to export a subset of the knowledge base rather than the entire database.

```
% Syntax: db_save_predicate(+Predicate/Arity, +File)
?- db_save_predicate(person/2, 'people.pl').
true.

% Only person/2 clauses are written to people.pl
```

persist/1

Purpose: Marks a predicate for automatic persistence. Changes to this predicate are automatically saved.

When to use: Use for predicates that should survive across sessions without explicitly calling db_save/1.

```
% Syntax: persist(+Predicate/Arity)
?- persist(config/2).
true.

?- assert(config(theme, dark)).
true.
% config/2 changes are automatically persisted
```

unpersist/1

Purpose: Removes automatic persistence marking from a predicate.

When to use: Use when you no longer need automatic persistence for a predicate.

```
% Syntax: unpersist(+Predicate/Arity)
?- unpersist(config/2).
true.
% config/2 changes are no longer automatically persisted
```

db_export_json/1

Purpose: Exports the knowledge base to a JSON file.

When to use: Use for interoperability with other systems that consume JSON data.

```
% Syntax: db_export_json(+File)
?- assert(employee(john, engineering)),
   db_export_json('kb.json').
true.

% kb.json contains a JSON representation of all facts and rules
```

db_import_json/1

Purpose: Imports facts and rules from a JSON file into the knowledge base.

When to use: Use to load data exported by db_export_json/1 or generated by external tools.

```
% Syntax: db_import_json(+File)
?- db_import_json('kb.json').
true.

?- employee(john, Dept).
Dept = engineering.
```

db_snapshot/1

Purpose: Captures a named snapshot of the current knowledge base state.

When to use: Use before making bulk changes so you can roll back if needed.

```
% Syntax: db_snapshot(+Name)
?- db_snapshot(before_update).
true.

?- retractall(person(_, _)).
true.
% Knowledge base is now empty for person/2, but snapshot is saved
```

db_restore/1

Purpose: Restores the knowledge base to a previously captured snapshot.

When to use: Use to roll back changes when an operation fails or produces incorrect results.

```
% Syntax: db_restore(+Name)
?- db_restore(before_update).
true.
```

```
?- person(alice, Age).
Age = 30.
% Knowledge base restored to its state at snapshot time
```

db_clear/0

Purpose: Removes all user-defined facts and rules from the knowledge base.

When to use: Use to reset the knowledge base to an empty state, for example before loading fresh data.

```
% Syntax: db_clear
?- db_clear.
true.

?- person(_, _).
false.
% All user-defined clauses have been removed
```

```
<!-- START_CHANGE: ISS-2025-0179 - Add db_transaction/1, db_sync/0, db_batch_assert/1 -->
```

db_transaction/1

Purpose: Executes a goal within a transaction; rolls back changes if the goal fails or throws.

```
% Syntax: db_transaction(+Goal)
?- db_transaction((assert(account(alice, 100)), assert(account(bob, 200)))).
true.

% If the goal fails, no changes are committed
?- db_transaction((assert(temp(1)), fail)).
false.
% temp(1) was not asserted
```

db_sync/0

Purpose: Forces all pending persistence writes to be flushed to disk.

```
% Syntax: db_sync
?- db_sync.
true.
```

db_batch_assert/1

Purpose: Asserts a list of clauses in bulk, more efficiently than individual assert calls.

```
% Syntax: db_batch_assert(+ClauseList)
?- db_batch_assert([fact(a), fact(b), fact(c)]).
true.

?- fact(X).
X = a ; X = b ; X = c.
```

```
<!-- END_CHANGE: ISS-2025-0179 -->
```

28. Graph Algorithm Predicates

Graph algorithm predicates provide common graph operations including path finding, shortest path computation, connectivity analysis, topological sorting, minimum spanning trees, and cycle detection. Graphs are represented as lists of edges in the form `edge(From, To)` or `edge(From, To, Weight)` for weighted graphs.

graph_path/4

Purpose: Finds a path between two vertices in a graph.

When to use: Use to determine if two nodes are connected and to retrieve the connecting path.

```
% Syntax: graph_path(+Edges, +Start, +End, -Path)
?- Edges = [edge(a,b), edge(b,c), edge(c,d), edge(b,d)],
   graph_path(Edges, a, d, Path).
Path = [a, b, c, d] ;
Path = [a, b, d].
```

shortest_path/4

Purpose: Finds the shortest (minimum weight) path between two vertices in a weighted graph.

When to use: Use for route planning, network optimization, or any problem requiring minimum-cost paths.

```
% Syntax: shortest_path(+Edges, +Start, +End, -Path)
?- Edges = [edge(a,b,1), edge(b,c,2), edge(a,c,10)],
   shortest_path(Edges, a, c, Path).
Path = [a, b, c].
```

graph_connected/2

Purpose: Checks whether two vertices are connected in the graph (i.e., a path exists between them).

When to use: Use for simple reachability checks without needing the actual path.

```
% Syntax: graph_connected(+Edges, +Vertex-Vertex)
?- Edges = [edge(a,b), edge(b,c)],
   graph_connected(Edges, a-c).
true.

?- graph_connected(Edges, a-d).
false.
```

graph_vertices/2

Purpose: Extracts the set of all vertices from a graph's edge list.

When to use: Use to enumerate all nodes in a graph.

```
% Syntax: graph_vertices(+Edges, -Vertices)
?- graph_vertices([edge(a,b), edge(b,c), edge(c,a)], Vs).
Vs = [a, b, c].
```

graph_edges/2

Purpose: Extracts all edges from a graph representation, normalizing them to a uniform format.

When to use: Use to inspect or iterate over all edges in a graph.

```
% Syntax: graph_edges(+Graph, -Edges)
?- graph_edges([edge(a,b,1), edge(b,c,2)], Es).
Es = [edge(a, b, 1), edge(b, c, 2)].
```

graph_neighbors/3

Purpose: Finds all neighbors (adjacent vertices) of a given vertex.

When to use: Use to explore the local structure of a graph around a specific node.

```
% Syntax: graph_neighbors(+Edges, +Vertex, -Neighbors)
?- Edges = [edge(a,b), edge(a,c), edge(b,d)],
   graph_neighbors(Edges, a, Ns).
Ns = [b, c].
```

topological_sort/2

Purpose: Produces a topological ordering of vertices in a directed acyclic graph (DAG).

When to use: Use for dependency resolution, task scheduling, or build ordering.

```
% Syntax: topological_sort(+Edges, -Sorted)
?- Edges = [edge(a,b), edge(a,c), edge(b,d), edge(c,d)],
   topological_sort(Edges, Sorted).
Sorted = [a, c, b, d].
```

graph_components/2

Purpose: Finds all connected components in an undirected graph.

When to use: Use to identify clusters or disconnected subgraphs.

```
% Syntax: graph_components(+Edges, -Components)
?- Edges = [edge(a,b), edge(c,d)],
   graph_components(Edges, Cs).
Cs = [[a, b], [c, d]].
```

minimum_spanning_tree/2

Purpose: Computes the minimum spanning tree of a weighted undirected graph.

When to use: Use for network design problems where you need to connect all nodes at minimum total cost.

```
% Syntax: minimum_spanning_tree(+Edges, -MST)
?- Edges = [edge(a,b,1), edge(b,c,2), edge(a,c,3)],
   minimum_spanning_tree(Edges, MST).
MST = [edge(a, b, 1), edge(b, c, 2)].
```

graph_degree/3

Purpose: Computes the degree (number of incident edges) of a vertex.

When to use: Use to analyze the connectivity of individual nodes, find hubs, or identify leaf nodes.

```
% Syntax: graph_degree(+Edges, +Vertex, -Degree)
?- Edges = [edge(a,b), edge(a,c), edge(a,d)],
   graph_degree(Edges, a, D).
D = 3.
```

graph_has_cycle/1

Purpose: Checks whether a directed graph contains a cycle.

When to use: Use to validate that a dependency graph is a DAG before performing topological sort.

```
% Syntax: graph_has_cycle(+Edges)
?- graph_has_cycle([edge(a,b), edge(b,c), edge(c,a)]).
true.

?- graph_has_cycle([edge(a,b), edge(b,c)]).
false.
```

graph_reachable/3

Purpose: Finds all vertices reachable from a given starting vertex.

When to use: Use to compute the transitive closure from a single source node.

```
% Syntax: graph_reachable(+Edges, +Start, -Reachable)
?- Edges = [edge(a,b), edge(b,c), edge(b,d)],
   graph_reachable(Edges, a, Rs).
Rs = [b, c, d].
```

<!-- START_CHANGE: ISS-2025-0179 - Add graph_scc/2 -->

graph_scc/2

Purpose: Computes the strongly connected components (SCCs) of a directed graph using Tarjan's algorithm.

```
% Syntax: graph_scc(+Edges, -SCCs)
?- Edges = [edge(a,b), edge(b,c), edge(c,a), edge(d,e)],
   graph_scc(Edges, SCCs).
SCCs = [[a, b, c], [d], [e]].
```

<!-- END_CHANGE: ISS-2025-0179 -->

29. Java FFI Predicates

The Java Foreign Function Interface (FFI) predicates allow Prolog programs to interact with Java classes, objects, methods, fields, and arrays. This enables seamless interoperability between Prolog logic and Java libraries.

Understanding Java FFI

The FFI bridges Prolog and Java by:

- Creating Java objects from Prolog
- Calling Java methods and accessing fields
- Working with Java arrays
- Converting between Java objects and Prolog terms

Java objects are referenced by opaque handles (atoms) that can be passed between FFI predicates.

java_new/3

Purpose: Create a new Java object by calling a constructor.

```
% java_new(+ClassName, +ArgsList, -ObjectRef)
?- java_new('java.util.ArrayList', [], Ref).
Ref = java_obj_1.

?- java_new('java.lang.StringBuilder', ['Hello'], Ref).
Ref = java_obj_2.
```

java_call/4

Purpose: Call a method on a Java object or class (for static methods).

```
% java_call(+ObjectRef, +MethodName, +ArgsList, -Result)
?- java_new('java.util.ArrayList', [], List),
   java_call(List, add, ['hello'], _),
   java_call(List, size, [], Size).
Size = 0.

% Static method call
?- java_call('java.lang.Math', max, [3, 7], Result).
Result = 7.
```

java_get_field/3

Purpose: Get the value of a field on a Java object or class.

```
% java_get_field(+ObjectOrClass, +FieldName, -Value)
?- java_get_field('java.lang.Integer', 'MAX_VALUE', Val).
Val = 2147483647.
```

java_set_field/3

Purpose: Set the value of a field on a Java object.

```
% java_set_field(+ObjectRef, +FieldName, +Value)
?- java_new('MyClass', [], Obj),
   java_set_field(Obj, count, 42).
```

java_instanceof/2

Purpose: Check if a Java object is an instance of a given class.

```
% java_instanceof(+ObjectRef, +ClassName)
?- java_new('java.util.ArrayList', [], Obj),
   java_instanceof(Obj, 'java.util.List').
true.
```

java_class/2

Purpose: Get the class name of a Java object.

```
% java_class(+ObjectRef, -ClassName)
?- java_new('java.util.HashMap', [], Obj),
   java_class(Obj, Class).
Class = 'java.util.HashMap'.
```

java_array_new/3

Purpose: Create a new Java array of a given type and size.

```
% java_array_new(+ElementType, +Size, -ArrayRef)
?- java_array_new(int, 5, Arr).
Arr = java_arr_1.
```

java_array_get/3

Purpose: Get an element from a Java array by index.

```
% java_array_get(+ArrayRef, +Index, -Value)
?- java_array_get(Arr, 0, Val).
Val = 0.
```

java_array_set/3

Purpose: Set an element in a Java array at a given index.

```
% java_array_set(+ArrayRef, +Index, +Value)
?- java_array_set(Arr, 0, 42).
true.
```

java_array_length/2

Purpose: Get the length of a Java array.

```
% java_array_length(+ArrayRef, -Length)
?- java_array_length(Arr, Len).
Len = 5.
```

java_to_term/2

Purpose: Convert a Java object to a Prolog term representation.

```
% java_to_term(+ObjectRef, -Term)
?- java_new('java.lang.Integer', [42], Obj),
```

```
java_to_term(Obj, Term).
Term = 42.
```

java_from_term/2

Purpose: Convert a Prolog term to a Java object.

```
% java_from_term(+Term, -ObjectRef)
?- java_from_term(hello, Obj),
   java_class(Obj, Class).
Class = 'java.lang.String'.
```

<!-- START_CHANGE: ISS-2025-0179 - Add java_release_ref/1 and java_gc/0 -->

java_release_ref/1

Purpose: Releases a Java object reference, allowing it to be garbage collected.

```
% Syntax: java_release_ref(+ObjectRef)
?- java_new('java.util.ArrayList', [], Ref),
   java_release_ref(Ref).
true.
```

java_gc/0

Purpose: Requests Java garbage collection to free unreferenced objects.

```
% Syntax: java_gc
?- java_gc.
true.
```

<!-- END_CHANGE: ISS-2025-0179 -->

<!-- START_CHANGE: ISS-2025-0466..0471 - engine v4 wave W6: the module system -->

30. Concurrent Execution Predicates

Parallel goal execution using Java threads.

Since v4.0.0 (ISS-2025-0480) every goal here runs on its own resolution machine over the same engine: the clause store is shared and thread-safe, the goal is copied so no variable is shared between workers, each worker carries the parent's inference budget in its own counter, and interrupting the parent cancels the workers. Goals still have to be independent - two workers asserting to the same predicate see each other's writes.

concurrent/3

Execute a list of goals using at most N worker threads. All must succeed.

```
concurrent(4, [goal1, goal2, goal3], [])
```

concurrent_maplist/2

Like maplist/2 but parallel. `call(Goal, Elem)` for each element.

```
concurrent_maplist(is_positive, [1, 2, 3, 4, 5])
```

concurrent_maplist/3

Like maplist/3 but parallel. `call(Goal, Elem, Result)` collecting results in order.

```
concurrent_maplist(square, [1,2,3,4], [1,4,9,16])
```

(Before v4.0.0 this was registered under the uncallable name `concurrent_maplist3` and `concurrent_maplist/3` raised an arity error - ISS-2025-0480.)

concurrent_maplist/4

Parallel maplist with two input lists.

```
concurrent_maplist(add, [1,2,3], [10,20,30], [11,22,33])
```

(Before v4.0.0: `concurrent_maplist4`, likewise uncallable.)

first_solution/3

Run goals in parallel, return bindings from the first to succeed. OR-parallelism.

```
first_solution(X, [search_db1(X), search_db2(X), search_db3(X)], [])
```

concurrent_and/2

AND-parallelism: all goals must succeed.

```
concurrent_and([check1, check2, check3], [])
```

concurrent_or/2

OR-parallelism: returns 1-based index of the first goal to succeed.

```
concurrent_or([strategy1, strategy2], WinnerIndex)
```

31. Module System

Version note. Everything in this section arrived with engine v4 (v3.13.0, the default since 4.0.0). The older `ModuleManager` behaviour - `Module:Goal` reaching only user-defined clauses, no library modules, no `autoload`, `meta_predicate/1` recorded but never consulted, no `current_module/1` - belonged to the engine deleted in 4.1.0.

The three kinds of module

Module	What it holds
user	The default. It is the flat knowledge base - the clauses <code>consult/1</code> , <code>assertz/1</code> and <code>listing/1</code> see.
system	The built-in predicates (both the v4 natives and the ~400 registry ones). It holds no clauses. <code>Prolog.enableSafeMode()</code> removes the host-touching built-ins from it.
library modules	<code>lists</code> , <code>apply</code> , <code>pairs</code> , <code>coroutining</code> - written in Prolog, shipped as classpath resources under <code>prelude/</code> , and autoloaded by predicate indicator : a module is parsed the first time one of its predicates is referenced.

Resolution order

For an unqualified call to `f/n` from module `M`:

```
M's own clauses -> M's imports (in import order, exported predicates only)
                  -> user (the flat knowledge base)
                  -> the autoloaded library modules
                  -> system (the built-ins)
```

Two practical consequences:

- **A definition in the calling context wins over the library.** A program that defines its own `partition/4` gets its own, while `apply:partition/4` stays reachable under its qualified name. A definition inside module `M` overrides only for `M` and the modules that import it.
- **Built-ins are still not redefinable.** `system` is consulted last in the list above, but the machine checks the built-in tables *before* the clause layers for compatibility, and `consult/1` refuses a clause whose head is a registered built-in (Cannot redefine built-in predicate `f/n`). The library layer is the documented way to override a library predicate.

Module:Goal

```
?- lists:append([1], [2], L).      % a library predicate
L = [1, 2].

?- system:atom_length(abc, N).    % a built-in, explicitly
N = 3.

?- user:my_fact(X).               % the flat knowledge base, explicitly
```

- The **innermost** qualification of a nested `a:b:Goal` wins, so `user:lists:append(...)` runs in `lists`.
- **Export enforcement:** a module that *defines* `f/n` answers a qualified call only if it also exports it. `:- module(secret, []). followed by hidden(42).` makes `secret:hidden(X)` fail.
- A module that does **not** define `f/n` falls through to the ordinary resolution in its own context, which is why `lists:length(L, N)` (a native) and `othermodule:my_user_fact(X)` (a user predicate) both work.
- An unknown module name behaves like `user:`.

Directives

Directive	Effect
<code>:- module(Name, [f/1, g/2]).</code>	Declare the current module and its export list. An empty list exports nothing.
<code>:- use_module(Name).</code>	Import every predicate <code>Name</code> exports into the current module.
<code>:- use_module(Name, [f/1]).</code>	Import only the listed predicates.
<code>:- use_module(library(X)).</code>	Accepted for the library modules; they autoload anyway, so it is a no-op.
<code>:- meta_predicate(Spec).</code>	See below.

meta_predicate/1

`:- meta_predicate(maplist(2, ?, ?)).` says that argument 1 of `maplist/3` is a goal that will be called with 2 extra arguments. When a predicate with such a declaration is called from module `C`, its **module-sensitive** arguments are qualified with `C` before the clause head is unified, so the callee's `call/N` runs them in the caller's context.

Argument specifiers: 0-9 (a goal called with that many extra arguments), `:` (a module-sensitive term), `^`, `//`; `+`, `-`, `?` and anything else are ordinary arguments.

```
:- module(m1, [go/1]).
helper(from_m1).
mk(_, R) :- helper(R).
go(L) :- maplist(mk, [x], L).      % mk/2 resolves in m1, not in apply and not in user
```

current_module/1

```
?- current_module(lists).
true.

?- current_module(M).          % enumerates: user first, then the rest, sorted
M = user ;
M = apply ;
M = coroutining ;
...
```

predicate_property/2 module properties

In addition to `built_in`, `dynamic`, `static` and `defined`:

Property	Meaning
<code>defined_in(Module)</code>	The module whose clauses would answer the call from the current context.
<code>exported</code>	That module exports the predicate.
<code>imported_from(Module)</code>	The predicate is visible here but defined elsewhere.

```
?- predicate_property(append(_, _, _), imported_from(lists)).
true.
```

What the library modules contain

Module	Predicates
<code>lists</code>	<code>member/2</code> , <code>memberchk/2</code> , <code>append/3</code> , <code>select/3</code> , <code>selectchk/3</code> , <code>nth0/3</code> , <code>nth1/3</code> , <code>last/2</code> , <code>reverse/2</code> , <code>plus length/2</code> , <code>msort/2</code> , <code>sort/2</code> , <code>sort/4</code> , <code>sum_list/2</code> , <code>sumlist/2</code> , <code>numlist/3</code> , <code>permutation/2</code> , <code>max_list/2</code> , <code>min_list/2</code> , <code>subtract/3</code> , <code>intersection/3</code> , <code>union/3</code>

apply	maplist/2..7, foldl/4..7, include/3, exclude/3, partition/4, partition/5
pairs	pairs_keys_values/3, pairs_keys/2, pairs_values/2
coroutining	freeze/2, frozen/2, when/2, dif/2, ?=/2

Most of `lists` is implemented as a native generator for speed (on a 1 000 000-element list a Prolog clause walk pushes one choice point per element); `member/2` and `append/3` additionally exist as the two-clause Prolog definitions of module `lists`, which is what the qualified form runs. The two are observationally identical.

Behaviour differences from the default engine

- `append(X, Y, Z)` with all three arguments open **enumerates** on v4 (`X = [], [], [], ...` lazily) where the default engine stops at the single standard solution. A program that relied on that termination will loop.
- `member(X, PartialList)` **extends** the open tail on v4, and `memberchk(a, L)` binds `L = [a|_]`; both fail on the default engine.
- `memberchk/2` and `current_module/1` do not exist at all on the default engine.

<!-- END_CHANGE: ISS-2025-0466..0471 -->

32. String predicates

JProlog strings ("text" with the default `double_quotes=string` flag) are a distinct atomic type. All `string_*` predicates also accept atoms and numbers as text input, and the `atom_*` predicates accept strings, so the two families interoperate freely.

string/1

Purpose: Type check - succeeds if the argument is a string object.

```
?- string("abc").
true.

?- string(abc).
false.
```

string_length/2

Purpose: `string_length(+Text, -Length)` - number of characters in a string (or atom/number).

```
?- string_length("hello", N).
N = 5.

?- string_length(hello, N).
N = 5.
```

string_concat/3

Purpose: `string_concat(?S1, ?S2, ?S3)` - concatenation of strings; with `S3` bound and the others unbound it enumerates every split on backtracking.

```
?- string_concat("ab", "cd", S).
S = "abcd".

?- string_concat(X, Y, "ab").
X = "", Y = "ab" ;
X = "a", Y = "b" ;
X = "ab", Y = "".
```

string_chars/2, string_codes/2

Purpose: Convert between a string and its list of characters / character codes; both directions work.

```
?- string_chars("hi", Cs), string_codes("hi", Codes).
Cs = [h, i], Codes = [104, 105].

?- string_chars(S, [o, k]).
S = "ok".
```

string_code/3

Purpose: `string_code(+Index, +String, -Code)` - code of the character at a 1-based index.

```
?- string_code(2, "abc", C).
C = 98.
```

sub_string/5

Purpose: `sub_string(+String, ?Before, ?Length, ?After, ?Sub)` - enumerate or test substrings, exactly like `sub_atom/5` but producing strings.

```
?- sub_string("hello world", 6, 5, _, S).
S = "world".

?- sub_string("abc", B, 1, A, S).
B = 0, A = 2, S = "a" ;
B = 1, A = 1, S = "b" ;
B = 2, A = 0, S = "c".
```

string_to_atom/2, atom_string/2

Purpose: Convert between strings and atoms in either direction (`string_to_atom(?String, ?Atom)`, `atom_string(?Atom, ?String)`). At least one argument must be bound.

```
?- atom_string(hello, S), string_to_atom("world", A).
S = "hello", A = world.

?- atom_string(A, "42").
A = '42'.
```

number_string/2

Purpose: `number_string(?Number, ?String)` - parse a string as a number (leading/trailing whitespace allowed) or render a number as a string. A float keeps its type.

```
?- number_string(N, " 3.0 "), number_string(42, S).
N = 3.0, S = "42".

?- number_string(N, "abc").
false.
```

Unlike `number_codes/2`, which raises `syntax_error(illegal_number)`, `number_string/2` fails silently on text that is not a number (SWI-Prolog behaviour).

```
?- catch(number_codes(N, "abc"), error(E, _), true).
E = syntax_error(illegal_number).
```

split_string/4

Purpose: `split_string(+String, +SepChars, +PadChars, -SubStrings)` - split on any separator character, stripping pad characters from each part.

```
?- split_string("a,b,,c", ",", "", P).
P = ["a", "b", "", "c"].

?- split_string(" key = value ", "=", " ", P).
P = ["key", "value"].
```

number_codes/2, atom_to_number/2, number_to_atom/2, to_codes/2

Purpose: `number_codes(?Number, ?Codes)` is the ISO conversion between a number and its code list (accepting `0x`, `0o`, `0b`, `0'c` notations and raising `syntax_error` on bad text). `atom_to_number(+Atom, -Number)` and `number_to_atom(+Number, -Atom)` are convenience aliases of `atom_number/2`; `to_codes(+Text, -Codes)` turns any atom, string or number into a code list.

```
?- number_codes(N, "0x1F"), number_codes(2.5, C).  
N = 31, C = [50, 46, 53].  
  
?- atom_to_number('12', N), number_to_atom(3.5, A), to_codes(hi, Cs).  
N = 12, A = '3.5', Cs = [104, 105].
```

upcase_atom/2, downcase_atom/2

Purpose: Case conversion of an atom (or string) into an atom.

```
?- upcase_atom('Hello', U), downcase_atom('Hello', D).  
U = 'HELLO', D = hello.
```

33. Stream predicates

Streams are created by `open/3,4` and referred to by the stream term it returns or by an alias (`user_input`, `user_output`, `user_error`, or an alias given with `open/4`). The predicates below complete the file/stream chapter of the ISO core.

current_input/1, current_output/1

Purpose: Unify the argument with the current input / output stream. The current output stream is per thread, so a query running in the IDE or in a worker thread sees its own console.

```
?- current_output(S), write(S, hello), nl(S).
hello
S = user_output.
```

set_input/1, set_output/1

Purpose: Make a stream (or alias) the current input / output for the subsequent `read/1`, `write/1`, `nl/0` ... calls that take no explicit stream.

```
capture(File, Goal) :-
    open(File, write, S),
    current_output(Old),
    set_output(S),
    call(Goal),
    set_output(Old),
    close(S).

?- capture('out.txt', (write(hello), nl)).
true.
```

flush_output/0, flush_output/1

Purpose: Force buffered output to be written (to the current output stream, or to the given one). Useful before a blocking read or a long computation.

```
?- write('Name? '), flush_output, read(Name).
```

close/1, close/2

Purpose: Close a stream. `close(Stream, Options)` accepts `force(true)` to ignore errors while closing. Closing `user_input`/`user_output` is a no-op.

```
?- open('data.txt', read, S), read(S, T), close(S, [force(true)]).
```

stream_property/2

Purpose: `stream_property(?Stream, ?Property)` - enumerate stream properties. Both arguments may be unbound, so the predicate enumerates every open stream and every property of it. The complete ISO set is reported: `file_name(F)`, `mode(M)`, `input`, `output`, `alias(A)`, `position(P)`, `end_of_stream(E)`, `eof_action(A)`, `reposition(B)`, `type(T)`, `text/binary`, `encoding(E)` and `line_count(N)`.

```
?- stream_property(user_error, alias(A)).
A = user_error.
```

```
?- open('data.txt', read, S), stream_property(S, mode(M)), stream_property(S, type(T)), close(S).
M = read, T = text.
```

An unbound first argument is bound to the **canonical** stream term ('\$stream'(N)), never to an alias: `stream_property(S, alias(user_error))` gives `S = '$stream'(2)`. Enumerating never blocks - `end_of_stream` is reported as not for a stream that cannot be repositioned (stdin, a socket), because deciding it there would mean waiting for input.

current_stream/3, set_stream/2

Purpose: `current_stream(?File, ?Mode, ?Stream)` enumerates the open streams with their file name and mode. `set_stream(+Stream, +Property)` changes a property of an open stream; `alias(A)` adds an alias that every stream argument then accepts.

```
?- open('data.txt', read, S), current_stream(F, M, S), close(S).
F = 'data.txt', M = read.

?- open('data.txt', read, S), set_stream(S, alias(input_file)),
   get_char(input_file, C), close(S).
C = h.
```

stream_position/2, set_stream_position/2, seek/4

Purpose: `stream_position(+Stream, -Pos)` reads the byte position of a file stream; `set_stream_position(+Stream, +Pos)` repositions it (permission_error(reposition, stream, S) for streams that cannot be repositioned); `seek(+Stream, +Offset, +Method, -NewPos)` moves relative to bof, current or eof. Since v3.14.0 a **text** stream is decoded one code point at a time through its own decoder, so repositioning really does change what the next `get_char/2` reads - it is no longer binary-only.

```
% data.txt contains the text "hello"
?- open('data.txt', read, S), get_char(S, C1), stream_position(S, P),
   get_char(S, C2), set_stream_position(S, P), get_char(S, C3), close(S).
C1 = h, P = 1, C2 = e, C3 = e.

?- open('data.txt', read, S, [type(binary)]), get_byte(S, B1),
   seek(S, 0, bof, N), get_byte(S, B2), close(S).
B1 = 104, N = 0, B2 = 104.
```

character_count/2, line_count/2, line_position/2, stream_position_data/3

Purpose: `character_count(+Stream, -N)`, `line_count(+Stream, -N)` and `line_position(+Stream, -N)` report how far a stream has been read or written: the number of characters consumed, the current line number (1-based) and the column within it (0-based). `stream_position_data(+Field, +Position, -Value)` pulls the same three fields out of the opaque position term that `stream_property(S, position(P))` yields; Field is `char_count`, `line_count` or `line_position`.

```
% data.txt contains the text "hello"
?- open('data.txt', read, S), get_char(S, _), get_char(S, _),
   character_count(S, N), line_count(S, L), line_position(S, C), close(S).
N = 2, L = 1, C = 2.

?- open('data.txt', read, S), get_char(S, _),
   stream_property(S, position(P)), stream_position_data(char_count, P, N), close(S).
N = 1.
```

peek_char/1,2, peek_code/1,2, peek_byte/1,2

Purpose: Look at the next character / code / byte of the input without consuming it. Return `end_of_file` (or -1 for codes and bytes) at the end of the stream.

```
skip_spaces(S) :- peek_char(S, C), C == ' ', !, get_char(S, _), skip_spaces(S).
skip_spaces(_).
```

put_byte/1,2, get_byte/1,2

Purpose: Binary I/O on streams opened with `type(binary)`.

```
?- open('raw.bin', write, S, [type(binary)]), put_byte(S, 255), put_byte(S, 0), close(S),
   open('raw.bin', read, R, [type(binary)]), get_byte(R, B1), get_byte(R, B2), close(R).
B1 = 255, B2 = 0.
```

at_end_of_stream/0,1

Purpose: Succeeds when the (current or given) input stream has no more data.

```
read_all(S, []) :- at_end_of_stream(S), !.
read_all(S, [C|Cs]) :- get_char(S, C), read_all(S, Cs).
```

34. Global variables

Global variables are a per-engine key/value store. `nb_setval/2` copies the value and keeps it across backtracking; `b_setval/2` stores the value so that it is restored on backtracking.

`nb_setval/2`, `nb_getval/2`

Purpose: Set / read a non-backtrackable global variable. Reading an unset name raises `existence_error(variable, Name)`.

```
count_solutions(Goal, N) :-
    nb_setval(cnt, 0),
    ( call(Goal), nb_getval(cnt, C0), C is C0 + 1, nb_setval(cnt, C), fail ; true ),
    nb_getval(cnt, N).

?- count_solutions(member(_, [a, b, c]), N).
N = 3.
```

`b_setval/2`, `b_getval/2`

Purpose: Backtrackable global variables: the previous value is restored when execution backtracks over the `b_setval/2` call.

```
?- b_setval(v, 1), ( b_setval(v, 2), fail ; true ), b_getval(v, V).
V = 1.
```

`nb_current/2`, `nb_delete/1`

Purpose: `nb_current(?Name, ?Value)` enumerates the defined global variables; `nb_delete(+Name)` removes one (silently succeeds if it does not exist).

```
?- nb_setval(a, 1), nb_setval(b, two), findall(K-V, nb_current(K, V), L), nb_delete(a).
L = [a-1, b-two].
```

35. Attributed variables and coroutining

Attributed variables carry extra data that is consulted when the variable is bound. They are the basis of `freeze/2`, `dif/2`, `when/2` and of the CLP(FD) solver. Attributes survive between top-level queries for the same variable name in the same engine.

`put_attr/3`, `get_attr/3`, `del_attr/2`, `attvar/1`

Purpose: `put_attr(+Var, +Module, +Value)` attaches (or replaces) the attribute `Module` of an unbound variable; `get_attr(+Var, +Module, -Value)` reads it; `del_attr(+Var, +Module)` removes it; `attvar(@Term)` succeeds if the term is a variable with at least one attribute.

```
?- put_attr(X, colour, red), get_attr(X, colour, C), attvar(X).
C = red.

?- put_attr(X, colour, red), del_attr(X, colour), attvar(X).
false.
```

`freeze/2`

Purpose: `freeze(?Var, :Goal)` - delay `Goal` until `Var` is bound; if `Var` is already bound the goal runs immediately. The goal runs at the moment of binding, inside the unification.

```
?- freeze(X, (write(bound(X)), nl)), X = 42.
bound(42)
X = 42.

?- freeze(X, X > 3), member(X, [1, 5, 2, 7]).
X = 5 ;
X = 7.
```

`dif/2`

Purpose: `dif(?X, ?Y)` - constrains two terms to be different; fails at once when they are identical, succeeds when they can never unify, and otherwise suspends until the decision can be made.

```
?- dif(X, a), member(X, [a, b, c]).
X = b ;
X = c.

?- dif(f(X), f(Y)), X = 1, Y = 1.
false.
```

`when/2`

Purpose: `when(+Condition, :Goal)` - run `Goal` as soon as `Condition` holds. Conditions: `nonvar(X)`, `ground(T)`, `?(X, Y)` (decidable comparison), and conjunctions/disjunctions of these.

```
?- when(ground(X-Y), (Z is X + Y, write(sum(Z)), nl)), X = 1, Y = 2.
sum(3)
X = 1, Y = 2, Z = 3.

?- when((nonvar(A) ; nonvar(B)), write(ready)), B = 1.
ready
B = 1.
```

A woken goal runs in the query's own binding context, so the bindings it makes (`Z` above) are ordinary bindings, it is traced like any other goal, it is charged to the inference budget, and it can throw into the

enclosing catch/3. (Before 4.0.0 a when/2-woken goal's bindings were lost on the then-default engine and only its side effects were visible; that is fixed.)

frozen/2, ?=/2

Purpose: frozen(?Var, -Goal) returns the conjunction of goals currently delayed on Var, or true when there are none. ?=(X, Y) succeeds when the comparison of X and Y is already decidable - that is, when they are identical or can never unify.

```
?- freeze(X, writeln(hi)), frozen(X, G).
G = writeln(hi).

?- frozen(Y, G).
G = true.

?- ?=(a, b).
true.

?- ?=(X, Y).
false.
```

term_attvars/2, copy_term/3, unifiable/3

Purpose: term_attvars(+Term, -Vars) collects the attributed variables reachable from Term. copy_term(+Term, -Copy, -Attributes) copies Term without its attributes and returns them as a list of goals that would restore them. unifiable(@X, @Y, -Unifier) returns the list of Var = Value bindings that unifying X and Y would make, **without** making them.

```
?- put_attr(X, mymod, 1), term_attvars(f(X, Y), Vs).
Vs = [X].

?- put_attr(X, mymod, 1), copy_term(X, Y, Attrs).
Attrs = [put_attr(Y, mymod, 1)].

?- unifiable(f(X, b), f(a, Y), U).
U = [X = a, Y = b].
```

36. More list and pair predicates

msort/2

Purpose: `msort(+List, -Sorted)` - sort in the standard order of terms **without** removing duplicates (`sort/2` removes them).

```
?- msort([c, a, b, a], L).
L = [a, a, b, c].
```

predsort/3

Purpose: `predsort(:Pred, +List, -Sorted)` - sort with a user comparison called as `call(Pred, Order, A, B)` where `Order` is `<`, `=` or `>`. Elements that compare `=` are merged into one.

```
by_length(0, A, B) :- atom_length(A, LA), atom_length(B, LB), compare(0, LA-A, LB-B).

?- predsort(by_length, [ccc, a, bb], L).
L = [a, bb, ccc].
```

min_list/2, max_list/2, sum_list/2, sumlist/2

Purpose: Minimum, maximum and sum of a list of numbers (`sumlist/2` is an alias of `sum_list/2`). The result keeps the type of the operands: a list containing a float sums to a float.

```
?- min_list([3, 1.5, 2], Min), max_list([3, 1.5, 2], Max), sum_list([1, 2, 3], S), sum_list([1.5, 1.5], F).
Min = 1.5, Max = 3, S = 6, F = 3.0.
```

nth0/3, nth1/3

Purpose: Element at a 0-based / 1-based index; with the index unbound they enumerate.

```
?- nth1(2, [a, b, c], X), nth0(I, [a, b, c], c).
X = b, I = 2.
```

partition/4, partition/5

Purpose: `partition(:Pred, +List, ?Included, ?Excluded)` splits a list in one pass; `partition(:Pred, +List, ?Less, ?Equal, ?Greater)` uses `call(Pred, X, Order)` with `Order` one of `<`, `=`, `>`. Both are library predicates written in Prolog, so **a user definition of `partition/4` overrides them** (many textbook quicksorts define their own, taking a pivot rather than a goal).

```
?- partition([X]>>(X > 2), [1, 2, 3, 4], Big, Small).
Big = [3, 4], Small = [1, 2].
```

exclude/3, include/3

Purpose: Filter a list with a goal: `include(:Goal, +List, -Kept)` keeps the elements for which `call(Goal, Elem)` succeeds, `exclude/3` keeps the others. (`partition/4` is deliberately not provided as a built-in so that programs may define their own `partition/4`, as quicksort examples commonly do.)

```
even(X) :- 0 is X mod 2.
```

```
?- exclude(even, [1, 2, 3, 4], Odd), include(even, [1, 2, 3, 4], Even).
Odd = [1, 3], Even = [2, 4].
```

pairs_keys_values/3, pairs_keys/2, pairs_values/2

Purpose: Convert between a list of Key-Value pairs and its keys / values.

```
?- pairs_keys_values(P, [a, b], [1, 2]), pairs_keys(P, K), pairs_values(P, V).
P = [a-1, b-2], K = [a, b], V = [1, 2].
```

37. Term inspection and modification

put_code/1,2

Purpose: `put_code(+Code)` writes the character with the given code to the current output; `put_code(+Stream, +Code)` writes it to `Stream`. The counterpart of `get_code/1, 2`.

```
?- put_code(0'h), put_code(0'i), nl.
hi
true.
```

v4.2.0: the two-argument form actually writes to `Stream`. Before, the arity entry existed but the implementation raised "put_code/1 requires exactly 1 argument".

unify_with_occurs_check/2

Purpose: ISO unification **with** the occurs check, whatever the `occurs_check` flag says: it fails rather than building a cyclic term.

```
?- unify_with_occurs_check(X, f(Y)).
X = f(Y).

?- unify_with_occurs_check(X, f(X)).
false.
```

setarg/3

Purpose: `setarg(+Index, +Term, +Value)` - destructively replace the `Index`-th argument of a compound term; the change is undone on backtracking.

```
?- T = f(a, b), setarg(1, T, z).
T = f(z, b).

?- T = f(a), ( setarg(1, T, z), fail ; true ).
T = f(a).
```

nb_setarg/3

Purpose: `nb_setarg(+N, +Compound, +Value)` replaces the `N`-th argument like `setarg/3`, but the change is **not undone on backtracking**. Used to accumulate a result across a failure-driven loop.

```
?- T = counter(0), forall(member(_, [a,b,c]),
    (arg(1, T, C), C1 is C + 1, nb_setarg(1, T, C1))), arg(1, T, N).
T = counter(3), N = 3.
```

numbervars/3, number_vars/3

Purpose: `numbervars(+Term, +Start, -End)` binds every free variable of `Term` to `'$VAR'(N)` with consecutive `N` from `Start`; `End` is the next free number. `write/1` and `print/1` render `'$VAR'(0)` as `A`, `'$VAR'(1)` as `B`, and so on. `number_vars/3` is an alias.

```
?- T = f(X, Y, X), numbervars(T, 0, End), print(T), nl.
f(A, B, A)
End = 2.
```

must_be/2

Purpose: `must_be(+Type, @Value)` - succeed if `Value` has the type, otherwise throw the appropriate ISO error (`instantiation_error` for an unbound value when the type requires one, `type_error(Type, Value)` or `domain_error(Type, Value)` otherwise). Types: `atom`, `atomic`, `integer`, `float`, `number`, `callable`, `compound`, `var`, `nonvar`, `ground`, `list`, `boolean`, `positive_integer`, `nonneg`.

```
?- must_be(integer, 3).
true.

?- catch(must_be(positive_integer, 0), error(E, _), true).
E = type_error(positive_integer, 0).
```

simple/1, partial_list/1, rational/1, acyclic_term/1

Purpose: Additional type tests: `simple/1` is true for atomic terms (`atom`, `number` or `string`); `partial_list/1` for a list whose tail is unbound; `rational/1` for a rational number term; `acyclic_term/1` succeeds for terms without cycles.

```
?- simple(abc), partial_list([a, b|_]), acyclic_term(f(X)).
true.

?- partial_list([a, b]).
false.
```

38. Clause inspection and operators

clause/2

Purpose: `clause(+Head, ?Body)` - enumerate the clauses of a user predicate; facts have body `true`. Built-in predicates raise `permission_error(access, private_procedure, PI)`.

```
grandparent(GP, GC) :- parent(GP, P), parent(P, GC).

?- clause(grandparent(A, B), Body).
Body = (parent(A, _P), parent(_P, B)).
```

predicate_property/2

Purpose: `predicate_property(:Head, ?Property)` - properties of a predicate: `built_in`, `dynamic`, `static`, `defined`, `undefined`. With `Property` unbound it enumerates them.

```
?- predicate_property(append(_, _, _), built_in).
true.

?- assertz(fact(1)), predicate_property(fact(_), dynamic).
true.
```

op/3, current_op/3

Purpose: `op(+Priority, +Type, +Name)` defines (priority 1..1200) or removes (priority 0) an operator; `Name` may be a list of atoms. `current_op(?Priority, ?Type, ?Name)` enumerates the active operators. Part III lists the default table.

```
?- op(700, xfx, is_bigger), X =.. [is_bigger, elephant, mouse].
X = elephant is_bigger mouse.

?- findall(P-T, current_op(P, T, mod), L).
L = [400-yfx].
```

char_conversion/2, current_char_conversion/2

Purpose: ISO character conversion table applied while reading terms when the `char_conversion` flag is `true`. `current_char_conversion(?In, ?Out)` enumerates the active mappings.

```
?- char_conversion(a, b), current_char_conversion(a, X).
X = b.
```

code_type/2, char_type/2

Purpose: Classify a character code (`code_type/2`) or a character (`char_type/2`). Supported types: `alpha`, `alnum`, `digit`, `xdigit`, `space`, `white`, `upper`, `lower`, `punct`, `csym`, `csymf`, `end_of_line`, `graph`, `print`, `ascii`, `cntrl`. The parameterised forms of SWI-Prolog (`digit(Weight)`, `upper(Lower)`, `to_lower(L, ...)`) are not supported; use `upcase_atom/2`, `downcase_atom/2` and `char_code/2` for conversions.

```
?- code_type(0'7, digit), char_type('A', upper), char_type(a, lower), char_type(' ', space).
true.
```

39. Debugging and profiling

profile/0, noprofile/0, profile_data/1, reset_profile/0

Purpose: Count predicate calls. `profile/0` starts counting, `noprofile/0` stops it, `profile_data(-Data)` unifies a list of Name/Arity-Count pairs sorted by count, and `reset_profile/0` clears the counters.

Only user-defined predicates are counted.

```
foo(1). foo(2).

?- reset_profile, profile, (foo(_), fail ; true), noprofile, profile_data(D).
D = [foo/1-1].
```

spy/1, nospy/1, spying/1, debugging/0, leash/1

Purpose: Spy points mark predicates whose ports are shown by the tracer. `spy(+PI)` and `nospy(+PI)` add/remove a spy point (Name/Arity), `spying(?PI)` enumerates them, `debugging/0` prints the trace state and spy points, and `leash(+Ports)` selects the ports (call, exit, redo, fail, all, none) at which an interactive tracer stops.

```
?- spy(foo/2), spying(P).
% Spy point set on foo/2
P = foo/2.

?- debugging.
Tracing is OFF
Spy points:
    foo/2
true.
```

trace/0, notrace/0

Purpose: Enable / disable four-port tracing (Call, Exit, Redo, Fail) lines are written to the current output for every user predicate and traced built-in).

```
?- trace, member(X, [1, 2]), X > 1, notrace.
% Tracing enabled
Call: (0) member(X,[1,2])
Exit: (0) member(1,[1,2])
Redo: (0) member(2,[1,2])
Exit: (0) member(2,[1,2])
    Call: (1) notrace
% Tracing disabled
X = 2.
```

cut/0

Purpose: A callable synonym of `!` kept for programs that write the cut as a plain atom goal through `call/1`; inside a clause body always use `!`.

40. System and memory

atom_gc/0, atom_table_size/1

Purpose: `atom_gc/0` releases interned atoms that are no longer referenced; `atom_table_size(-N)` reports the number of interned atoms.

```
?- atom_gc, atom_table_size(N).
N = 75.
```

shell2/2, sleep/1

Purpose: `shell2(+Command, -ExitCode)` runs a shell command and returns its exit status (`shell/1,2` are the other forms); `sleep(+Seconds)` pauses the current thread (fractions allowed).

```
?- shell2('ls /nonexistent', Code).
Code = 2.

?- sleep(0.5).
true.
```

working_directory/2, file_modified/2, delete_directory/1

Purpose: `working_directory(-Old, +New)` reads and changes the process working directory (`working_directory(D, D)` only reads it); `file_modified(+Path, -Millis)` gives the modification time in epoch milliseconds; `delete_directory(+Path)` removes an empty directory.

```
?- working_directory(D, D).
D = '/home/user/project'.

?- open('note.txt', write, W), close(W), file_modified('note.txt', T).
T = 1787644907965.
```

format_time/3, parse_time/3

Purpose: `format_time(+Pattern, +Timestamp, -Text)` formats epoch milliseconds (or a datetime atom) with a Java `DateTimeFormatter` pattern; `parse_time(+Pattern, +Text, -Millis)` is the inverse.

```
?- parse_time('yyyy-MM-dd HH:mm:ss', '2026-03-20 14:30:00', Ms),
   format_time('dd/MM/yyyy', Ms, Day).
Ms = 1774013400000, Day = '20/03/2026'.
```

The millisecond value depends on the local time zone.

41. JDBC database access

The JDBC predicates connect a Prolog program to any database with a JDBC driver on the class path. Connections, prepared statements and callable statements are represented by handle atoms (`'$jdbc_conn_1', ...`). All predicates raise `existence_error` for an unknown handle and a `jdbc_error(Message)` ball for SQL failures. They are removed by `enableSafeMode()`.

`jdbc_driver_load/1`

Purpose: Load a driver class by name (needed only by drivers that do not self-register).

```
?- jdbc_driver_load('org.h2.Driver').
true.
```

`jdbc_connect/2`, `jdbc_connect/4`, `jdbc_disconnect/1`

Purpose: Open a connection from a JDBC URL, optionally with user and password; close it with `jdbc_disconnect/1`.

```
?- jdbc_connect('jdbc:h2:mem:testdb', C).
C = '$jdbc_conn_1'.

?- jdbc_connect('jdbc:postgresql://localhost/mydb', 'user', 'pass', C), jdbc_disconnect(C).
```

`jdbc_query/3`, `jdbc_execute_update/3`

Purpose: `jdbc_query(+Conn, +SQL, -Rows)` runs a SELECT and returns a list of row(`Col1, ...`) terms; `jdbc_execute_update(+Conn, +SQL, -Count)` runs INSERT/UPDATE/DELETE/DDl and returns the number of affected rows.

```
?- jdbc_execute_update(C, 'CREATE TABLE users(id INT, name VARCHAR(50))', _),
   jdbc_execute_update(C, 'INSERT INTO users VALUES (1, 'Alice')', N),
   jdbc_query(C, 'SELECT id, name FROM users', Rows).
N = 1, Rows = [row(1, 'Alice')].
```

`jdbc_prepare/3`, `jdbc_set_param/3`, `jdbc_set_params/2`, `jdbc_execute_prepared_query/2`, `jdbc_execute_prepared_update/2`, `jdbc_close_statement/1`

Purpose: Prepared statements with ? placeholders: prepare, bind parameters (1-based, or all at once from a list; use the atom `null` for SQL NULL), execute as a query or an update, close.

```
insert_user(C, Id, Name) :-
    jdbc_prepare(C, 'INSERT INTO users(id, name) VALUES (?, ?)', S),
    jdbc_set_params(S, [Id, Name]),
    jdbc_execute_prepared_update(S, 1),
    jdbc_close_statement(S).

older_than(C, Age, Rows) :-
    jdbc_prepare(C, 'SELECT name FROM users WHERE age > ?', S),
    jdbc_set_param(S, 1, Age),
    jdbc_execute_prepared_query(S, Rows),
    jdbc_close_statement(S).
```

jdbc_set_autocommit/2, jdbc_commit/1, jdbc_rollback/1

Purpose: Transaction control. Disable autocommit, run statements, then commit or roll back.

```
transfer(C, From, To, Amount) :-
    jdbc_set_autocommit(C, false),
    catch(( debit(C, From, Amount), credit(C, To, Amount), jdbc_commit(C) ),
        E,
        ( jdbc_rollback(C), throw(E) )),
    jdbc_set_autocommit(C, true).
```

jdbc_tables/2, jdbc_columns/3

Purpose: Metadata: the list of table names, and the columns of a table as column(Name, TypeName, Size) terms.

```
?- jdbc_tables(C, Ts), jdbc_columns(C, 'USERS', Cols).
Ts = ['USERS'], Cols = [column('ID', 'INTEGER', 10), column('NAME', 'VARCHAR', 50)].
```

**jdbc_prepare_call/3, jdbc_call_set_param/3, jdbc_call_register_out/3,
jdbc_call_execute/1, jdbc_call_get_result/3, jdbc_call_get_resultset/2**

Purpose: Stored procedures through callable statements: prepare {call proc(?, ?)}, set IN parameters, register OUT parameters with a type (integer, bigint, double, decimal, varchar, boolean, date, timestamp), execute, then read OUT values or the returned result set as row/N terms.

```
user_count(C, Table, Count) :-
    jdbc_prepare_call(C, '{call get_user_count(?, ?)}', S),
    jdbc_call_set_param(S, 1, Table),
    jdbc_call_register_out(S, 2, integer),
    jdbc_call_execute(S),
    jdbc_call_get_result(S, 2, Count),
    jdbc_close_statement(S).
```

**jdbc_set_clob/3, jdbc_get_clob/3, jdbc_set_blob/3, jdbc_set_blob_bytes/3,
jdbc_get_blob_bytes/3, jdbc_get_blob_to_file/3**

Purpose: Large objects: set a CLOB parameter from text, a BLOB parameter from a file path or a byte list; read a CLOB as an atom, a BLOB as a byte list, or save a BLOB straight to a file. The get variants take a SELECT returning a single LOB column.

```
store_doc(C, Id, Text) :-
    jdbc_prepare(C, 'INSERT INTO docs(id, body) VALUES (?, ?)', S),
    jdbc_set_param(S, 1, Id), jdbc_set_clob(S, 2, Text),
    jdbc_execute_prepared_update(S, _), jdbc_close_statement(S).

?- jdbc_get_clob(C, 'SELECT body FROM docs WHERE id = 1', Text).
Text = 'A long document ...'.

?- jdbc_get_blob_bytes(C, 'SELECT data FROM images WHERE id = 1', Bytes).
Bytes = [137, 80, 78, 71|...].
```

42. TCP, UDP and DNS

Socket predicates exchange UTF-8 text; handles are atoms such as '\$socket_1'. `tcp_accept/2`, `tcp_receive/3` and `udp_receive/4` block until data arrives; a closed peer yields the atom `end_of_stream`. These predicates are removed by `enableSafeMode()`.

tcp_connect/3, tcp_send/2, tcp_receive/3, tcp_close/1

Purpose: TCP client: connect to `Host:Port`, send a string, receive up to `MaxBytes` bytes, close.

```
echo_client(Host, Port, Msg, Reply) :-
    tcp_connect(Host, Port, S),
    tcp_send(S, Msg),
    tcp_receive(S, Reply, 4096),
    tcp_close(S).

?- echo_client(localhost, 7777, 'ping', R).
R = pong.
```

tcp_server_socket/2, tcp_accept/2

Purpose: TCP server: bind a listening socket to a port, then accept client connections one at a time.

```
serve_once(Port) :-
    tcp_server_socket(Port, SS),
    tcp_accept(SS, Client),
    tcp_receive(Client, Data, 4096),
    tcp_send(Client, Data),           % echo
    tcp_close(Client),
    tcp_close(SS).
```

udp_socket/2, udp_send/4, udp_receive/4, udp_close/1

Purpose: UDP datagrams: `udp_socket(+Port, -Sock)` binds a socket (port 0 = any free port), `udp_send(+Sock, +Host, +Port, +Data)`, `udp_receive(+Sock, -Data, -From, +MaxBytes)` with `From = from(IP, Port)`.

```
udp_ping(Host, Port, Reply) :-
    udp_socket(0, S),
    udp_send(S, Host, Port, ping),
    udp_receive(S, Reply, from(_, _), 1024),
    udp_close(S).
```

hostname_address/2, hostname/1

Purpose: `hostname_address(+Name, -IP)` resolves a host name; `hostname(-Name)` returns the local host name.

```
?- hostname_address(localhost, IP).
IP = '127.0.0.1'.
```

http_request/4, http_post/4

Purpose: Minimal HTTP client: `http_request(+Method, +URL, -Status, -Body)` with `Method` one of `get`, `head`, `delete`, `put`, `post`; `http_post(+URL, +RequestBody, -Status, -Body)` sends a form-encoded body. Both use 30-second timeouts. The richer `http_get/3`, `http_open/3` and the JSON-aware `http_client_*` predicates are described in Chapter 26.

```
?- http_request(get, 'http://httpbin.org/get', Status, Body), Status == 200.  
Status = 200, Body = '{"args":{}, ...}'.
```

43. DCG support predicates

dcg_translate_rule/2, dcg_body//2

Purpose: `dcg_translate_rule(+Rule, -Clause)` returns the ordinary clause `a --> rule` is translated to, exposing the translator used by `consult/1` and `assertz/1`. `dcg_body//2` is a reserved grammar non-terminal (two arguments plus the difference list) whose current implementation simply succeeds; it is kept for source compatibility.

```
?- dcg_translate_rule((a --> [x], b), C).  
C = (a(S0, S) :- S0 = [x|S1], b(S1, S)).
```

Appendix A - Prolog flags

`current_prolog_flag(?Flag, ?Value)` reads and `set_prolog_flag(+Flag, +Value)` sets the flags below. Flags marked *read-only* raise `permission_error(modify, flag, Flag)` when set.

Flag	Values	Notes
<code>bounded</code>	<code>false</code>	read-only; integers are unbounded
<code>max_integer, min_integer</code>	<code>integer</code>	read-only; limits of the fast 64-bit representation
<code>integer_rounding_function</code>	<code>toward_zero</code>	read-only
<code>max_arity</code>	<code>unbounded</code>	read-only
<code>double_quotes</code>	<code>codes, chars, atom, string</code>	how "text" is read (default string)
<code>unknown</code>	<code>error, fail, warning</code>	what a call to an undefined procedure does (default error)
<code>occurs_check</code>	<code>true, false</code>	occurs check in ordinary unification (default false)
<code>char_conversion</code>	<code>true, false</code>	apply the <code>char_conversion/2</code> table when reading
<code>debug</code>	<code>on, off</code>	debug mode
<code>syntax_errors</code>	<code>error, fail, quiet</code>	behaviour of <code>read/1</code> on a syntax error
<code>character_escapes</code>	<code>true, false</code>	escape processing in quoted atoms
<code>strict_iso</code>	<code>true, false</code>	reject non-ISO extensions in <code>read/1</code>
<code>dialect, prolog_version, version, version_data</code>	<code>atom / term</code>	read-only identification (jprolog)
<code>encoding</code>	<code>atom</code>	default text encoding (utf8)
<code>argv</code>	<code>list</code>	command-line arguments
<code>gc, optimize, traditional, write_strings, trace, initialization, stack_limit, toplevel_print_options</code>	see <code>current_prolog_flag/2</code>	SWI-compatible flags kept for portability

A flag takes effect for the text read after it is set (a query is parsed as a whole before it runs):

```
?- set_prolog_flag(double_quotes, codes).
true.

?- X = "ab".
X = [97, 98].

?- set_prolog_flag(unknown, fail), no_such_pred.
false.
```

Appendix B - Engine configuration and embedding

Subsystem selection (system properties). The clean-room implementations are the defaults. There is **one resolution engine** (`core.engine.v4`, since 4.0.0); the parser, the CLP(FD) solver and the DCG translator each keep an older implementation that a property set before the engine starts can select:

Property	Selects
<code>-Djprolog.parser=legacy</code>	the recursive-descent parser instead of the operator-precedence parser
<code>-Djprolog.clpfd=legacy</code>	the first CLP(FD) implementation (read once, when the engine is created)
<code>-Djprolog.dcg=legacy</code>	the first DCG translator

Only the literal value `legacy` selects a fallback. `-Djprolog.engine` no longer selects anything: the recursive engine it could name as `legacy` was deleted in 4.0.0 and the iterative 3.x machine it could name as `v2` in **4.1.0**, so any value logs a warning and runs the v4 core. The parser and DCG switches are also available at runtime as `Prolog.setUseV2Parser/Dcg(boolean)`; the CLP(FD) choice must be made before constructing the engine (or upgraded with `prolog.enableV2Clpfd()`).

Java API summary (`it.denzosoft.jprolog.core.engine.Prolog`):

Method	Purpose
<code>consult(String program)</code>	load clauses and run directives (throws on the first error)
<code>consultWithDiagnostics(program, fileName)</code>	per-clause compilation returning every error with its line
<code>solve(String query)</code>	all solutions as <code>List<Map<String, Term>></code>
<code>solveStream(query, sink)</code>	lazy enumeration; the sink returns <code>false</code> to stop
<code>compileFile("p.pl"), consultCompiled("p.jpc"), consultSmart("p.pl")</code>	binary <code>.jpc</code> compilation and cached loading
<code>asserta/assertz/retract(String clause)</code>	modify the database from Java
<code>nbSetval/nbGetval/nbDelete</code>	global variables from Java
<code>enableSafeMode()</code>	sandbox: remove every OS, FFI, filesystem, network, HTTP, JDBC, persistence and threading built-in (irreversible for the instance)
<code>setInferenceBudget(long steps)</code>	abort a query with <code>InferenceLimitException</code> after the given number of resolution steps (0 = unlimited); enforced inside meta-calls and worker threads too
<code>getFlags(), setTracing(boolean), isTracing()</code>	per-engine ISO flag store and trace/0 state (each Prolog instance is isolated)
<code>getPredicateIndicatorAtLine(int line)</code>	map a source line to its clause (IDE breakpoints)
<code>getEngineContext()</code>	the durable per-engine context: <code>setDebugController/getDebugController</code> (IDE debugger wiring) and the running query's <code>ResourceGuard</code>
<code>getEngineState(), getStreams(), getOps()</code>	the engine's own stream table and operator store
<code>residualGoals(solution)</code>	the constraints still attached to an answer's variables

Resource control. A runaway query stops with `InferenceLimitException` when the budget is exceeded and with `QueryCancelledException` when the solver thread is interrupted (the IDE Stop button). Both are plain Java runtime exceptions that a Prolog catch/3 cannot intercept, so untrusted programs cannot defeat them. Deep terms or deeply nested input are reported as the catchable ISO errors `resource_error(stack_overflow)` and `resource_error(parser_nesting)`.

Appendix C - ISO error terms

Built-in predicates signal errors by throwing `error(Formal, Context)`. `Context` is `context(Name/Arity, Message)` or a variable. The `Formal` terms used by JProlog:

Formal term	Raised when
<code>instantiation_error</code>	an argument that must be bound is a variable
<code>type_error(Type, Culprit)</code>	an argument has the wrong type (integer, atom, callable, list, evaluable, ...)
<code>domain_error(Domain, Culprit)</code>	the type is right but the value is outside the domain (<code>not_less_than_zero</code> , <code>operator_priority</code> , ...)
<code>existence_error(Kind, What)</code>	a procedure, stream, file, variable or source does not exist
<code>permission_error(Action, Type, Culprit)</code>	e.g. modifying a static procedure, repositioning a stream, accessing a private procedure
<code>representation_error(Flag)</code>	a value cannot be represented (<code>character_code</code> , <code>max_arity</code> , <code>cyclic_term</code>)
<code>evaluation_error(Error)</code>	arithmetic: <code>zero_divisor</code> , <code>undefined</code> , <code>float_overflow</code> , <code>int_overflow</code> , <code>negative_shift</code>
<code>resource_error(Resource)</code>	<code>memory</code> , <code>stack_overflow</code> , <code>parser_nesting</code>
<code>syntax_error(Message)</code>	invalid text in <code>read/1</code> , <code>atom_to_term/3</code> , <code>number_codes/2</code> , ...
<code>system_error</code>	an internal failure inside a built-in

JProlog follows the strict ISO reading: `atom_length(123456, L)` raises `type_error(atom, 123456)` instead of counting the digits as SWI-Prolog does.

```
?- catch(atom_length(123456, L), error(E, _), true).
E = type_error(atom, 123456).

?- catch(atom_length(X, L), error(Err, _), true).
Err = instantiation_error.

?- catch(foo(1), error(existence_error(procedure, PI), _), true).
PI = foo/1.
```

Appendix D - Standard order of terms and text conversion summary

Standard order (used by @<, compare/3, sort/2, msort/2, setof/3, keysort/2): Var < Number < Atom < String < Compound. Numbers are compared by value, and when an integer and a float are equal the float comes first. Compound terms are ordered by arity, then by functor name, then by arguments left to right.

Text conversions at a glance:

From \ To	atom	string	codes	chars	number
atom	-	atom_string/2	atom_codes/2	atom_chars/2	atom_number/2
string	string_to_atom/2	-	string_codes/2	string_chars/2	number_string/2
number	atom_number/2	number_string/2	number_codes/2	number_chars/2	-
any term	term_to_atom/2	with_output_to/2	-	-	-

Appendix E - Libraries not loaded by default

The source tree also documents experimental libraries that are **not** registered in the default engine and are therefore outside the scope of this manual: expert system (`es_*`), NLP, inference engine, AI planner, fuzzy logic, Bayesian networks, genetic algorithms, neural networks, optimisation, simulation, workflow, CLP(R) (`clpr_*`), knowledge graphs, parsing/DSL, Datalog, RDF, model checking, CHR (`chr_*`), BDI agents (`agent_*`), ASP (`asp_*`), explainable AI, type inference, theorem proving, symbolic mathematics, meta-interpretation, event calculus (`ec_*`), probabilistic logic, SAT solving, game playing, term rewriting and description logic (`dl_*`). See `docs/references/BUILTIN_PREDICATES_REFERENCE.md` sections 29-60 for their descriptions and `docs/guides/guide-extension.md` for how to register an extension with `Prolog.registerBuiltinPredicate(name, builtin)`.

Index of predicates and operators

Every name/arity heading of this manual with the page where it is described.

abolish/1	65	copy_term/2	22	file_base_name/2	100
abolish_all_tables/0	121	copy_term/3	148	file_directory_name/2	101
abolish_table/1	121	cpu_count/1	103	file_exists/1	98
absolute_file_name/2	100	crypto_aes_decrypt/4	91	file_extension/2	100
acyclic_term/1	19	crypto_aes_encrypt/4	91	file_modified/2	155
aggregate_all/3	51	crypto_hash/3	89	file_size/2	99
all_different/1	117	crypto_hash_password/2	91	findall/3	48
all_distinct/1	117	crypto_random_int/3	91	findall/4	49
append/3	27	crypto_verify_password/2	92	first_solution/3	135
arg/3	20	csv_parse/2	112	flatten/2	33
assert/1	63	csv_read_file/2	112	float/1	16
asserta/1	63	csv_serialize/2	112	flush_output/0	143
assertz/1	63	csv_write_file/2	112	flush_output/1	143
at_end_of_stream/0	58	current_char_conversion/2	61	foldl/4	35
atom/1	15	current_input/1	143	foldl/5	35
atom_chars/2	70	current_module/1	138	foldl/6	35
atom_codes/2	70	current_op/3	153	foldl/7	35
atom_concat/3	68	current_output/1	143	forall/2	44
atom_gc/0	155	current_predicate/1	66	format/2	55
atom_length/2	68	current_prolog_flag/2	86	format/3	55
atom_number/2	71	current_stream/3	60	format_date/3	96
atom_string/2	141	current_table/2	122	format_time/3	155
atom_table_size/1	155	cut/0	154	free_memory/1	103
atom_to_number/2	74	cyclic_term/1	19	freeze/2	147
atom_to_term/3	26	date_add/4	95	frozen/2	148
atomic/1	16	date_diff/4	96	functor/3	20
atomic_list_concat/2	73	date_parts/4	96	get_attr/3	147
atomic_list_concat/3	73	day_of_week/2	96	get_byte/1	57
attvar/1	147	db_batch_assert/1	128	get_char/1	57
b_getval/2	146	db_clear/0	128	get_code/1	57
b_setval/2	146	db_export_json/1	127	get_time/1	95
bagof/3	49	db_import_json/1	127	getenv/2	102
base64_decode/2	90	db_load/1	126	graph_components/2	130
base64_encode/2	90	db_restore/1	127	graph_connected/2	129
between/3	38	db_save/1	126	graph_degree/3	130
call/1	47	db_save_predicate/2	126	graph_edges/2	129
call_cleanup/2	46	db_snapshot/1	127	graph_has_cycle/1	131
call_dcg/3	81	db_sync/0	128	graph_neighbors/3	130
callable/1	17	db_transaction/1	128	graph_path/4	129
catch/3	83	dcg_translate_rule/2	81	graph_reachable/3	131
char_code/2	75	debugging/0	154	graph_scc/2	131
char_conversion/2	61	del_attr/2	147	graph_vertices/2	129
char_type/2	76	delete/3	34	ground/1	18
character_count/2	60	delete_directory/1	155	halt/0	84
clause/2	153	delete_file/1	98	halt/1	84
close/1	61	dif/2	147	hmac/4	90
close/2	143	directory_exists/1	98	hostname/1	103
code_type/2	153	directory_files/2	100	hostname_address/2	158
compare/3	24	discontiguous/1	9	http_client_get/2	124
compound/1	17	div/2	11	http_client_post/3	124
concurrent/3	135	downcase_atom/2	77	http_get_request/2	123
concurrent_and/2	135	dynamic/1	9	http_handler/3	123
concurrent_maplist/2	135	enhanced_phrase/2	79	http_open/3	124
concurrent_maplist/3	135	enhanced_phrase/3	79	http_post/4	158
concurrent_maplist/4	135	exclude/3	34	http_reply/4	124
concurrent_or/2	136	fd_dom/2	119	http_reply_json/3	124
copy_file/2	99	fd_size/2	119	http_request/4	158

http_server/2	123	labeling/2	118	partial_list/1	152
http_stop/1	123	last/2	33	partition/4	35
ignore/1	45	leash/1	88	partition/5	35
in/2	12	length/2	28	peek_byte/1	58
include/3	34	line_count/2	60	peek_char/1	57
indomain/1	118	line_position/2	60	peek_code/1	57
ins/2	12	listing/0	66	permutation/2	30
integer/1	15	listing/1	66	persist/1	126
intersection/3	34	log_debug/1	114	phrase/2	78
is/2	10	log_error/1	114	phrase/3	78
is_list/1	18	log_info/1	114	phrase_with_options/4	80
java_array_get/3	133	log_level/1	115	pid/1	103
java_array_length/2	133	log_to_file/1	115	plus/3	40
java_array_new/3	133	log_warning/1	114	portray_clause/1	59
java_array_set/3	133	make_directory/1	99	predicate_property/2	138
java_call/4	132	make_directory_path/1	99	predsort/3	149
java_class/2	133	maplist/2	34	print/1	54
java_from_term/2	134	maplist/3	34	print_message/2	59
java_gc/0	134	maplist/4	34	profile/0	154
java_get_field/3	132	maplist/5	34	profile_data/1	154
java_instanceof/2	133	max_list/2	33	proper_list/1	19
java_new/3	132	md5_hash/2	89	put_attr/3	147
java_release_ref/1	134	member/2	28	put_byte/1	57
java_set_field/3	132	message_queue_create/1	110	put_char/1	57
java_to_term/2	133	meta_predicate/1	9	put_code/1	151
jdbc_call_execute/1	157	min_list/2	33	random_token/2	91
jdbc_call_get_result/3	157	minimum_spanning_tree/2	130	rational/1	152
jdbc_call_get_resultset/2	157	mod/2	11	rdiv/2	11
jdbc_call_register_out/3	157	module_transparent/1	9	re_escape/2	106
jdbc_call_set_param/3	157	msort/2	31	re_findall/3	106
jdbc_close_statement/1	156	multifile/1	9	re_match/2	105
jdbc_columns/3	157	must_be/2	152	re_matchsub/3	105
jdbc_commit/1	157	nb_current/2	146	re_replace/4	105
jdbc_connect/2	156	nb_delete/1	146	re_split/3	106
jdbc_connect/4	156	nb_getval/2	146	read/1	55
jdbc_disconnect/1	156	nb_setarg/3	151	read/2	55
jdbc_driver_load/1	156	nb_setval/2	146	read_file_to_atom/2	99
jdbc_execute_prepared_qu..	156	nl/0	53	read_term/2	56
jdbc_execute_prepared_up..	156	nonvar/1	14	rem/2	11
jdbc_execute_update/3	156	noprofile/0	154	rename_file/2	98
jdbc_get_blob_bytes/3	157	nospy/1	154	repeat/0	44
jdbc_get_blob_to_file/3	157	notrace/0	154	reset_profile/0	154
jdbc_get_clob/3	157	now/1	95	retract/1	64
jdbc_prepare/3	156	nth0/3	30	retractall/1	64
jdbc_prepare_call/3	157	nth1/3	30	reverse/2	29
jdbc_query/3	156	number/1	15	seek/4	61
jdbc_rollback/1	157	number_chars/2	71	select/3	31
jdbc_set_autocommit/2	157	number_codes/2	71	set_input/1	62
jdbc_set_blob/3	157	number_string/2	141	set_output/1	62
jdbc_set_blob_bytes/3	157	number_to_atom/2	74	set_prolog_flag/2	86
jdbc_set_clob/3	157	number_vars/3	151	set_stream/2	60
jdbc_set_param/3	156	numbervars/3	26	set_stream_position/2	61
jdbc_set_params/2	156	numlist/3	33	setarg/3	151
jdbc_tables/2	157	once/1	43	setenv/2	102
json_get/3	93	op/3	153	setof/3	50
json_keys/2	94	open/3	61	setup_call_cleanup/3	45
json_member/3	94	os_name/1	102	sha256_hash/2	89
json_parse/2	93	pairs_keys/2	150	sha512_hash/2	89
json_serialize/2	93	pairs_keys_values/3	150	shell/1	103
json_set/4	94	pairs_values/2	150	shell/2	104
keysort/2	33	parse_date/3	97	shell2/2	155
label/1	118	parse_time/3	155	shell_output/3	104

shortest_path/4	129	unify_with_occurs_check/2	151
simple/1	152	union/3	34
sleep/1	155	unpersist/1	127
sort/2	31	upcase_atom/2	77
sort/4	31	url_decode/2	125
split_string/4	72	url_encode/2	124
spy/1	154	uuid/1	90
spying/1	154	var/1	14
statistics/2	87	when/2	147
stream_position/2	61	with_output_to/2	62
stream_position_data/3	60	working_directory/2	155
stream_property/2	60	write/1	53
string/1	16	write_atom_to_file/2	99
string_chars/2	140	write_canonical/1	58
string_code/3	74	write_term/2	58
string_codes/2	140	writeln/1	53
string_concat/3	140	writeq/1	54
string_length/2	140	xml_parse/2	107
string_to_atom/2	73	xml_serialize/2	107
sub_atom/5	69	xor/2	11
sub_string/5	141	xpath/3	107
subsumes_term/2	25	!/0	9
subtract/3	34	#</2	12
succ/2	39	#=/2	12
sum_list/2	33	#=</2	12
sumlist/2	33	#>/2	12
system_time/1	102	#>=/2	12
tab/1	62	#\=/2	12
table/1	9	**/2	11
tcp_accept/2	158	*->/2	9
tcp_close/1	158	*/2	11
tcp_connect/3	158	+/1	11
tcp_receive/3	158	+/2	11
tcp_send/2	158	,/2	9
tcp_server_socket/2	158	-->/2	8
term_attvars/2	148	-/1	8
term_string/2	26	-/2	8
term_to_atom/2	25	->/2	9
term_variables/2	24	../2	12
thread_create/2	108	///2	11
thread_create/3	108	//2	11
thread_detach/1	109	/\2	11
thread_get_message/1	110	</2	10
thread_get_message/2	111	<</2	11
thread_is_alive/1	110	=../2	10
thread_join/2	109	=/2	10
thread_peek_message/2	111	=</2	10
thread_self/1	109	==/2	10
thread_send_message/2	110	=\=/2	10
thread_sleep/1	109	>/2	10
throw/1	83	>=/2	10
time_parts/4	96	>>/2	11
to_codes/2	141	@</2	10
today/1	95	@=</2	10
topological_sort/2	130	@>/2	10
total_memory/1	103	@>=/2	10
trace/0	154	\+/1	9
udp_close/1	158	\//2	11
udp_receive/4	158	\/1	11
udp_send/4	158	\=/2	10
udp_socket/2	158	\==/2	10
unifiable/3	148	^/2	11